

The Lambdust Language Specification

Version 0.2.0

Lambdust Development Team

August 19, 2025

Abstract

This document specifies the Lambdust programming language, a conservative extension of R7RS-small Scheme that adds gradual dependent typing, algebraic effects with handlers, actor-based concurrency, memory-safe foreign function interface, and proof-carrying code capabilities. The language maintains complete R7RS compatibility while providing advanced features for high-assurance software development.

2 Lambdust Language Specification

Copyright © 2024 Lambdust Development Team.
This document is released under the Creative Commons Attribution 4.0 International License.

1	Introduction	5
1.1	Design Philosophy	5
1.2	Language Features Overview	5
1.3	Implementation Status	7
1.4	Relationship to Other Languages	8
1.5	Specification Structure	8
1.6	Conventions	8
1.7	Acknowledgments	8
2	Lexical Structure	8
2.1	Lexical Elements	8
2.2	Character Set	9
2.3	Comments	9
2.4	Identifiers	9
2.5	Keywords	9
2.6	Numbers	10
2.7	Strings	10
2.8	Characters	10
2.9	Booleans	10
2.10	Delimiters	10
2.11	Quote and Unquote	11
2.12	Lambdust Extensions	11
2.13	Token Classification	11
2.14	Lexical Precedence	13
2.15	Error Handling	13
2.16	Implementation Notes	13
3	Basic Syntax and Semantics	13
3.1	Syntactic Categories	13
3.2	Constants and Literals	13
3.3	Variables	14
3.4	Procedure Calls	14
3.5	Lambda Expressions	14
3.6	Conditionals	14
3.7	Assignment	14
3.8	Derived Expressions	14
3.9	Conditionals	14
3.10	Boolean Operations	15
3.11	Binding Forms	15
3.12	Sequencing	15
3.13	Iteration	15

3.14	Definitions	15	5.7	Universe Hierarchy	27
3.15	Quoting and Quasiquoting	15	5.8	Proof Terms and Tactics	27
3.16	Quote	16	5.9	Basic Proof Construction	27
3.17	Quasiquote	16	5.10	Proof Tactics	28
3.18	Type Expressions	16	5.11	Program Extraction	28
3.19	Effect Expressions	16	5.12	Type-Level Programming	28
3.20	Concurrency Expressions	16	5.13	Totality and Termination	29
3.21	Foreign Function Interface	17	5.14	Structural Recursion	29
3.22	Syntax Extensions	17	5.15	Well-Founded Recursion	29
3.23	Evaluation Order	17	5.16	Coinduction and Corecursion	29
3.24	Tail Calls	17	5.17	Integration with Gradual Typing	29
3.25	Continuations	18	5.18	Standard Dependent Types	30
3.26	Multiple Values	18	5.19	Error Messages and Inference	30
3.27	Compatibility Guarantees	18	6	Effect System	30
4	Gradual Type System	18	6.1	Effects Overview	30
4.1	Type System Overview	19	6.2	Effect Declaration	31
4.2	Type Syntax	19	6.3	Effect Performance	31
4.3	Base Types	19	6.4	Effect Handlers	31
4.4	Refined Numeric Types	20	6.5	Effect Types	32
4.5	Compound Types	20	6.6	Built-in Effects	32
4.6	Union and Intersection Types	21	6.7	State Effect	32
4.7	Polymorphic Types	22	6.8	Exception Effect	33
4.8	Type Annotations	22	6.9	I/O Effect	33
4.9	Type Inference	23	6.10	Effect Composition	33
4.10	Gradual Typing	23	6.11	Effect Scoping	33
4.11	Subtyping	24	6.12	Monadic Effects	34
4.12	Type Classes	24	6.13	Effect Inference	34
4.13	Type-Level Computation	24	6.14	Effect Safety	35
4.14	Error Messages	24	6.15	Performance Considerations	35
4.15	Optimization	24	6.16	R7RS Integration	35
4.16	Migration Strategies	25	7	Concurrency System	36
4.17	Implementation Notes	25	7.1	Concurrency Overview	36
5	Dependent Type System	25	7.2	Actor Model	36
5.1	Dependent Types Overview	25	7.3	Actor Creation	36
5.2	Pi Types (Dependent Functions)	26	7.4	Message Passing	36
5.3	Sigma Types (Dependent Pairs)	26	7.5	Actor Hierarchies and Supervision	37
5.4	Refinement Types	26	7.6	Futures and Promises	37
5.5	Indexed Types	26	7.7	Future Creation	37
5.6	Equality Types	27	7.8	Future Composition	38

4 Lambdust Language Specification

7.9	Channels	38	10.1	Library Organization	54
7.10	Software Transactional Memory	39	10.2	R7RS-small Compatibility	54
7.11	Transactional Variables	39	10.3	Extended Base Library	54
7.12	Transaction Composition	39	10.4	Type System Integration	55
7.13	Parallel Processing	39	10.5	Effects Library	56
7.14	Distributed Computing	40	10.6	Concurrency Library	56
7.15	Concurrency Types	40	10.7	Collections Library	56
7.16	Performance and Optimization	41	10.8	Text Processing	57
7.17	Debugging and Monitoring	41	10.9	I/O Library	57
8	Foreign Function Interface	42	10.10	FFI Library	58
8.1	FFI Overview	42	10.11	Metaprogramming Library	58
8.2	Foreign Declarations	42	10.12	SRFI Support	59
8.3	C Type System	42	10.13	Testing Library	59
8.4	Primitive Types	42	10.14	Debugging Library	60
8.5	Compound Types	44	11	Formal Denotational Semantics	60
8.6	Function Calls	44	12	Examples	70
8.7	Memory Management	44	12.1	Web Server with Effects	70
8.8	Automatic Memory Management	44	12.2	Type-Safe Database Access	72
8.9	Manual Memory Management	45	12.3	Distributed Actor System	74
8.10	Capability System	45	12.4	Compiler with Dependent Types	78
8.11	Capability Declaration	45	12.5	Proof-Carrying Code	82
8.12	Capability Usage	46	A	R7RS Compatibility Reference	86
8.13	Error Handling	46	B	SRFI Support Matrix	86
8.14	Threading and Concurrency	47	C	Migration Guide from R7RS	88
8.15	Platform Integration	47	D	Implementation Notes	88
8.16	Performance Considerations	48		References	88
8.17	Security Considerations	48			
9	Macro System Extensions	48			
9.1	Macro System Overview	48			
9.2	R7RS Macro Support	49			
9.3	Procedural Macros	49			
9.4	Type-Aware Macros	50			
9.5	Compile-Time Computation	50			
9.6	Advanced Pattern Matching	51			
9.7	Macro Debugging	51			
9.8	Hygienic Capture	52			
9.9	Macro Libraries	52			
9.10	Macro Performance	53			
10	Standard Library	54			

1. Introduction

Lambdust is a programming language that extends R7RS-small Scheme with advanced features for modern software development while maintaining complete backward compatibility. The language addresses three critical challenges in contemporary programming: **correctness** through dependent types and formal verification, **composability** through algebraic effects that eliminate callback hell and provide structured error handling, and **scalability** through fault-tolerant actor-based concurrency.

Rather than forcing developers to choose between simplicity and power, Lambdust enables gradual adoption of advanced features. A program can start as pure R7RS Scheme and incrementally adopt type annotations, effect tracking, and formal proofs where they provide the most value. This design philosophy ensures that existing Scheme code continues to work unchanged while providing a clear upgrade path to more sophisticated programming techniques.

The language integrates gradual dependent typing, algebraic effects with handlers, actor-based concurrency, memory-safe foreign function interface, and proof-carrying code capabilities into a unified, theoretically sound framework. Each feature is designed to work seamlessly with the others: types can track effects, actors can be statically verified, and proofs can constrain concurrent behavior.

1.1. Design Philosophy

Lambdust follows several key design principles:

- **Conservative Extension:** Every valid R7RS-small program is a valid Lambdust program with identical semantics.
- **Gradual Enhancement:** Advanced features can be adopted incrementally without breaking existing code.
- **Mathematical Foundation:** All language features have precise mathematical semantics suitable for formal verification.
- **Practical Power:** Advanced type systems and effect tracking enable catching more errors at compile time while maintaining expressiveness.
- **Implementation Flexibility:** The specification supports multiple implementation strategies from interpreters to native code compilation.

1.2. Language Features Overview

1.2.1. R7RS-small Compatibility

Lambdust provides complete R7RS-small compatibility:

- All standard special forms: `lambda`, `if`, `set!`, `quote`
- All derived expressions: `cond`, `case`, `and`, `or`, `let`, `letrec`
- Complete numeric tower with exact and inexact arithmetic
- Full string and character support with Unicode
- Standard I/O procedures and file operations
- Proper tail recursion and continuation semantics via `call/cc`
- Hygienic macro system with `syntax-rules`

R7RS Compatibility 1.2.1 (R7RS Compatibility). Any program that runs correctly under R7RS-small will run identically under Lambdust when operated in Dynamic mode. The formal semantics guarantee this compatibility through conservative extension properties.

1.2.2. Gradual Dependent Typing

The type system operates at four levels of sophistication, allowing developers to choose the right balance between flexibility and safety for each part of their program:

1. **Dynamic Mode:** No static type checking (identical to R7RS-small). This mode provides maximum flexibility for rapid prototyping, interactive development, and interfacing with untyped code. All type checking happens at runtime through Scheme's built-in predicates.
2. **Gradual Mode:** Optional type annotations with runtime contract checking. When type annotations are provided, the system inserts runtime checks that verify contracts are satisfied. This catches type errors immediately rather than letting them propagate, making debugging much easier. The system can also optimize code paths where types are known.
3. **Static Mode:** Hindley-Milner style type inference with extensions. The type checker attempts to infer types for all expressions and reports conflicts at compile time. This provides strong correctness guarantees with minimal annotation burden. Extensions include row polymorphism for records and higher-kinded types for advanced abstractions.

4. **Dependent Mode:** Full dependent types with compile-time proof obligations. Types can depend on values, enabling precise specifications like “a sorted list of length n ” or “a function that preserves array bounds”. The type checker becomes a theorem prover, ensuring that programs meet their specifications.

Programs can mix modes freely—a single codebase might use dynamic typing for scripting code, gradual typing for application logic, static typing for core algorithms, and dependent typing for security-critical components.

Example 1.2.2 (Gradual Type Enhancement).

```
;; Level 1: Pure R7RS (Dynamic)
(define (factorial n)
  (if (<= n 1) 1 (* n (factorial (- n 1)))))

;; Level 2: Gradual typing with contracts
(define (factorial n)
  #:type (-> Natural Natural)
  (if (<= n 1) 1 (* n (factorial (- n 1)))))

;; Level 3: Static typing with inference
(define (factorial :: (-> Natural Natural))
  (lambda (n)
    (if (<= n 1) 1 (* n (factorial (- n 1)))))

;; Level 4: Dependent types with proofs
(define (factorial :: (Pi (n : Natural)
                        (Refinement Natural (lambda (r)
                                              (>= r 1)))))
  (lambda (n)
    (if (<= n 1)
        (the (Refinement Natural (lambda (r) (= r 1))) 1)
        (* n (factorial (- n 1)))))
```

1.2.3. Algebraic Effects and Handlers

Lambdust provides a structured approach to side effects through algebraic effects, solving many problems that plague traditional approaches to I/O, exceptions, and stateful computation:

- **Effect Declaration:** Effects are declared with their operation signatures, making all side effects explicit in the type system. This eliminates hidden dependencies and makes code more predictable and testable.
- **Effect Performance:** Code can perform effects using the `perform` form, which suspends computation and transfers control to the nearest handler. This is more composable than exceptions because it allows resumption with multiple values.
- **Effect Handling:** Effects are handled using the `handle` form with resumption capabilities. Handlers can inspect the operation, its arguments, and the continuation, then decide whether to resume normally,

provide an alternative value, or propagate the effect further.

- **Effect Tracking:** The type system tracks which effects a computation may perform, preventing effect leakage and enabling local reasoning about code. Functions that don’t declare any effects are guaranteed to be pure.

This approach eliminates callback hell, provides structured error handling that composes properly, and enables powerful abstractions like cooperative threading, backtracking, and probabilistic programming. Unlike monads, algebraic effects compose naturally without monad transformer complexity.

Example 1.2.3 (Effects and Handlers).

```
;; Declare a State effect
(define-effect State
  (get :: (-> Unit Natural))
  (set :: (-> Natural Unit)))

;; Use the effect
(define (increment)
  (let ((current (perform State get)))
    (perform State set (+ current 1))))

;; Handle the effect
(handle
  (begin (increment) (increment) (perform State get))
  (State
    ((get k) (k state))
    ((set new-state k) (k 'unit)))))
```

1.2.4. Actor-Based Concurrency

Lambdust provides lightweight, fault-tolerant concurrency through the actor model, addressing the complexity and fragility of shared-memory concurrency:

- **Actor Spawning:** Create new concurrent actors with `spawn`. Actors are cheap to create (thousands can run simultaneously) and completely isolated from each other, eliminating race conditions and making concurrent code much easier to reason about.
- **Message Passing:** Actors communicate via asynchronous message passing with automatic serialization. This eliminates shared mutable state and data races while providing natural back-pressure and flow control mechanisms.
- **Pattern Matching:** Receive messages using pattern-based selection with priority queues and timeout handling. This provides structured concurrency where actors can handle different types of requests appropriately without complex dispatch logic.

- **Supervision:** Actors can supervise others for fault tolerance, implementing "let it crash" philosophy where failures are contained and recovery is automatic. Supervision trees provide hierarchical error handling that scales to large distributed systems.
- **Location Transparency:** Actors can be distributed across networks without code changes. The same message-passing interface works whether actors are on the same thread, different processes, or remote machines, enabling seamless scaling.

This model has been proven in production systems handling millions of concurrent connections (Erlang/OTP) and provides natural solutions to distributed systems problems like partial failures, network partitions, and service discovery.

Example 1.2.4 (Actor Concurrency).

```
;; Define a counter actor
(define (counter-actor initial-count)
  (letrec ((loop (lambda (count)
                   (receive
                     (('increment sender)
                      (send sender (+ count 1))
                      (loop (+ count 1)))
                     (('get sender)
                      (send sender count)
                      (loop count))))))
    (loop initial-count)))

;; Spawn and use the actor
(let ((counter (spawn (counter-actor 0))))
  (send counter '(increment self))
  (send counter '(get self))
  (receive
    ((value) (display value)))) ; Prints: 1
```

1.2.5. Memory-Safe Foreign Function Interface

Lambdust provides safe interoperability with C libraries through a capability-based FFI:

- **Type Safety:** Foreign function signatures are statically checked
- **Memory Safety:** Automatic marshaling prevents buffer overflows and dangling pointers
- **Capability Security:** Access to foreign functions requires explicit capabilities
- **Error Handling:** Foreign function errors are converted to Scheme exceptions

Example 1.2.5 (Foreign Function Interface).

```
;; Declare foreign function binding
(foreign-declare "math.h"
  (sqrt :: (-> Float64 Float64)))

;; Use with capability
(with-capability 'math-library
  (lambda ()
    (let ((result (foreign-call sqrt 16.0)))
      (display result)))) ; Prints: 4.0
```

1.2.6. Proof-Carrying Code

Through the Curry-Howard correspondence, Lambdust enables extracting verified programs from constructive proofs:

- **Propositions as Types:** Logical propositions correspond to types
- **Proofs as Programs:** Constructive proofs correspond to programs
- **Proof Checking:** The type checker verifies proof correctness
- **Program Extraction:** Verified programs are extracted from proofs

Example 1.2.6 (Proof-Carrying Code).

```
;; Prove a property and extract the implementation
(define sorted-insert-proof
  (prove (Pi (x : Natural) (Pi (xs : SortedList)
    (SortedList-property (insert x xs))))
    (lambda (x xs)
      ;; Constructive proof by induction...
      (proof-by-induction xs
        (base-case (list x))
        (inductive-step ...))))

;; Extract the verified implementation
(define sorted-insert (extract sorted-insert-proof))
```

1.3. Implementation Status

This specification describes Lambdust version 0.2.0. The current implementation includes:

- **Complete:** R7RS-small compatibility, lexer, parser, evaluator
- **Complete:** Basic gradual typing with type inference
- **Complete:** Algebraic effects with handler implementation
- **Experimental:** Actor-based concurrency system

- **Experimental:** Foreign function interface
- **Research:** Dependent types and proof system

1.4. Relationship to Other Languages

Lambdust draws inspiration from several language communities:

- **Scheme Family:** Maintains the elegance and simplicity of Scheme while adding modern features
- **ML Family:** Adopts sophisticated type systems from languages like OCaml and Haskell
- **Effect Systems:** Incorporates algebraic effects from languages like Koka and Effekt
- **Dependent Types:** Integrates dependent typing from Agda, Coq, and Lean
- **Actor Systems:** Uses proven concurrency models from Erlang and Akka

1.5. Specification Structure

This specification is organized as follows:

- Chapter 2: Lexical structure and tokenization rules
- Chapter 3: Basic syntax and core language forms
- Chapter 4: Gradual typing system and type inference
- Chapter 5: Dependent types and proof obligations
- Chapter 6: Algebraic effects and handler semantics
- Chapter 7: Actor-based concurrency and message passing
- Chapter 8: Foreign function interface and capability system
- Chapter 9: Macro system extensions beyond R7RS
- Chapter 10: Standard library and built-in procedures
- Chapter 12: Comprehensive programming examples
- Chapter 11: Formal denotational semantics

The appendices provide R7RS compatibility information, SRFI support matrices, migration guides, and implementation notes for language developers.

1.6. Conventions

This specification uses the following conventions:

- **Syntax:** Grammar productions use extended BNF notation
- **Semantics:** Denotational semantics follow standard mathematical notation
- **Examples:** Code examples use syntax highlighting for clarity
- **Types:** Type expressions use mathematical notation where appropriate
- **Cross-references:** All major concepts are cross-referenced and indexed

Compatibility notes highlight differences from R7RS-small, and implementation notes provide guidance for language implementers.

1.7. Acknowledgments

Lambdust builds upon decades of programming language research and the dedicated work of the Scheme community. We particularly acknowledge the R7RS-small working group for providing an excellent foundation, the authors of the many SRFIs that inform our design, and the researchers who developed the theoretical foundations for dependent types, algebraic effects, and gradual typing that make Lambdust possible.

2. Lexical Structure

The lexical structure of Lambdust extends R7RS-small with additional token types to support gradual dependent typing, effects, concurrency, and other advanced features while maintaining complete backward compatibility.

2.1. Lexical Elements

A Lambdust program consists of a sequence of lexical elements:

- Whitespace and comments (ignored)
- Tokens (significant elements)
- End-of-file marker

2.2. Character Set

Lambdust source code uses Unicode UTF-8 encoding. All R7RS-small character classifications are preserved:

- **Whitespace:** Space, tab, newline, carriage return, form feed
- **Delimiters:** Whitespace, parentheses, brackets, braces, quotes, semicolon
- **Letters:** Unicode letter categories (Alphabetic property)
- **Digits:** ASCII digits 0-9
- **Special Initial:** ! \$ % & * / : < = > ? _ ^ ~
- **Special Subsequent:** + - . @

R7RS Compatibility 2.2.1 (Character Set Compatibility). All R7RS-small character classifications and identifier rules are preserved unchanged in Lambdust.

2.3. Comments

Lambdust supports all R7RS comment forms plus nested block comments:

2.3.1. Line Comments

Line comments begin with semicolon and extend to end of line:

```
; This is a line comment
(define x 42) ; Another comment
```

2.3.2. Block Comments

Block comments are delimited by #| and |# and may be nested:

```
#| This is a block comment
  #| with nested comments |#
  that can span multiple lines |#
(define y 17)
```

2.3.3. Expression Comments

Expression comments use #; to comment out the following expression:

```
(list 1 #;(* 2 3) 4) ; Equivalent to (list 1 4)
```

2.4. Identifiers

identifier Identifiers follow R7RS rules with no extensions:

```
identifier → initial subsequent*
           | vertical-line symbol-element* vertical-line
           | peculiar-identifier

initial → letter
        | special-initial

letter → a | b | c | ... | z
       | A | B | C | ... | Z

special-initial → ! | $ | % | & | *
                | / | : | < | = | >
                | ? | ` | _ | ~

subsequent → initial | digit
            | special-subsequent

special-subsequent → + | - | . | @

peculiar-identifier → + | - | ... | →
```

Example 2.4.1 (Valid Identifiers).

```
lambda           ; Standard identifier
list->vector      ; With special characters
string-length    ; Hyphenated style
camelCase        ; Camel case style
PascalCase       ; Pascal case style
+                ; Single character
...              ; Ellipsis
->               ; Arrow (peculiar identifier)
```

2.5. Keywords

keyword Lambdust extends R7RS with keyword literals for named parameters and metadata:

```
keyword → #: identifier
```

Keywords are self-evaluating and distinct from symbols:

```
#:type           ; Type annotation keyword
#:inline         ; Optimization hint keyword
#:doc            ; Documentation keyword
#:test           ; Test specification keyword
```

Example 2.5.1 (Keyword Usage).

```
(define (factorial n)
  #:type (-> Natural Natural)
  #:doc "Compute factorial of natural number"
  (if (<= n 1) 1 (* n (factorial (- n 1)))))
```

2.6. Numbers

number Lambdust supports the complete R7RS numeric tower with all exactness and radix specifications:

number → *num-2* | *num-8* | *num-10* | *num-16*

2.6.1. Integer Numbers

```
42          ; Decimal integer
#b1010      ; Binary integer (10)
#o52        ; Octal integer (42)
#x2A        ; Hexadecimal integer (42)
#e42        ; Exact integer
#i42        ; Inexact integer
+42         ; Positive integer
-42         ; Negative integer
```

2.6.2. Real Numbers

```
3.14159     ; Decimal real
1.23e10     ; Scientific notation
1.23E-5     ; Scientific notation
#i3.14      ; Explicitly inexact
#e1.5       ; Exact rational (3/2)
.5          ; Leading decimal point
5.          ; Trailing decimal point
```

2.6.3. Rational Numbers

```
1/2         ; One half
-3/4        ; Negative three quarters
#e22/7      ; Exact rational
#i1/3       ; Inexact rational
```

2.6.4. Complex Numbers

```
3+4i       ; Rectangular form
3-4i       ; Rectangular form
+i         ; Pure imaginary unit
-i         ; Negative imaginary unit
3.0+0i     ; Real with imaginary zero
0+1.5i     ; Pure imaginary
```

2.7. Strings

string String literals follow R7RS syntax with full escape sequence support:

string → " *string-element** "

string-element → *any character other than " or *
 | *\ " | \ *
 | *\ n | \ r | \ t*
 | *\ intraline-whitespace newline intraline-whitespace*
 | *\ x hex-digit hex-digit** ;

Example 2.7.1 (String Literals).

```
"Hello, World!" ; Simple string
"String with \"quotes\"" ; Escaped quotes
"Line 1\nLine 2" ; Newline escape
"Unicode: \x03BB;" ; Unicode lambda
"Long string \
can span lines" ; Line continuation
"" ; Empty string
```

2.8. Characters

character Character literals use R7RS syntax with named characters:

character → #\ *any-character*
 | #\ *character-name*
 | #\ x *hex-scalar-value*

Example 2.8.1 (Character Literals).

```
#\a          ; Letter 'a'
#\A          ; Letter 'A'
#\space      ; Space character
#\newline    ; Newline character
#\tab        ; Tab character
#\return     ; Carriage return
#\x03BB      ; Unicode lambda ($\lambda$)
#\x0         ; Null character
```

2.9. Booleans

boolean Boolean literals follow R7RS with both short and long forms:

boolean → #t | #f | #true | #false

```
#t          ; True (short form)
#f          ; False (short form)
#true       ; True (long form)
#false      ; False (long form)
```

2.10. Delimiters

delimiter Lambdust uses R7RS delimiters plus additional ones for type expressions:

2.10.1. Parentheses

() ; Standard list delimiters

2.10.2. Brackets

[] ; Alternative list delimiters (equivalent to parentheses)

2.10.3. Braces

`{ }` ; Set literals and type expressions

2.11. Quote and Unquote

`quote` `quasiquote` `unquote` All R7RS quote forms are supported:

abbreviation → *abbrev-prefix datum*

abbrev-prefix → ' | ` | , | ,@

Example 2.11.1 (Quote Forms).

```
'(a b c) ; Quote
`(list ,x ,@y) ; Quasiquote with unquote and
               unquote-splicing
```

2.12. Lambdust Extensions

`type annotation` `arrow` `pipe` `tilde`

Lambdust adds several new token types for advanced features:

2.12.1. Type Annotation

type-annotation → ::

Used for explicit type ascription:

```
((:: expr Type) ; Type ascription
(define foo :: Type expr) ; Definition with type
```

2.12.2. Arrow Types

arrow → →
fat-arrow → =>
tilde-arrow → >

Used in type expressions:

```
(> A B) ; Function type A to B
(>= P Q) ; Implication or constraint
(> A B) ; Effectful function type
```

2.12.3. Type Operators

colon → :
pipe → |
tilde → ~

Used in type expressions:

```
(x : Type) ; Parameter with type
A | B ; Union type or type alternative
~Effect ; Effect annotation
```

2.13. Token Classification

The lexer recognizes the following token types:

Table 2.1: Lambdust Token Types

Token Type	Examples	Usage
LeftParen	(	List start, procedure calls
RightParen	)	List end, procedure calls
LeftBracket	[	Alternative list notation
RightBracket	]	Alternative list notation
LeftBrace	{	Set literals, type expressions
RightBrace	}	Set literals, type expressions
Quote	'	Quote abbreviation
Quasiquote	`	Quasiquote abbreviation
Unquote	,	Unquote abbreviation
UnquoteSplicing	,@	Unquote-splicing abbreviation
Dot	.	Pair notation, module access
TypeAnnotation	::	Type ascription
Colon	:	Parameter type annotation
Arrow	→	Function types
FatArrow	⇒	Constraints, implications
Pipe		Union types, alternatives
Tilde	~	Effect annotations
TildeArrow	~>	Effectful function types
IntegerNumber	42, #x2A	Integer literals
RealNumber	3.14, 1e10	Real number literals
RationalNumber	1/2, 22/7	Rational literals
ComplexNumber	3+4i, +i	Complex number literals
String	"hello"	String literals
Character	#\a, #\space	Character literals
Boolean	#t, #false	Boolean literals
Keyword	#:type, #:doc	Keyword literals
Identifier	lambda, +, foo	Variable names, operators
LineComment	; comment	Documentation (ignored)
BlockComment	# comment #	Documentation (ignored)
Error	Invalid input	Lexical errors
Eof	End of input	End of file marker

2.14. Lexical Precedence

When multiple token patterns could match, the lexer uses longest-match with the following precedence:

1. Numbers (including complex forms)
2. Keywords (`#:identifier`)
3. Booleans (`#t`, `#f`, `#true`, `#false`)
4. Characters (`#\...`)
5. Multi-character operators (`::`, `→`, `⇒`, `>`, `~`, `@`)
6. Single-character tokens
7. Identifiers
8. Comments

2.15. Error Handling

lexical error The lexer reports errors for:

- Invalid character sequences
- Unterminated strings or comments
- Invalid numeric formats
- Invalid character literals
- Invalid Unicode escape sequences

Each error includes source location information (line, column, file) for precise error reporting.

2.16. Implementation Notes

2.16.1. Performance Considerations

The lexer is optimized for:

- Single-pass tokenization
- Minimal backtracking
- Efficient Unicode handling
- Memory-efficient token storage

2.16.2. Compatibility Mode

Implementations may provide a compatibility mode that accepts only R7RS tokens, rejecting Lambdust extensions for strict R7RS compliance checking.

2.16.3. Extensibility

The lexical structure is designed to allow future extensions without breaking existing code. Reserved token patterns may be allocated for future language features.

3. Basic Syntax and Semantics

This section defines the core syntactic forms of Lambdust, extending R7RS-small with new constructs while maintaining complete backward compatibility. The syntax is organized into expressions, which are the primary syntactic category.

3.1. Syntactic Categories

Lambdust programs consist of expressions organized into the following syntactic categories:

program → *import-declaration** *expression**

expression → *constant*
 | *variable*
 | *procedure-call*
 | *lambda-expression*
 | *conditional*
 | *assignment*
 | *derived-expression*
 | *macro-use*
 | *type-expression*
 | *effect-expression*
 | *concurrency-expression*
 | *foreign-expression*

3.2. Constants and Literals

constant literal

Constants are self-evaluating expressions:

constant → *boolean* | *number*
 | *character* | *string*
 | *keyword*

Example 3.2.1 (Constants).

```
#t           ; Boolean true
42           ; Integer
3.14         ; Real number
1/3         ; Rational number
3+4i        ; Complex number
#\lambda    ; Character
"Hello"     ; String
#:type      ; Keyword
```

3.3. Variables

variable

Variables are identifiers that refer to bound values:

$$\text{variable} \rightarrow \text{identifier}$$

Variable references are resolved lexically according to R7RS scoping rules.

3.4. Procedure Calls

procedure call application

Procedure calls apply a procedure to arguments:

$$\begin{aligned} \text{procedure-call} &\rightarrow (\text{operator operand}^*) \\ \text{operator} &\rightarrow \text{expression} \\ \text{operand} &\rightarrow \text{expression} \end{aligned}$$

Example 3.4.1 (Procedure Calls).

```
(+ 1 2 3)           ; Add three numbers
(cons 'a '(b c))     ; Cons onto list
((lambda (x) (+ x 1)) 5) ; Apply lambda
(f x y z)            ; Call f with three arguments
```

3.5. Lambda Expressions

lambda procedure

Lambda expressions create procedures:

$$\begin{aligned} \text{lambda-expression} &\rightarrow (\text{lambda } \text{formals } \text{expression}^*) \\ \text{formals} &\rightarrow (\text{identifier}^*) \\ &\quad | (\text{identifier}^* . \text{identifier}) \\ &\quad | \text{identifier} \end{aligned}$$

Example 3.5.1 (Lambda Expressions).

```
(lambda (x) (+ x 1))           ; Single parameter
(lambda (x y) (* x y))        ; Multiple parameters
(lambda args (length args))   ; Variable arguments
(lambda (x . rest)
  (cons x rest))              ; Fixed + variable
                                arguments
```

3.6. Conditionals

if conditional

Conditional expressions perform branches based on test values:

$$\begin{aligned} \text{conditional} &\rightarrow (\text{if } \text{test } \text{consequent } \text{alternate}) \\ &\quad | (\text{if } \text{test } \text{consequent}) \end{aligned}$$

Example 3.6.1 (Conditionals).

```
(if (> x 0) 'positive 'non-positive) ; Full
conditional
(if (null? lst) 'empty)              ; No alternate
```

3.7. Assignment

set! assignment

Assignment expressions modify variable bindings:

$$\text{assignment} \rightarrow (\text{set! } \text{variable } \text{expression})$$

Example 3.7.1 (Assignment).

```
(set! x 42)           ; Assign 42 to x
(set! counter (+ counter 1)) ; Increment counter
```

3.8. Derived Expressions

derived expression

Derived expressions are syntactic sugar that can be expressed in terms of more primitive forms.

3.9. Conditionals

Syntax: (cond *clause**)

Multi-way conditional with optional else clause.

```
(cond ((< x 0) 'negative)
      ((= x 0) 'zero)
      (else 'positive))
```

Syntax: (case *key clause**)

Dispatch on datum equality.

```
(case op
  ((+) (+ x y))
  ((-) (- x y))
  (else (error "unknown op")))
```

3.10. Boolean Operations

Syntax: (and *expression**)

Short-circuit logical and.

```
(and (> x 0) (< x 100)) ; Check range
```

Syntax: (or *expression**)

Short-circuit logical or.

```
(or (null? lst) (pair? lst)) ; Check list type
```

3.11. Binding Forms

Syntax: (let (*binding**) *expression**)

Sequential local binding.

```
(let ((x 1) (y 2))
  (+ x y))
```

Syntax: (let* (*binding**) *expression**)

Sequential binding with each binding visible to later ones.

```
(let* ((x 1) (y (+ x 1)))
  (+ x y))
```

Syntax: (letrec (*binding**) *expression**)

Recursive binding for mutual recursion.

```
(letrec ((even? (lambda (n)
  (if (= n 0) #t (odd? (- n 1)))))
  (odd? (lambda (n)
  (if (= n 0) #f (even? (- n 1)))))
  (even? 42))
```

3.12. Sequencing

Syntax: (begin *expression**)

Sequential evaluation returning last result.

```
(begin
  (display "Hello ")
  (display "World")
  (newline))
```

3.13. Iteration

Syntax: (do (*variable* init step*) (*test result**) *command**)

General iteration construct.

```
(do ((i 0 (+ i 1))
    (sum 0 (+ sum i)))
    ((>= i 10) sum)) ; Sum 0 to 9
```

3.14. Definitions

define definition

Definitions bind identifiers to values or procedures:

```
definition → (define variable expression)
              | (define (variable identifier*) expression*)
              | (define (variable . identifier) expression*)
```

Example 3.14.1 (Definitions).

```
(define pi 3.14159) ; Variable
definition
```

```
(define (square x) (* x x)) ; Procedure
definition
```

```
(define (list-of-squares . numbers) ; Variadic
  procedure
  (map square numbers))
```

3.15. Quoting and Quasiquoting

quote quasiquote

3.16. Quote

Quote prevents evaluation and returns literal data:

$$\begin{array}{lcl} \text{quotation} & \rightarrow & (\text{quote datum}) \\ & | & \text{'datum} \end{array}$$

Example 3.16.1 (Quotation).

```
(quote (a b c)) ; List of symbols
'(a b c) ; Abbreviated form
'symbol ; Symbol
'42 ; Number (self-evaluating)
```

3.17. Quasiquote

Quasiquote allows selective evaluation within quoted structure:

$$\begin{array}{lcl} \text{quasiquote} & \rightarrow & (\text{quasiquote template}) \\ & | & \text{'template} \\ \text{template} & \rightarrow & \text{datum} \\ & | & (\text{unquote expression}) \\ & | & \text{,expression} \\ & | & (\text{unquote-splicing expression}) \\ & | & \text{,@expression} \end{array}$$

Example 3.17.1 (Quasiquote).

```
`(list ,(+ 1 2) 4) ; Evaluates to (list 3 4)
'(a ,@(b c) d) ; Evaluates to (a b c d)
`((f x) = ,(f x)) ; Template with evaluation
```

3.18. Type Expressions

type expression

Lambdust extends syntax with type expressions for gradual typing:

$$\begin{array}{lcl} \text{type-expression} & \rightarrow & (:: \text{expression type}) \\ & | & (\text{expression} :: \text{type}) \\ \\ \text{type} & \rightarrow & \text{base-type} \\ & | & \text{compound-type} \\ & | & \text{dependent-type} \\ \\ \text{base-type} & \rightarrow & \text{Dynamic} \mid \text{Number} \mid \text{String} \\ & | & \text{Boolean} \mid \text{Symbol} \mid \text{Character} \\ \\ \text{compound-type} & \rightarrow & (\rightarrow \text{type}^* \text{type}) \\ & | & (\text{List type}) \\ & | & (\text{Vector type}) \\ & | & (\text{type} \mid \text{type}) \end{array}$$

Example 3.18.1 (Type Expressions).

```
(:: 42 Number) ; Type ascription
(:: (+ 1 2) Natural) ; Refined type
(:: factorial (-> Natural Natural)) ; Function type
(:: lst (List Number)) ; Parameterized type
```

3.19. Effect Expressions

effect perform handle

Effect expressions provide structured side effect management:

$$\begin{array}{lcl} \text{effect-expression} & \rightarrow & (\text{perform effect-name expression}^*) \\ & | & (\text{handle expression handler}) \\ & | & (\text{with-effect effect-name expression}) \end{array}$$

$$\begin{array}{lcl} \text{handler} & \rightarrow & (\text{effect-name operation-clause}^*) \\ \text{operation-clause} & \rightarrow & (\text{operation parameters body}) \end{array}$$

Example 3.19.1 (Effect Expressions).

```
;; Perform an effect
(perform State get)
(perform State set 42)

;; Handle effects
(handle
  (begin
    (perform State set 10)
    (perform State get))
  (State
    ((get k) (k current-state))
    ((set new-state k)
     (set! current-state new-state)
     (k 'unit))))
```

3.20. Concurrency Expressions

spawn send receive

Concurrency expressions support actor-based parallel programming:

$$\begin{array}{lcl} \text{concurrency-expression} & \rightarrow & (\text{spawn expression}) \\ & | & (\text{send expression expression}) \\ & | & (\text{receive message-clause}^*) \\ & | & (\text{future expression}) \\ & | & (\text{await expression}) \\ \\ \text{message-clause} & \rightarrow & (\text{pattern expression}) \end{array}$$

Example 3.20.1 (Concurrency Expressions).

```
;; Spawn an actor
(define pid (spawn (lambda () (server-loop))))
```

```
;; Send a message
(send pid '(request 42))

;; Receive messages with pattern matching
(receive
  (('request data)
   (handle-request data))
  (('stop)
   (shutdown-server))
  (other)
  (log-unknown-message other)))

;; Futures for async computation
(let ((f (future (expensive-computation))))
  (do-other-work)
  (await f)) ; Wait for result
```

3.21. Foreign Function Interface

foreign-call foreign-data

Foreign function interface expressions enable C library integration:

foreign-expression → (foreign-call *name type expression**) evaluation order
 | (foreign-data *name type*)
 | (with-capability *capability expression*)

Example 3.21.1 (Foreign Function Interface).

```
;; Declare foreign function
(foreign-declare "math.h"
  (sqrt :: (-> Float64 Float64)))

;; Call foreign function with capability
(with-capability 'math-library
  (lambda ()
    (foreign-call sqrt (-> Float64 Float64) 16.0)))

;; Access foreign data
(foreign-data errno (-> Unit Int32))
```

3.22. Syntax Extensions

syntax-rules syntax-case

Lambdust supports R7RS hygienic macros plus extensions:

macro-definition → (define-syntax *name transformer*)

transformer → (syntax-rules *literals rule**)
 | (syntax-case *expression literals clause**)
 | (lambda (*stx*) *body*)

Example 3.22.1 (Syntax Extensions).

```
;; Simple macro with syntax-rules
```

```
(define-syntax when
  (syntax-rules ()
    ((when test body ...)
     (if test (begin body ...))))))
```

```
;; Pattern matching macro
(define-syntax match
  (syntax-rules ()
    ((match expr
      (pattern body ...) ...)
     (match-implementation expr
      (list (cons pattern (lambda () body ...)) ...))))))
```

```
;; Usage
(when (> x 0)
  (display "positive")
  (newline))
```

```
(match lst
  (() 'empty)
  ((x) 'singleton)
  ((x y) 'pair)
  (other 'longer))
```

3.23. Evaluation Order

- Procedure calls evaluate operator first, then operands left to right
- Conditional test is evaluated before branches
- Assignment evaluates expression before storing
- Let bindings evaluate initializers left to right
- Lambda expressions are not evaluated (they create closures)

3.24. Tail Calls

tail call proper tail recursion

Lambdust provides proper tail recursion as required by R7RS:

Definition 3.24.1 (Tail Position). An expression is in tail position if its value is returned directly as the value of the enclosing procedure without further computation.

Example 3.24.2 (Tail Recursion).

```
;; Tail recursive factorial
(define (factorial n acc)
  (if (<= n 1)
      acc
      (factorial (- n 1) (* n acc)))) ; Tail position ; Tail call
```

```
;; Not tail recursive
```

```
(define (factorial-bad n)
  (if (<= n 1)
      1
      (* n (factorial-bad (- n 1)))) ; Not in tail
                                     position
```

3.25. Continuations

call/cc continuation

Lambdust supports full continuations via
call-with-current-continuation:

Procedure: call-with-current-continuation

call-with-current-continuation *proc* $\rightarrow$ *obj*

Captures the current continuation and passes it as a procedure to *proc*. The continuation procedure, when called, abandons the current computation and returns its argument as the result of the call-with-current-continuation call.

Example 3.25.1 (Continuations).

```
;; Early return from loop
(define (find-positive lst)
  (call/cc
   (lambda (return)
     (for-each
      (lambda (x)
        (when (> x 0) (return x)))
      lst)
     #f)))

;; Non-local exit
(define (deep-computation x)
  (call/cc
   (lambda (escape)
     (let ((result (complex-calculation x)))
       (if (valid? result)
           result
           (escape 'error))))))
```

3.26. Multiple Values

values call-with-values

Lambdust supports multiple value returns:

Procedure: values *obj**

Returns multiple values.

Procedure: call-with-values *producer consumer*

Calls *producer* with no arguments and passes all returned values to *consumer*.

Example 3.26.1 (Multiple Values).

```
;; Return multiple values
(define (quotient-remainder x y)
  (values (quotient x y) (remainder x y)))

;; Consume multiple values
(call-with-values
 (lambda () (quotient-remainder 17 5))
 (lambda (q r)
  (display "Quotient: ") (display q) (newline)
  (display "Remainder: ") (display r) (newline)))

;; Using let-values (derived form)
(let-values (((q r) (quotient-remainder 17 5)))
  (list q r))
```

3.27. Compatibility Guarantees

R7RS Compatibility 3.27.1 (R7RS Semantic Compatibility). All R7RS-small expressions have identical semantics in Lambdust when run in Dynamic mode. This includes:

- Lexical scoping and variable binding
- Procedure call evaluation order
- Proper tail recursion requirements
- Continuation semantics
- Multiple value protocols
- Numeric tower operations

The formal semantics in Chapter 11 provide mathematical precision for these compatibility guarantees.

4. Gradual Type System

Lambdust provides a gradual type system that solves the traditional tension between the flexibility of dynamic typing and the safety guarantees of static typing. Rather than forcing developers to choose one approach for an entire program, Lambdust allows different parts of the same codebase to use different levels of type checking based on their specific needs.

This approach is particularly valuable for large codebases where different components have different correctness requirements. User interface code might benefit from dynamic typing's flexibility for rapid iteration, while financial calculations might require the precision guarantees of

dependent types. The gradual type system allows both approaches to coexist safely within the same program.

The system operates at four sophistication levels, from completely dynamic (R7RS-compatible) to fully static with dependent types. Each level provides progressively stronger guarantees while requiring more explicit type information from the programmer.

4.1. Type System Overview

The gradual type system provides smooth transitions between typing disciplines, enabling developers to adopt stronger typing guarantees where they provide the most benefit:

1. **Dynamic Mode:** Pure R7RS-small compatibility with no type checking. This mode is ideal for exploratory programming, scripting, and interfacing with external systems where types are not known in advance. All values carry runtime type information, and type errors are detected when operations are attempted on inappropriate values.
2. **Gradual Mode:** Optional type annotations with runtime contracts. When type annotations are provided, the system automatically inserts contract checks at function boundaries and variable assignments. This catches type errors closer to their source, making debugging easier, while allowing untyped code to interoperate freely. The type system can also optimize known-type code paths.
3. **Static Mode:** Hindley-Milner type inference with extensions for modern programming patterns. The type checker attempts to infer types for all expressions and reports conflicts before program execution. This provides strong correctness guarantees with minimal annotation burden. Extensions include row polymorphism for flexible record types, higher-kinded types for generic programming, and effect types for tracking computational effects.
4. **Dependent Mode:** Full dependent types with compile-time proof obligations. Types can depend on program values, enabling precise specifications like "a list of exactly n elements" or "a function that preserves sorting order". The type checker becomes a theorem prover, ensuring that programs satisfy their specifications mathematically.

Programs can mix code from different levels seamlessly, with automatic insertion of runtime checks at boundaries between typed and untyped regions. This provides a smooth migration path where teams can adopt stronger

typing incrementally as they gain confidence and experience with the type system.

4.2. Type Syntax

type syntax

<i>type</i>	→	<i>base-type</i>
		<i>compound-type</i>
		<i>type-variable</i>
		<i>quantified-type</i>
<i>base-type</i>	→	Dynamic Number String
		Boolean Symbol Character
		Null Void Any
<i>compound-type</i>	→	(→ <i>type</i> * <i>type</i>)
		(List <i>type</i>)
		(Vector <i>type</i>)
		(Pair <i>type</i> <i>type</i>)
		(<i>type</i> <i>type</i>)
		(<i>type</i> & <i>type</i>)
<i>type-variable</i>	→	<i>identifier</i>
<i>quantified-type</i>	→	(All (<i>type-variable</i> *) <i>type</i>)
		(Exists (<i>type-variable</i> *) <i>type</i>)

4.3. Base Types

base type

4.3.1. Dynamic Type

The **Dynamic** type represents untyped values from R7RS code:

Type: Dynamic

The universal type that accepts any value and provides no static guarantees. All R7RS expressions have type **Dynamic** in Dynamic mode.

4.3.2. Primitive Types

Type: Number

Represents all numeric values in the R7RS numeric tower: integers, rationals, reals, and complex numbers.

Type: String

Represents immutable Unicode strings.

Type: Boolean

Represents boolean values `#t` and `#f`.

Type: Symbol

Represents interned symbolic identifiers.

Type: Character

Represents Unicode scalar values.

Type: Null

Represents the empty list `()`.

Type: Void

Represents the unspecified value returned by some procedures.

4.4. Refined Numeric Types

The type system provides refined numeric types:

Type: Integer

Exact integers (subset of **Number**).

Type: Natural

Non-negative integers $\{0, 1, 2, \dots\}$.

Type: Positive

Positive integers $\{1, 2, 3, \dots\}$.

Type: Real

Real numbers (excludes complex numbers with non-zero imaginary parts).

Type: Float

Inexact real numbers.

Type: Rational

Exact rational numbers.

Type: Complex

Complex numbers including reals.

4.5. Compound Types

compound type

4.5.1. Function Types

Function types specify parameter and return types:

Type: $(\rightarrow T_1 T_2 \dots T_n R)$

Function accepting parameters of types $T_1, T_2, \dots, T_n$ and returning type R .

Type: $(\rightarrow^* (T_1 T_2 \dots) (U_1 U_2 \dots) R)$

Function with required parameters $T_1, T_2, \dots$ and optional parameters $U_1, U_2, \dots$ returning R .

Example 4.5.1 (Function Types).

```
;; Simple function type
(define add :: (-> Number Number Number))

;; Higher-order function
(define map :: (All (A B) (-> (-> A B) (List A) (List B))))

;; Variadic function
(define list :: (All (A) (->* () (A) (List A))))
```

4.5.2. List Types**Type:** (List T)Homogeneous list of elements of type T .**Type:** (Listof T)

Synonym for (List T).

Type: (List T1 T2 ... Tn)Heterogeneous list with exactly n elements of specified types.**Example 4.5.2** (List Types).

```
;; Homogeneous list
(define numbers :: (List Number) '(1 2 3 4 5))

;; Heterogeneous list
(define person :: (List String Natural Boolean)
  '("Alice" 30 #t))

;; List of functions
(define ops :: (List (-> Number Number Number))
  (list + - * /))
```

4.5.3. Vector Types**Type:** (Vector T)Vector of elements of type T .**Type:** (Vector T1 T2 ... Tn)Vector with exactly n elements of specified types.**4.5.4. Pair Types****Type:** (Pair T U)Pair with car of type T and cdr of type U .**Example 4.5.3** (Pair Types).

```
;; Association list entry
(define entry :: (Pair String Number) '("count" . 42))

;; Nested pairs forming a list
(define point :: (Pair Number (Pair Number Null))
  '(3.14 . (2.71 . ())))
```

4.6. Union and Intersection Types

union type intersection type

4.6.1. Union Types

Union types represent values that can be one of several types:

Type: (U T1 T2 ... Tn)Value is one of types $T_1, T_2, \dots, T_n$.**Type:** (T1 | T2 | ... | Tn)

Alternative syntax for union types.

Example 4.6.1 (Union Types).

```
;; Optional value
(define maybe-string :: (U String Null)
  (if (> (random) 0.5) "hello" '()))

;; Result type
(define (safe-divide :: (-> Number Number (U Number String)))
  (lambda (x y)
    (if (= y 0)
        "Division by zero"
        (/ x y))))
```

4.6.2. Intersection Types

Intersection types represent values that satisfy multiple type constraints:

Type: ($\& T_1 \& T_2 \dots \& T_n$)

Value satisfies all types $T_1, T_2, \dots, T_n$.

Example 4.6.2 (Intersection Types).

```
;; Function that's both numeric and callable
(define numeric-proc :: (& Number (-> Void Number))
  (let ((value 42))
    (case-lambda
      ((() value)
       ((new-val) (set! value new-val))))))
```

4.7. Polymorphic Types

polymorphism type variable

4.7.1. Universal Quantification

Type: (All (X1 X2 ... Xn) T)

Type T parameterized by type variables $X_1, X_2, \dots, X_n$.

Example 4.7.1 (Polymorphic Types).

```
;; Generic identity function
(define identity :: (All (A) (-> A A))
  (lambda (x) x))

;; Generic list operations
(define length :: (All (A) (-> (List A) Natural))
  (lambda (lst)
    (if (null? lst) 0 (+ 1 (length (cdr lst))))))

;; Polymorphic data structure
(define-struct (Box A) (value :: A))

(define make-box :: (All (A) (-> A (Box A))))
(define box-value :: (All (A) (-> (Box A) A)))
```

4.7.2. Existential Quantification

Type: (Exists (X1 X2 ... Xn) T)

Existential type hiding implementation details.

Example 4.7.2 (Existential Types).

```
;; Abstract data type
(define counter :: (Exists (State)
  (& (-> Void State)
    initialize
    (-> State Void)
    increment
    (-> State Natural)))) ; get

(let ((count 0))
  (values
    (lambda () count)
    initialize (returns current)
    (lambda (state) (set! count (+ count 1))) ; increment
    (lambda (state) count)))) ; get
```

4.8. Type Annotations

type annotation

4.8.1. Variable Annotations

Syntax: (define name :: type expression)

Define variable with explicit type.

Syntax: (:: expression type)

Type ascription for any expression.

Example 4.8.1 (Variable Annotations).

```
;; Variable with type
(define pi :: Real 3.14159)

;; Type ascription
(define x (:: (+ 1 2) Natural))

;; Function parameter types
(define (factorial (n :: Natural)) :: Natural
  (if (<= n 1) 1 (* n (factorial (- n 1)))))
```

4.8.2. Function Annotations

Syntax: #:type type-expression

Metadata-style type annotation.

Example 4.8.2 (Function Annotations).

```
(define (add x y)
  #:type (-> Number Number Number)
  (+ x y))

(define factorial
  #:type (-> Natural Natural)
  (lambda (n)
    (if (<= n 1) 1 (* n (factorial (- n 1))))))

;; With documentation
(define (safe-car lst)
  #:type (-> (List Any) (U Any Null))
  #:doc "Returns car or null for empty list"
  (if (null? lst) '() (car lst)))
```

4.9. Type Inference

type inference

The type checker uses Algorithm W (Hindley-Milner) with extensions for gradual typing:

4.9.1. Local Type Inference

Types can often be inferred from context:

Example 4.9.1 (Type Inference).

```
;; Inferred types shown in comments
(define (map f lst)                ; f: (A -> B),
  lst: (List A)                    ;
  (if (null? lst)                  ; -> (List B)
    '()
    (cons (f (car lst))
          (map f (cdr lst)))))

;; Usage determines specific types
(map (:: + (-> Number Number Number)) ; map: (-> Number
  Number Number)                     ;
  '(1 2 3))                           ; (List
  Number) -> (List Number)
```

4.9.2. Constraint Generation

The type checker generates and solves constraints:

Example 4.9.2 (Constraint Generation).

```
;; Function with constraints
(define (compose f g)
  ;; Generated constraints:
  ;; f: X -> Y
  ;; g: Z -> X
  ;; compose: (X -> Y) -> (Z -> X) -> (Z -> Y)
  (lambda (x) (f (g x))))

;; Constraint solving determines:
;; compose :: (All (A B C) (-> (-> A B) (-> C A) (-> C
  B)))
```

4.10. Gradual Typing

gradual typing consistency

4.10.1. Type Consistency

The consistency relation $\sim$ allows **Dynamic** to be consistent with any type:

$$\tau \sim \text{Dynamic} \quad (4.1)$$

$$\text{Dynamic} \sim \tau \quad (4.2)$$

$$\tau \sim \tau \quad (4.3)$$

$$\tau_1 \rightarrow \tau_2 \sim \sigma_1 \rightarrow \sigma_2 \quad \text{if } \tau_1 \sim \sigma_1 \text{ and } \tau_2 \sim \sigma_2 \quad (4.4)$$

4.10.2. Contract Generation

At boundaries between typed and untyped code, contracts are automatically inserted:

Example 4.10.1 (Contract Generation).

```
;; Typed function
(define (typed-add :: (-> Number Number Number))
  (lambda (x y) (+ x y)))

;; Untyped usage - contract inserted automatically
(define untyped-result
  (typed-add "hello" "world")) ; Runtime error: contract
  violation

;; Contract wrapping (conceptual)
(define wrapped-typed-add
  (contract (-> Number Number Number)
    typed-add
    'positive-blame
    'negative-blame))
```

4.10.3. Blame Tracking

Contract violations identify which component is at fault:

Example 4.10.2 (Blame Tracking).

```
;; Typed library function
(define (library-sort :: (-> (List Number) (List Number)))
  (lambda (lst) (sort lst <)))

;; User calls with wrong type
(library-sort '("b" "a"))
;; Contract error:
;; library-sort: contract violation
;; expected: Number
;; given: "b"
;; in: first argument of (-> (List Number) (List Number))
;; blame: caller
```


4.11. Subtyping

subtyping

The type system includes structural subtyping:

4.11.1. Numeric Subtyping

Natural <: Integer <: Rational <: Real <: Complex <: Number (4.5)

Positive <: Natural (4.6)

4.11.2. Function Subtyping

Function types are contravariant in parameters and covariant in results:

$$(\tau_1 \rightarrow \tau_2) <: (\sigma_1 \rightarrow \sigma_2) \quad \text{if } \sigma_1 <: \tau_1 \text{ and } \tau_2 <: \sigma_2 \quad (4.7)$$

4.11.3. List Subtyping

$$(\text{List } \tau) <: (\text{List } \sigma) \quad \text{if } \tau <: \sigma \quad (4.8)$$

4.12. Type Classes

type class

Type classes provide constrained polymorphism:

Example 4.12.1 (Type Classes).

```
;; Define a type class
(define-class (Eq A)
  (equal? :: (-> A A Boolean)))

;; Instance for numbers
(define-instance (Eq Number)
  (equal? =))

;; Instance for strings
(define-instance (Eq String)
  (equal? string=?))

;; Constrained polymorphic function
(define (member :: (All (A) (=> (Eq A) (-> A (List A) Boolean))))
  (lambda (x lst)
    (cond ((null? lst) #f)
          ((equal? x (car lst)) #t)
          (else (member x (cdr lst))))))
```

4.13. Type-Level Computation

type-level computation

The type system supports limited computation at the type level:

Example 4.13.1 (Type-Level Computation).

```
;; Type-level natural numbers
(define-type-family (Add m n)
  (Add Zero n) = n
  (Add (Succ m) n) = (Succ (Add m n)))

;; Vector with statically known length
(define-struct (Vec n A) (length :: n) (data :: (Vector A)))

;; Concatenation preserves length information
(define vec-append :: (All (m n A)
  (-> (Vec m A) (Vec n A) (Vec (Add m n) A))))
```

4.14. Error Messages

The type checker provides detailed error messages:

Example 4.14.1 (Type Error Messages).

```
;; Type mismatch
(define (bad-add x y) (+ x "hello"))
;; Error: Type mismatch
;; Expected: Number
;; Actual: String
;; In expression: "hello"
;; Function: bad-add at line 1

;; Arity mismatch
(+ 1 2 3 "oops")
;; Error: Argument type mismatch
;; Function: +
;; Expected: Number
;; Actual: String
;; Argument position: 4
```

4.15. Optimization

Type information enables optimizations:

4.15.1. Specialization

Polymorphic functions can be specialized based on usage:

Example 4.15.1 (Specialization).

```
;; Generic function
(define map :: (All (A B) (-> (-> A B) (List A) (List B))))

;; Specialized versions generated automatically
;; map-number-number :: (-> (-> Number Number) (List Number) (List Number))
;; map-string-string :: (-> (-> String String) (List String) (List String))
```

4.15.2. Unboxing

Numeric types can be unboxed for efficiency:

Example 4.15.2 (Unboxing).

```
(define (sum-list :: (-> (List Real) Real))
  (lambda (lst)
    ;; Real values can be unboxed to native floating-point
    (fold-left + 0.0 lst)))
```

4.16. Migration Strategies

4.16.1. Incremental Typing

Add types gradually to existing R7RS code:

Example 4.16.1 (Incremental Typing).

```
;; Step 1: Original R7RS code
(define (factorial n)
  (if (<= n 1) 1 (* n (factorial (- n 1)))))

;; Step 2: Add simple type annotation
(define (factorial n)
  #:type (-> Number Number)
  (if (<= n 1) 1 (* n (factorial (- n 1)))))

;; Step 3: Refine with more specific types
(define (factorial n)
  #:type (-> Natural Natural)
  (if (<= n 1) 1 (* n (factorial (- n 1)))))

;; Step 4: Add parameter type
(define (factorial :: (-> Natural Natural))
  (lambda ((n :: Natural))
    (if (<= n 1) 1 (* n (factorial (- n 1))))))
```

4.16.2. Type-Driven Development

Use types to guide implementation:

Example 4.16.2 (Type-Driven Development).

```
;; Start with type signature
(define process-data :: (-> (List String) (List Natural)
  (U String Natural)))

;; Implementation guided by types
(define (process-data strings numbers)
  (if (= (length strings) (length numbers))
    ;; Success case: return sum
    (fold-left + 0 numbers)
    ;; Error case: return message
    "Length mismatch"))
```

4.17. Implementation Notes

4.17.1. Type Checker Architecture

The type checker operates in phases:

1. Parse type annotations and infer missing types
2. Generate and solve type constraints
3. Check consistency between typed and untyped regions
4. Insert contracts at boundaries
5. Report errors with blame information

4.17.2. Runtime System Integration

Types interact with the runtime system:

- Contract wrappers are inserted transparently
- Type tags can be checked for optimization
- Blame tracking maintains stack traces
- Gradual typing preserves R7RS semantics

The gradual type system provides a smooth path from dynamic R7RS Scheme to statically typed functional programming while maintaining the flexibility and expressiveness that makes Scheme powerful.

5. Dependent Type System

Lambdust's dependent type system extends the gradual typing foundation with types that can depend on values, enabling precise specification of program properties and proof-carrying code through the Curry-Howard correspondence.

5.1. Dependent Types Overview

Dependent types allow types to be parameterized by values, not just other types. This enables expressing precise invariants and specifications:

- **Pi Types:** Dependent function types $(x : A) \rightarrow B(x)$
- **Sigma Types:** Dependent pair types $(x : A) \times B(x)$
- **Refinement Types:** Types with logical predicates $\{x : A \mid P(x)\}$
- **Indexed Types:** Types parameterized by values like $\text{Vector}(n, A)$
- **Equality Types:** Path equality types $a =_A b$

5.2. Pi Types (Dependent Functions)

$(\rightarrow S \text{ (Maybe Any } S)))$; pop

Pi type dependent function

Pi types generalize function types to allow the result type to depend on the parameter value:

$$\begin{aligned} \text{pi-type} &\rightarrow (\text{Pi } (\text{parameter type}) \text{ body-type}) \\ &\mid ((\text{parameter} : \text{type}) \rightarrow \text{body-type}) \end{aligned}$$

Example 5.2.1 (Pi Types).

```
;; Vector lookup with length-dependent result
(define vector-ref :: (Pi (n : Natural)
  (→ (Vector n Any)
    (i : (Range 0 n))
    Any)))

;; Type-level computation in return type
(define replicate :: (Pi (n : Natural) (A : Type)
  (→ A (Vector n A))))

;; Dependent elimination for natural numbers
(define nat-elim :: (Pi (P : (→ Natural Type))
  (→ P(Zero)
    ((n : Natural) → P(n) →
      P(Succ(n)))
    (n : Natural) → P(n))))
```

5.3. Sigma Types (Dependent Pairs)

Sigma type dependent pair

Sigma types generalize pair types where the second component's type depends on the first:

$$\begin{aligned} \text{sigma-type} &\rightarrow (\text{Sigma } (\text{parameter type}) \text{ body-type}) \\ &\mid ((\text{parameter} : \text{type}) \times \text{body-type}) \end{aligned}$$

Example 5.3.1 (Sigma Types).

```
;; Dependent pair with length-indexed vector
(define sized-vector :: (Sigma (n : Natural) (Vector n
  String)))

;; Create a sized vector
(define my-data :: sized-vector
  (pair 3 #("hello" "world" "!")))

;; Access components
(define get-size :: (→ sized-vector Natural))
(define (get-size sv) (proj1 sv))

(define get-vector :: (Pi (sv : sized-vector)
  (Vector (get-size sv) String)))
(define (get-vector sv) (proj2 sv))

;; Existential type (special case of Sigma)
(define abstract-stack :: (Sigma (S : Type)
  (& (→ Unit S)
    ; empty
    (→ S Any S)
    ; push
```

5.4. Refinement Types

refinement type predicate

Refinement types restrict base types with logical predicates:

$$\begin{aligned} \text{refinement-type} &\rightarrow (\text{Refinement } (\text{variable base-type}) \text{ predicate}) \\ &\mid \{ \text{variable} : \text{base-type} \mid \text{predicate} \} \end{aligned}$$

Example 5.4.1 (Refinement Types).

```
;; Non-zero numbers
(define NonZero (Refinement (x Number) (not (= x 0))))

;; Safe division
(define safe-divide :: (→ Number NonZero Number))
(define (safe-divide x y) (/ x y))

;; Sorted lists
(define SortedList
  (Refinement (lst (List Number))
    (sorted? lst)))

;; Insert maintains sortedness
(define sorted-insert :: (→ Number SortedList
  SortedList))

;; Even numbers
(define Even (Refinement (n Integer) (= (modulo n 2) 0)))

;; Odd numbers
(define Odd (Refinement (n Integer) (= (modulo n 2) 1)))

;; Adding evens gives even
(define add-even :: (→ Even Even Even))
```

5.5. Indexed Types

indexed type type family

Indexed types are parameterized by values at the type level:

Example 5.5.1 (Indexed Types).

```
;; Length-indexed vectors
(define-indexed-type (Vector n A)
  (vector-data :: (RawVector A))
  (vector-length :: n))

;; Type-safe vector operations
(define make-vector :: (Pi (n : Natural) (A : Type) (→ A
  (Vector n A))))

(define vector-append :: (Pi (m n : Natural) (A : Type)
  (→ (Vector m A) (Vector n A)
    (Vector (+ m n) A))))
```

```

;; Finite sets indexed by size
(define-indexed-type (FinSet n)
  (elements :: (Vector n Natural)))

;; Matrix with dimensions
(define-indexed-type (Matrix rows cols A)
  (data :: (Vector (* rows cols) A)))

;; Type-safe matrix multiplication
(define matrix-multiply :: (Pi (m n p : Natural) (A :
  Type)
    (=> (Ring A)
      (-> (Matrix m n A)
        (Matrix n p A)
          (Matrix m p A))))))

```

5.6. Equality Types

equality type path equality

Equality types represent proofs that two values are equal:

$$\begin{array}{l} \text{equality-type} \rightarrow (\text{term1} = \text{term2}) \\ \quad | (\text{term1} =_{\text{type}} \text{term2}) \end{array}$$

Example 5.6.1 (Equality Types).

```

;; Reflexivity proof
(define refl :: (Pi (A : Type) (x : A) (x = x)))
(define (refl A x) (equal-refl x))

;; Symmetry proof
(define sym :: (Pi (A : Type) (x y : A) (-> (x = y) (y =
  x))))
(define (sym A x y prf) (equal-sym prf))

;; Transitivity proof
(define trans :: (Pi (A : Type) (x y z : A)
  (-> (x = y) (y = z) (x = z))))
(define (trans A x y z prf1 prf2) (equal-trans prf1 prf2))

;; Substitution (congruence)
(define subst :: (Pi (A B : Type) (f : (-> A B)) (x y : A)
  (-> (x = y) ((f x) = (f y)))))
(define (subst A B f x y prf) (equal-cong f prf))

;; Arithmetic equality proofs
(define plus-zero :: (Pi (n : Natural) ((+ n 0) = n)))
(define plus-succ :: (Pi (n m : Natural) ((+ n (Succ m))
  = (Succ (+ n m)))))

```

5.7. Universe Hierarchy

universe type of types

To avoid Russell's paradox, types are organized in a hierarchy of universes:

Example 5.7.1 (Universe Hierarchy).

```
;; Universe levels
```

```

Type0 :: Type1      ; Base types live in Type0
Type1 :: Type2      ; Type0 itself has type Type1
Type2 :: Type3      ; And so on...

```

```

;; Base types
Number :: Type0
String :: Type0
Boolean :: Type0

```

```

;; Type constructors
List :: (-> Type0 Type0)
Vector :: (-> Natural Type0 Type0)

```

```

;; Polymorphic types
(All (A : Type0) (-> A A)) :: Type0      ; Monomorphic
polymorphism
(All (A : Type1) (-> A A)) :: Type1      ; Type-polymorphic

```

```

;; Universe polymorphism
(define id :: (Pi (l : Level) (A : Type l) (-> A A)))
(define (id l A x) x)

```

5.8. Proof Terms and Tactics

proof term tactic

Dependent types enable proof-carrying code through the Curry-Howard correspondence:

5.9. Basic Proof Construction

Example 5.9.1 (Basic Proofs).

```

;; Theorem: addition is associative
(define plus-assoc :: (Pi (m n p : Natural)
  (((+ (+ m n) p) = (+ m (+ n
    p))))))

;; Proof by induction on m
(define (plus-assoc m n p)
  (induction m
    ;; Base case: m = 0
    (base-case
      (trans (+ (+ 0 n) p) ; Start with goal
        (+ n p) ; Simplify (+ 0 n) =
          n
        (+ 0 (+ n p)) ; Target form
        (plus-zero n) ; (+ 0 n) = n
        (sym (plus-zero (+ n p)))) ; (+ 0 (+ n p))
        = (+ n p)

    ;; Inductive case: m = (Succ k)
    (inductive-step k ih
      (trans (+ (+ (Succ k) n) p) ; Start
        (+ (Succ (+ k n)) p) ; Definition of +
        (Succ (+ (+ k n) p)) ; Definition of +
        (Succ (+ k (+ n p))) ; Induction
          hypothesis
        (+ (Succ k) (+ n p)) ; Definition of +
        (plus-succ-left k n) ; Lemma
        (plus-succ k (+ n p)) ; Another lemma
        ih)))) ; Use IH

```

5.10. Proof Tactics

High-level tactics automate common proof patterns:

Example 5.10.1 (Proof Tactics).

```
;; Define a theorem with tactic proof
(theorem plus-comm :: (Pi (m n : Natural) ((+ m n) = (+ n
  m))))
  (proof-by-induction m
    (base-case
      (by-simplification))
    (inductive-case k ih
      (by-rewriting
        (plus-succ-right n k)
        ih
        (plus-succ-left k n))))))

;; Automated tactics
(theorem list-length-app :: (Pi (A : Type) (xs ys : (List
  A)))
  ((length (append xs ys)) =
    (+ (length xs) (length
      ys))))
  (auto-induction xs) ; Automatic induction

;; Interactive proof development
(theorem insertion-sort-correct :: (Pi (xs : (List
  Natural))
  (sorted?
    (insertion-sort
      xs))))
  (interactive-proof
    ; Proof script will be filled in interactively
  ))
```

5.11. Program Extraction

program extraction Curry-Howard

Constructive proofs can be automatically extracted to executable programs:

Example 5.11.1 (Program Extraction).

```
;; Theorem with computational content
(theorem exists-sqrt :: (Pi (x : Positive)
  (Exists (y : Positive) ((* y
    y) = x))))
  (proof
    ; Constructive proof using Newton's method
    (let ((y (newton-sqrt x)))
      (pair y (newton-sqrt-correct x)))))

;; Extract the computational part
(define sqrt-function (extract exists-sqrt))
;; sqrt-function :: (-> Positive Positive)

;; Theorem: every list can be sorted
(theorem sort-exists :: (Pi (A : Type) (ord : (Order A))
  (xs : (List A))
  (Exists (ys : (List A))
    (& (sorted? ys)
      (permutation? xs ys)))))
  (proof
```

```
;; Constructive proof using merge sort
(let ((ys (merge-sort xs)))
  (pair ys (merge-sort-correct xs)))))

;; Extract sorting algorithm
(define sort-algorithm (extract sort-exists))
;; sort-algorithm :: (Pi (A : Type) (-> (Order A) (List
  A) (List A)))

;; Verified compiler optimization
(theorem beta-reduce-correct ::
  (Pi (A B : Type) (f : (-> A B)) (x : A)
    (((lambda (y) (f y)) x) = (f x))))
  (proof (refl (f x))))

(define beta-reduce (extract beta-reduce-correct))
;; beta-reduce :: (Pi (A B : Type) (-> (-> A B) A B))
```

5.12. Type-Level Programming

type-level programming type family

Dependent types enable computation at the type level:

Example 5.12.1 (Type-Level Programming).

```
;; Type-level natural numbers
(define-type Nat
  Zero :: Nat
  (Succ Nat) :: Nat)

;; Type-level addition
(define-type-function (Plus m n)
  (Plus Zero n) = n
  (Plus (Succ m) n) = (Succ (Plus m n)))

;; Type-level comparison
(define-type-function (Less-Than m n)
  (Less-Than Zero (Succ n)) = True
  (Less-Than (Succ m) Zero) = False
  (Less-Than (Succ m) (Succ n)) = (Less-Than m n)
  (Less-Than m m) = False)

;; Statically sized data structures
(define-indexed-type (Array n A)
  (data :: (RawArray A))
  (size :: n)
  (constraint :: (= (raw-array-size data) n)))

;; Type-safe array operations
(define array-get :: (Pi (n : Nat) (A : Type)
  (-> (Array n A)
    (i : Nat)
    (constraint :: (Less-Than i
      n) = True)
    A)))

;; Type-level lists
(define-type (TypeList k)
  TypeNil :: (TypeList k)
  (TypeCons k (TypeList k)) :: (TypeList k))

;; Heterogeneous lists with type-level schema
(define HList :: (-> (TypeList Type) Type)
  (define-type-function HList
    (HList TypeNil) = Unit
```

```

(HList (TypeCons A As)) = (Pair A (HList As)))

;; Example usage
(define schema :: (TypeList Type)
  (TypeCons String (TypeCons Natural (TypeCons Boolean
    TypeNil))))

(define person :: (HList schema)
  (pair "Alice" (pair 30 (pair #t unit))))

```

5.13. Totality and Termination

totality termination

The dependent type system ensures all functions are total:

5.14. Structural Recursion

Example 5.14.1 (Structural Recursion).

```

;; Structural recursion on natural numbers (always
terminates)
(define plus :: (-> Natural Natural Natural))
(define (plus m n)
  (match m
    (Zero n)
    ((Succ k) (Succ (plus k n)))) ; Recursive call on
smaller argument

;; Structural recursion on lists
(define length :: (Pi (A : Type) (-> (List A) Natural)))
(define (length A lst)
  (match lst
    (() Zero)
    ((cons x xs) (Succ (length A xs)))) ; Recursive
call on tail

```

5.15. Well-Founded Recursion

Example 5.15.1 (Well-Founded Recursion).

```

;; Division using well-founded recursion on
decreasing measure
(define div :: (-> Natural Natural Natural))
(define (div m n)
  (termination-proof (measure m)) ; Prove m decreases
  (if (< m n)
    Zero
    (Succ (div (- m n) n)))) ; m - n < m when n
> 0

;; Ackermann function with lexicographic ordering
(define ack :: (-> Natural Natural Natural))
(define (ack m n)
  (termination-proof (lex-measure m n)) ; (m,n)
decreases lexicographically
  (match m
    (Zero (Succ n))
    ((Succ k) (match n

```

```

(Zero (ack k (Succ Zero)))
((Succ j) (ack k (ack (Succ k) j))))))

```

5.16. Coinduction and Corecursion

For infinite data structures, Lambdust supports coinduction:

Example 5.16.1 (Coinductive Types).

```

;; Coinductive streams
(define-coinductive-type (Stream A)
  (stream-head :: A)
  (stream-tail :: (Stream A)))

;; Corecursive stream generation
(define nats :: (Stream Natural)
  (corecursion 0
    (lambda (n)
      (stream n (+ n 1)))))

;; Stream map (productive corecursion)
(define stream-map :: (Pi (A B : Type) (-> (-> A B)
  (Stream A) (Stream B)))
(define (stream-map A B f s)
  (corecursion s
    (lambda (s)
      (stream (f (stream-head s))
        (stream-tail s)))))

```

5.17. Integration with Gradual Typing

gradual dependent types

Dependent types integrate smoothly with the gradual type system:

Example 5.17.1 (Gradual Dependent Types).

```

;; Gradual refinement from dynamic to dependent
;; Level 1: Dynamic
(define vector-get
  (lambda (vec i) (vector-ref vec i)))

;; Level 2: Simple types
(define vector-get :: (-> (Vector Any) Natural Any)
  (lambda (vec i) (vector-ref vec i)))

;; Level 3: Bounded index
(define vector-get :: (-> (vec : (Vector Any))
  (i : Natural)
  (constraint :: (< i
    (vector-length vec)))
  Any)
  (lambda (vec i) (vector-ref vec i)))

;; Level 4: Full dependent types
(define vector-get :: (Pi (n : Natural) (A : Type)
  (-> (Vector n A) (Fin n) A))
  (lambda (n A vec i) (vector-ref vec i)))

;; Automatic contract generation at boundaries

```

```
(define untyped-caller
  ;; This automatically gets contract checking
  (vector-get some-vector some-index))
```

5.18. Standard Dependent Types

Lambdust provides a standard library of dependent types:

Example 5.18.1 (Standard Dependent Types).

```
;; Finite sets
(define Fin :: (-> Natural Type))
(define-indexed-type (Fin n)
  (value :: Natural)
  (constraint :: (< value n)))

;; Bounded natural numbers
(define (Range lower upper)
  (Refinement (x Natural) (and (<= lower x) (<= x
    upper)))))

;; Non-empty lists
(define (NonEmptyList A)
  (Refinement (xs (List A)) (not (null? xs))))

;; Sorted vectors
(define (SortedVector n A)
  (Refinement (v (Vector n A)) (sorted-vector? v)))

;; Matrices with matching dimensions
(define matrix-multiply :: (Pi (m n p : Natural)
  (-> (Matrix m n Number)
    (Matrix n p Number)
    (Matrix m p Number))))

;; Red-black trees with color invariants
(define-indexed-type (RBTree A color height)
  (node-data :: A)
  (left :: (Maybe (RBTree A black-or-red height)))
  (right :: (Maybe (RBTree A black-or-red height)))
  (color-invariant :: (valid-rb-coloring? color left
    right))
  (height-invariant :: (valid-rb-height? height left
    right)))
```

5.19. Error Messages and Inference

The dependent type checker provides informative error messages:

Example 5.19.1 (Dependent Type Errors).

```
;; Index out of bounds
(vector-get #(1 2 3) 5)
;; Error: Index out of bounds
;; Expected: (Fin 3)
;; Actual: 5
;; Note: Vector has length 3, but index 5 >= 3

;; Refinement violation
(define positive-only :: (-> Positive Natural))
(positive-only -5)
```

```
;; Error: Refinement type violation
;; Expected: Positive (x > 0)
;; Actual: -5
;; Constraint violated: -5 > 0 is false

;; Proof obligation not discharged
(define (unsafe-div :: (-> Natural Natural Natural))
  (lambda (x y) (/ x y)))
;; Error: Missing proof obligation
;; Need to prove: y is not equal 0
;; Suggestion: Use (y : NonZero) instead of (y : Natural)
```

The dependent type system provides powerful tools for program verification while maintaining integration with Lambdust’s gradual typing approach, allowing developers to choose their level of specification precision.

6. Effect System

Lambdust provides structured side effect management through algebraic effects with handlers, revolutionizing how programs handle I/O, exceptions, state, and other side effects. Traditional approaches to side effects often lead to “callback hell,” complex error handling, and code that is difficult to reason about. Algebraic effects solve these problems by making effects explicit, composable, and locally scoped.

Unlike traditional exception handling, which can only propagate errors upward, algebraic effects allow handlers to resume computation with different values, enabling powerful abstractions like backtracking, cooperative threading, and probabilistic programming. Unlike monads, which can be difficult to compose and require complex monad transformer stacks, algebraic effects compose naturally and can be added or removed without restructuring entire programs.

This system enables safe, composable, and reasoning-friendly side effects while maintaining complete R7RS compatibility. Programs can use traditional Scheme I/O operations unchanged, or adopt structured effects for better composability and testing.

6.1. Effects Overview

The effect system treats side effects as first-class language constructs that can be declared, performed, handled, and composed in a principled manner:

- **Effect Declaration:** Effects are declared with operation signatures, making all possible side effects explicit in the type system. This eliminates hidden dependencies and makes code more predictable. For example, a `FileSystem` effect might declare `read-file` and `write-file` operations with their respective type signatures.

- **Effect Performance:** Code performs effects using the `perform` construct, which suspends the current computation and transfers control to the nearest matching handler. This is more flexible than exceptions because it allows bidirectional communication between the performing code and the handler.
- **Effect Handling:** Effects are intercepted and handled with full resumption capabilities. Handlers can examine the operation being performed, its arguments, and the continuation representing the rest of the computation. They can then decide whether to resume with a value, perform a different effect, or propagate the effect to outer handlers.
- **Effect Tracking:** The type system tracks which effects computations may perform, preventing effect leakage and enabling local reasoning. Functions that don't perform any effects are guaranteed to be pure, while functions that perform effects have those effects recorded in their type signatures.
- **Effect Composition:** Multiple effects compose naturally without the complexity of monad transformers. A computation can use file I/O, state manipulation, and exception handling simultaneously, and handlers can be nested and combined in arbitrary ways to implement complex control flow patterns.

6.2. Effect Declaration

effect declaration `define-effect`

Effects are declared by specifying their operations and signatures:

effect-declaration → (define-effect *effect-name operation-signature*)

operation-signature → (operation-name :: function-type)

Example 6.2.1 (Effect Declarations).

```
;; State effect with get and set operations
(define-effect State
  (get :: (-> Unit Any))
  (set :: (-> Any Unit)))

;; Exception effect with raise operation
(define-effect Exception
  (raise :: (-> String Nothing)))

;; I/O effect with read and write operations
(define-effect IO
  (read-line :: (-> Unit String))
  (write-line :: (-> String Unit))
  (read-file :: (-> String String))
  (write-file :: (-> String String Unit)))

;; Nondeterminism effect
(define-effect Choose
```

```
(choose :: (-> (List Any) Any))
(fail :: (-> Unit Nothing)))

;; Async effect for concurrent operations
(define-effect Async
  (async :: (Pi (A : Type) (-> (-> Unit A) (Future A))))
  (await :: (Pi (A : Type) (-> (Future A) A))))
```

6.3. Effect Performance

perform effect performance

Effects are performed using the `perform` construct:

effect-performance → (perform *effect-name operation argument**)

Example 6.3.1 (Effect Performance).

```
;; Using the State effect
(define (increment)
  (let ((current (perform State get)))
    (perform State set (+ current 1))))

;; Using the Exception effect
(define (safe-divide x y)
  (if (= y 0)
      (perform Exception raise "Division by zero")
      (/ x y)))

;; Using the Choose effect for nondeterminism
(define (choose-positive-number)
  (let ((n (perform Choose choose '(1 2 3 -1 -2))))
    (if (> n 0)
        n
        (perform Choose fail))))

;; Using multiple effects
(define (interactive-counter)
  (perform IO write-line "Current count:")
  (let ((current (perform State get)))
    (perform IO write-line (number->string current))
    (perform IO write-line "Enter increment:"))
  (let ((input (perform IO read-line)))
    (let ((increment (string->number input)))
      (if increment
          (perform State set (+ current increment))
          (perform Exception raise "Invalid number"))))))
```

6.4. Effect Handlers

handle effect handler continuation

Effects are handled using the `handle` construct with resumable continuations:

effect-handling → (handle *computation* *effect-handler**)

effect-handler → (*effect-name* *operation-clause**)

operation-clause → (*operation-name* (*parameter**) resume)

Example 6.4.1 (Effect Handlers).

```
;; Handle State effect with mutable reference
(define (run-stateful initial computation)
  (let ((state (make-box initial)))
    (handle computation
      (State
        ((get () k)
         (k (unbox state)))
        ((set new-value k)
         (set-box! state new-value)
         (k 'unit))))))

;; Handle Exception effect with error recovery
(define (try-catch computation error-handler)
  (handle computation
    (Exception
      ((raise message k)
       (error-handler message)))))

;; Handle Choose effect with backtracking
(define (run-nondeterministic computation)
  (handle computation
    (Choose
      ((choose choices k)
       (map k choices)) ; Try all choices
      ((fail () k)
       '()))) ; Return empty list

;; Multiple effect handlers
(define (run-program computation)
  (let ((state (make-box 0)))
    (handle
      (handle computation
        (Exception
          ((raise msg k)
           (begin (display "Error: ") (display msg)
                  (newline)
                  #f))))
      (State
        ((get () k) (k (unbox state)))
        ((set val k) (set-box! state val) (k 'unit)))
      (IO
        ((read-line () k) (k (read-line)))
        ((write-line text k) (begin (display text)
                                    (newline) (k 'unit)))))))
```

6.5. Effect Types

effect type effectful function

The type system tracks effects in function types:

effectful-type → (*parameter-types* $\succ$ [*effect-set*] *return-type*)
 | (*parameter-types* → *return-type*)

effect-set → {*effect-name**}

Example 6.5.1 (Effect Types).

```
;; Pure function (no effects)
(define add :: (-> Number Number Number))

;; Function with State effect
(define increment :: (Unit -> Unit ~> {State} Unit))

;; Function with multiple effects
(define interactive-program :: (-> Unit ~> {State, IO,
Exception} Unit))

;; Higher-order function preserving effects
(define twice :: (Pi (A : Type) (E : EffectSet)
  (-> (-> A ~> E A) A ~> E A)))
(define (twice E f x) (f (f x)))

;; Effect polymorphic function
(define (map-with-effects :: (Pi (A B : Type) (E :
EffectSet)
  (-> (-> A ~> E B) (List
A) ~> E (List B))))
  (lambda (f lst)
    (if (null? lst)
      '()
      (cons (f (car lst))
            (map-with-effects f (cdr lst))))))
```

6.6. Built-in Effects

built-in effects

Lambdust provides several built-in effects:

6.7. State Effect

Example 6.7.1 (State Effect Usage).

```
;; Counter with state effect
(define counter-program :: (-> Unit ~> {State} Natural))
(define (counter-program)
  (perform State set 0)
  (perform State set (+ (perform State get) 1))
  (perform State set (+ (perform State get) 1))
  (perform State get))
```

```
;; Run with initial state
(define result (run-stateful 0 counter-program)) ;
Returns 2
```

```
;; State transformer monad-style
(define (state-computation :: (-> Number ~> {State}
Number))
  (lambda (x)
    (let ((current (perform State get)))
      (perform State set (+ current x))
      (perform State get))))
```

6.8. Exception Effect

Example 6.8.1 (Exception Effect Usage).

```
;; Safe division with exceptions
(define (safe-operations :: (-> Number Number ~>
  {Exception} Number))
  (lambda (x y)
    (let ((result1 (safe-divide x y)))
      (let ((result2 (safe-divide result1 2)))
        result2))))

;; Exception handling with multiple handlers
(define (robust-computation)
  (try-catch
    (lambda ()
      (let ((input (string->number (read-line))))
        (if input
          (safe-divide 100 input)
          (perform Exception raise "Invalid input")))))
    (lambda (msg)
      (begin (display "Recovered from error: ") (display
        msg) (newline)
        0)))))
```

6.9. I/O Effect

Example 6.9.1 (I/O Effect Usage).

```
;; File processing with I/O effects
(define (process-file :: (-> String String ~> {IO,
  Exception} Unit))
  (lambda (input-file output-file)
    (let ((content (perform IO read-file input-file)))
      (let ((processed (string-upcase content)))
        (perform IO write-file output-file processed)))))

;; Interactive program
(define (interactive-shell :: (-> Unit ~> {IO, State}
  Unit))
  (lambda ()
    (perform IO write-line "Enter command:")
    (let ((command (perform IO read-line)))
      (cond
        ((equal? command "quit") 'done)
        ((equal? command "status")
         (let ((count (perform State get)))
           (perform IO write-line (string-append "Count: "
            (number->string count))))
         (interactive-shell)))
        (else
         (perform State set (+ (perform State get) 1))
         (interactive-shell))))))
```

6.10. Effect Composition

effect composition effect lifting

Effects compose naturally and can be lifted between contexts:

Example 6.10.1 (Effect Composition).

```
;; Computation using multiple effects
(define (complex-computation :: (-> Unit ~> {State, IO,
  Exception, Choose} Natural))
  (lambda ()
    (perform IO write-line "Starting computation")
    (let ((choice (perform Choose choose '(1 2 3 4 5))))
      (perform State set choice)
      (if (even? choice)
        (begin
          (perform IO write-line "Even choice")
          (perform State get))
        (perform Exception raise "Odd choice not
          supported")))))

;; Effect lifting
(define (lift-io-to-full :: (-> (-> Unit ~> {IO} A) (->
  Unit ~> {State, IO, Exception} A)))
  (lambda (computation)
    (lambda () (computation))))

;; Effect combination
(define (combine-effects computation1 computation2)
  (lambda ()
    (let ((result1 (computation1)))
      (let ((result2 (computation2)))
        (list result1 result2)))))

;; Higher-order effect handling
(define (with-logging :: (Pi (E : EffectSet) (A : Type)
  (-> (-> Unit ~> E A) (-> Unit ~> {IO, E} A))))
  (lambda (E A computation)
    (lambda ()
      (perform IO write-line "Starting operation")
      (let ((result (computation)))
        (perform IO write-line "Operation completed"
          result)))))
```

6.11. Effect Scoping

effect scoping effect masking

Effects can be scoped and masked for fine-grained control:

Example 6.11.1 (Effect Scoping).

```
;; Mask certain effects in a computation
(define (mask-io :: (Pi (E : EffectSet) (A : Type)
  (-> (-> Unit ~> {IO, E} A) (-> Unit ~> E A))))
  (lambda (E A computation)
    (lambda ()
      ;; IO effects are handled internally and not propagated
      (handle (computation)
        IO
        ((read-line () k) (k ""))
        ((write-line text k) (k 'unit))))))

;; Local state that doesn't escape
(define (with-local-state :: (Pi (E : EffectSet) (A :
  Type)
  (-> (-> Unit ~> {State, E} A) (-> Unit ~> E A))))
```

```

(lambda (E A initial computation)
  (lambda ()
    (run-stateful initial computation))))

;; Effect restriction - only allow certain operations
(define (restrict-io :: (-> (-> Unit ~> {IO} A) (-> Unit
  ~> {RestrictedIO} A)))
  (lambda (computation)
    (lambda ()
      (handle (computation)
        (IO
          ((read-line () k) (perform RestrictedIO
            safe-read-line k))
          ((write-line text k) (perform RestrictedIO
            safe-write-line text k))
          ((read-file path k) (perform Exception raise
            "File read not allowed"))
          ((write-file path content k) (perform Exception
            raise "File write not allowed"))))))))

```

6.12. Monadic Effects

monad monadic effect

Effects naturally form monads and can interoperate with monadic code:

Example 6.12.1 (Monadic Effects).

```

;; State monad using effects
(define-monad (State s)
  (return :: (Pi (A : Type) (-> A (State s A))))
  (bind :: (Pi (A B : Type) (-> (State s A) (-> A (State
    s B)) (State s B)))))

(define (return-state x)
  (lambda () x))

(define (bind-state m f)
  (lambda ()
    (let ((a (m)))
      ((f a)))))

;; Do-notation for effects
(define-syntax do-effects
  (syntax-rules (<-)
    ((do-effects expr)
      ((do-effects (var <- computation) rest ...)
        (perform-bind computation (lambda (var) (do-effects
          rest ...))))))
    ((do-effects computation rest ...)
      (begin computation (do-effects rest ...))))))

;; Usage with do-notation
(define counter-with-do :: (-> Unit ~> {State} Natural))
(define (counter-with-do)
  (do-effects
    (perform State set 0)
    (count1 <- perform State get)
    (perform State set (+ count1 1))
    (count2 <- perform State get)
    (perform State set (+ count2 1))
    (perform State get)))

;; Effect/monad transformer stack
(define-effect-transformer (StateT s m)

```

```

(run-state-t :: (Pi (A : Type) (-> (StateT s m A) s (m
  (Pair A s)))))

;; Lift between effect layers
(define (lift-effect :: (Pi (E1 E2 : EffectSet) (A : Type)
  (-> (-> Unit ~> E1 A) (-> Unit
    ~> {E1, E2} A))))
  (lambda (E1 E2 A computation)
    (lambda () (computation))))

```

6.13. Effect Inference

effect inference

The type system automatically infers effect requirements:

Example 6.13.1 (Effect Inference).

```

;; Effect inference in action
(define inferred-function :: (Pi (A : Type) (-> Natural
  ~> {State, IO} Unit))
  (lambda (n)
    (perform IO write-line "Processing...") ; Adds IO
      to effect set
    (perform State set n) ; Adds
      State to effect set
    (if (> n 10)
      (perform IO write-line "Large number") ; IO
        already in set
      'unit)))

;; Higher-order function effect inference
(define (map-with-side-effects f lst) ; Inferred
  effects from f
  (if (null? lst)
    '()
    (begin
      (f (car lst)) ; f's effects
      propagate here
      (cons (f (car lst))
        (map-with-side-effects f (cdr lst))))))

;; Effect constraints in polymorphic functions
(define (conditional-effect :: (Pi (A : Type) (E :
  EffectSet)
  (-> Boolean (-> Unit ~>
    E A) (-> Unit ~> {
      A ~> E A})))
  (lambda (A E condition effectful-branch pure-branch)
    (if condition
      (effectful-branch) ; May have effects E
      (pure-branch)))) ; No effects

;; Usage determines concrete effects
(define example-usage
  (conditional-effect Boolean {IO}
    #t
    (lambda () (perform IO write-line
      "Hello")) ; IO effect
    (lambda () #f)))

; No
effects

```

6.14. Effect Safety

effect safety purity

The effect system ensures effect safety through static checking:

Example 6.14.1 (Effect Safety).

```
;; Pure functions cannot perform effects
(define pure-function :: (-> Natural Natural))
(define (pure-function n)
  ;; This would be a compile-time error:
  ;; (perform IO write-line "Hello") ; Error: IO effect
  ;; not allowed in pure context
  (+ n 1))

;; Effect boundaries are enforced
(define (safe-wrapper :: (-> Natural Natural))
  (lambda (n)
    (handle
      ;; This computation may have IO effects
      (lambda ()
        (perform IO write-line "Processing...")
        (+ n 1))
      IO
      ;; But we handle them, so safe-wrapper is pure
      ((write-line text k) (k 'unit))))))

;; Effect masking prevents effect escape
(define (contained-effects :: (-> Natural Natural))
  (lambda (n)
    (with-local-state 0 ; State effects don't escape
      (lambda ()
        (perform State set n)
        (let ((doubled (perform State get)))
          (perform State set (* doubled 2))
          (perform State get))))))

;; Effect tracking in data structures
(define effectful-list :: (List (-> Unit ~> {IO} Unit)))
(define pure-list :: (List (-> Unit Unit)))

;; Cannot mix effectful and pure without explicit
;; conversion
;; (append effectful-list pure-list) ; Type error

;; Must use effect lifting
(define mixed-list
  (append effectful-list
    (map (lift-effect {} {IO}) pure-list)))
```

6.15. Performance Considerations

effect performance effect optimization

The effect system is designed for efficiency:

Example 6.15.1 (Effect Optimization).

```
;; Zero-cost abstractions for pure code
(define (pure-computation x y) ; No effect overhead
  (+ (* x x) (* y y)))

;; Effect specialization
```

```
(define (specialized-state-code :: (-> Unit ~> {State}
  Natural))
  ;; State operations can be compiled to direct memory
  ;; access
  (lambda ()
    (perform State get)))

;; Effect fusion for multiple operations
(define (fused-operations :: (-> Unit ~> {State} Unit))
  (lambda ()
    ;; Multiple state operations can be optimized together
    (perform State set 0)
    (perform State set (+ (perform State get) 1))
    (perform State set (+ (perform State get) 1))))

;; Handler inlining for known effects
(define optimized-result
  ;; When handler is statically known, can be inlined
  (run-stateful 0
    (lambda ()
      (perform State set 42)
      (perform State get)))))
```

6.16. R7RS Integration

R7RS compatibility effect compatibility

Effects integrate seamlessly with R7RS code:

R7RS Compatibility 6.16.1 (R7RS Effect Compatibility). All R7RS procedures are treated as potentially effectful with a universal *Dynamic* effect, ensuring compatibility while allowing gradual adoption of structured effects.

Example 6.16.2 (R7RS Integration).

```
;; R7RS procedures have Dynamic effect
(define r7rs-display :: (-> Any ~> {Dynamic} Unit))
(define r7rs-read :: (-> Unit ~> {Dynamic} Any))

;; Gradual migration from R7RS to structured effects
;; Step 1: Use R7RS procedures directly
(define (step1-program)
  (display "Hello") ; Dynamic effect
  (newline)) ; Dynamic effect

;; Step 2: Add effect annotations
(define (step2-program :: (-> Unit ~> {Dynamic} Unit))
  (lambda ()
    (display "Hello")
    (newline)))

;; Step 3: Use structured effects
(define (step3-program :: (-> Unit ~> {IO} Unit))
  (lambda ()
    (perform IO write-line "Hello")))

;; Effect boundaries handle R7RS integration
(define (mixed-program :: (-> Unit ~> {IO} Unit))
  (lambda ()
    (perform IO write-line "Structured output")
    (handle
      (lambda () (display "R7RS output")) ; Dynamic
      effect
      (Dynamic
        ;; Convert R7RS effects to structured effects
```

```
((io-operation args k) (perform IO write-line
  (apply format args)) (k 'unit))))))
```

The effect system provides powerful abstractions for managing side effects while maintaining the flexibility and compatibility that makes Scheme practical for real-world programming.

7. Concurrency System

Lambdust provides lightweight, fault-tolerant concurrency through an actor model with message passing, futures for asynchronous computation, and software transactional memory for shared state management.

7.1. Concurrency Overview

The concurrency system is built on several key abstractions:

- **Actors:** Lightweight concurrent entities that communicate via messages
- **Futures:** Asynchronous computations with eventual values
- **Channels:** Typed communication channels for CSP-style concurrency
- **STM:** Software Transactional Memory for composable shared state
- **Parallelism:** Data parallel operations and work-stealing

7.2. Actor Model

actor spawn send receive

Actors are the fundamental unit of concurrency in Lambdust:

7.3. Actor Creation

Syntax: (spawn computation)

Creates a new actor running *computation* and returns its process identifier (PID).

Example 7.3.1 (Actor Creation).

```
;; Simple counter actor
(define (counter-actor initial-count)
  (letrec ((loop (lambda (count)
    (receive
      (('get sender)
       (send sender count)
       (loop count))
      (('increment sender)
       (send sender (+ count 1))
       (loop (+ count 1)))
      (('set value sender)
       (send sender 'ok)
       (loop value))))))
    (loop initial-count)))

;; Spawn the actor
(define counter-pid (spawn (counter-actor 0)))

;; Echo actor that reflects messages
(define echo-pid
  (spawn
    (letrec ((loop (lambda ()
      (receive
        ((msg sender)
         (send sender msg)
         (loop))))))
      (loop))))
```

7.4. Message Passing

Syntax: (send pid message)

Sends *message* asynchronously to the actor identified by *pid*.

Syntax: (receive (pattern body)*)

Receives messages matching patterns, blocking until a match is found.

Example 7.4.1 (Message Passing).

```
;; Send messages to counter actor
(send counter-pid `(get ,self))
(send counter-pid `(increment ,self))
(send counter-pid `(set 42 ,self))

;; Receive responses
(receive
  ((count) (display "Current count: ") (display count)
   (newline)))

;; Pattern matching in receive
(define (server-loop)
  (receive
    ;; Handle different request types
    (('add x y client)
```

```

(send client (+ x y))
(server-loop))

(('multiply x y client)
 (send client (* x y))
 (server-loop))

(('ping client)
 (send client 'pong)
 (server-loop))

(('shutdown)
 (display "Server shutting down") (newline))

;; Catch-all for unknown messages
((unknown-msg)
 (display "Unknown message: ") (display unknown-msg)
 (newline)
 (server-loop))))

;; Request-response pattern
(define (call pid request timeout)
  (send pid '(,request ,self))
  (receive
   ((response) response)
   timeout) ; Handle timeout

```

7.5. Actor Hierarchies and Supervision

supervision fault tolerance

Actors can supervise other actors for fault tolerance:

Example 7.5.1 (Actor Supervision).

```

;; Supervisor that manages worker actors
(define (supervisor-actor worker-count worker-fn)
  (let ((workers (map (lambda (i) (spawn (worker-fn i)))
                      (range 0 worker-count))))
    (letrec ((loop (lambda (active-workers)
                     (receive
                      ;; Worker crashed, restart it
                      (('EXIT pid reason)
                       (display "Worker crashed: ")
                       (display reason) (newline)
                       (let ((new-worker (spawn
                                           (worker-fn (length
                                                         active-workers))))
                         (loop (cons new-worker (remove
                                     pid active-workers)))))))
                     ;; Distribute work to workers
                     (('work task client)
                      (if (null? active-workers)
                          (send client
                               'no-workers-available)
                          (begin
                           (send (car active-workers)
                                '(,task ,client))
                           (loop (append (cdr
                                           active-workers) (list
                                                         (car
                                                         active-workers)))))))
                     ;; Shutdown all workers
                     (('shutdown)
                      (loop))))))

```

```

(for-each (lambda (worker) (send
                             worker 'shutdown))
          active-workers)
'supervisor-shutdown))))
(loop workers))))

;; Worker actor that can fail
(define (worker-actor id)
  (letrec ((loop (lambda ()
                   (receive
                    (('heavy-computation x client)
                     ;; Simulate potential failure
                     (if (= (random 10) 0)
                         (error "Worker failure")
                         (send client (* x x)))
                     (loop)))
                    (('shutdown)
                     'worker-shutdown))))))
    (loop)))

;; Create supervised worker pool
(define supervisor (spawn (supervisor-actor 3
                                           worker-actor)))

```

7.6. Futures and Promises

future promise async await

Futures provide asynchronous computation with eventual values:

7.7. Future Creation

Syntax: (future computation)

Creates a future that will eventually contain the result of *computation*.

Syntax: (promise)

Creates a promise/future pair for manual completion.

Example 7.7.1 (Futures).

```

;; Asynchronous computation
(define future-result
  (future
   (begin
    (sleep 1000) ; Simulate long computation
    (+ 20 22))))

;; Continue with other work
(display "Doing other work...") (newline)

```

```
;; Wait for result when needed
(define result (await future-result))
(display "Result: ") (display result) (newline)

;; Multiple futures
(define f1 (future (expensive-computation-1)))
(define f2 (future (expensive-computation-2)))
(define f3 (future (expensive-computation-3)))

;; Wait for all to complete
(define results (map await (list f1 f2 f3)))

;; Promise for manual completion
(define-values (promise resolver) (make-promise))

;; In another thread/actor
(spawn
  (lambda ()
    (let ((data (fetch-from-network)))
      (resolve! resolver data))))

;; Wait for the promise
(define network-data (await promise))
```

7.8. Future Composition

Example 7.8.1 (Future Composition).

```
;; Future chaining with then
(define chained-future
  (future-then
    (future (read-file "input.txt"))
    (lambda (content)
      (future (process-content content)))
    (lambda (processed)
      (future (write-file "output.txt" processed)))))

;; Future combination
(define combined-future
  (future-combine
    (list (future (computation-1))
          (future (computation-2))
          (future (computation-3)))
    (lambda (results)
      (apply + results))))

;; Racing futures (first to complete wins)
(define fastest-result
  (future-race
    (list (future (slow-network-request))
          (future (cache-lookup))
          (future (backup-computation)))))

;; Timeout handling
(define result-with-timeout
  (future-timeout
    (future (potentially-slow-operation))
    5000 ; 5 second timeout
    'timeout-value))

;; Error handling in futures
(define safe-future
  (future-catch
    (future (risky-computation))
    (lambda (error)
```

```
(begin
  (log-error error)
  'default-value)))
```

7.9. Channels

channel CSP

Channels provide CSP-style communication:

Example 7.9.1 (Channels).

```
;; Create typed channels
(define numbers-channel :: (Channel Natural))
(define numbers-channel (make-channel))

;; Producer process
(spawn
  (lambda ()
    (do ((i 0 (+ i 1)))
      ((>= i 10))
      (channel-put! numbers-channel i)
      (sleep 100))))

;; Consumer process
(spawn
  (lambda ()
    (let loop ()
      (let ((value (channel-get numbers-channel)))
        (display "Received: ") (display value) (newline)
        (loop)))))

;; Buffered channels
(define buffered-channel (make-channel 100)) ; Buffer
size 100

;; Select on multiple channels
(define (multi-channel-select ch1 ch2 ch3)
  (select
    ((ch1 value)
     (begin (display "From ch1: ") (display value)
              (newline)))
    ((ch2 value)
     (begin (display "From ch2: ") (display value)
              (newline)))
    ((ch3 value)
     (begin (display "From ch3: ") (display value)
              (newline))))
  (timeout 1000
    (display "Timeout occurred") (newline)))

;; Channel transformations
(define (channel-map f input-ch)
  (let ((output-ch (make-channel)))
    (spawn
      (lambda ()
        (let loop ()
          (let ((value (channel-get input-ch)))
            (channel-put! output-ch (f value))
            (loop))))
      output-ch))

;; Pipeline processing
(define (pipeline input-ch . transforms)
  (fold-left channel-map input-ch transforms))
```


7.10. Software Transactional Memory

STM transaction atomic

STM provides composable shared state management:

7.11. Transactional Variables

Example 7.11.1 (Transactional Variables).

```
;; Create transactional variables
(define account1 (make-tvar 1000)) ; Account with $1000
(define account2 (make-tvar 500))  ; Account with $500

;; Atomic transaction
(define (transfer amount from-account to-account)
  (atomic
    (let ((from-balance (tvar-read from-account))
          (to-balance (tvar-read to-account)))
      (if (>= from-balance amount)
          (begin
            (tvar-write! from-account (- from-balance
                                          amount))
            (tvar-write! to-account (+ to-balance amount))
            #t) ; Success
          #f))) ; Insufficient funds

;; Transfer money atomically
(if (transfer 200 account1 account2)
    (display "Transfer successful")
    (display "Transfer failed"))

;; Compose transactions
(define (multi-transfer transfers)
  (atomic
    (and-map (lambda (transfer-spec)
                (apply transfer transfer-spec))
              transfers)))

;; Retry on failure
(define (wait-for-funds account minimum)
  (atomic
    (let ((balance (tvar-read account)))
      (if (>= balance minimum)
          balance
          (retry)))) ; Retry when condition met
```

7.12. Transaction Composition

Example 7.12.1 (Transaction Composition).

```
;; Composable transactions
(define (deposit amount account)
  (atomic
    (let ((balance (tvar-read account)))
      (tvar-write! account (+ balance amount)))))

(define (withdraw amount account)
  (atomic
    (let ((balance (tvar-read account)))
      (if (>= balance amount)
          (begin
```

```
            (tvar-write! account (- balance amount))
            amount)
      (error "Insufficient funds")))))

;; Compose into larger transaction
(define (compound-transaction)
  (atomic
    ;; All or nothing - if any part fails, entire
    ;; transaction aborts
    (deposit 100 account1)
    (withdraw 50 account1)
    (deposit 50 account2)
    (let ((total (+ (tvar-read account1) (tvar-read
                                          account2))))
      (if (< total 1000)
          (error "Total too low"
                total))))))

;; Alternative transactions (OrElse)
(define (try-withdraw-from-either amount acc1 acc2)
  (or-else
    (withdraw amount acc1) ; Try first account
    (withdraw amount acc2))) ; Fallback to second
                             account

;; Conditional transactions
(define (conditional-transfer condition from to amount)
  (atomic
    (if (condition)
        (transfer amount from to)
        (retry)))) ; Wait for condition
```

7.13. Parallel Processing

parallel work stealing

Lambdust provides data parallelism and work-stealing:

Example 7.13.1 (Parallel Processing).

```
;; Parallel map
(define (parallel-map f lst)
  (let ((chunks (partition lst (processor-count))))
    (let ((futures (map (lambda (chunk) (future (map f
                                                       chunk)))
                        chunks)))
      (apply append (map await futures)))))

;; Parallel fold
(define (parallel-fold f identity lst)
  (if (<= (length lst) 100) ; Sequential threshold
      (fold-left f identity lst)
      (let ((mid (quotient (length lst) 2)))
        (let ((left (take lst mid))
              (right (drop lst mid)))
          (let ((left-future (future (parallel-fold f
                                                    identity left)))
                (right-future (future (parallel-fold f
                                                    identity right))))
            (f (await left-future) (await
                               right-future)))))))

;; Work-stealing pool
(define (work-stealing-map f lst)
  (let ((work-queue (make-work-queue lst))
        (result-queue (make-channel))
        (worker-count (processor-count)))
```



```

;; Spawn worker threads
(do ((i 0 (+ i 1)))
  ((>= i worker-count))
  (spawn
    (lambda ()
      (let loop ()
        (let ((work (work-queue-steal! work-queue)))
          (if work
              (begin
                (channel-put! result-queue (f work))
                (loop))
              'worker-done))))))

;; Collect results
(let ((results '()))
  (do ((i 0 (+ i 1)))
    ((>= i (length lst)) (reverse results))
    (set! results (cons (channel-get result-queue)
                        results))))

;; GPU parallel computation (when available)
(define (gpu-parallel-map f lst)
  (if (gpu-available?)
      (gpu-compute (gpu-compile-function f)
                    (list->gpu-array lst))
      (parallel-map f lst))) ; Fallback to CPU
                             parallelism

```

7.14. Distributed Computing

distributed location transparency

Actors support location transparency for distributed computing:

Example 7.14.1 (Distributed Computing).

```

;; Remote actor spawning
(define remote-node "worker-node-1.cluster.local")
(define remote-actor (spawn-remote remote-node
                                   (worker-computation)))

;; Location-transparent message passing
(send remote-actor `(process-data ,large-dataset ,self))

;; Distributed supervision
(define (distributed-supervisor nodes worker-fn)
  (let ((workers (map (lambda (node)
                        (spawn-remote node (worker-fn)))
                       nodes)))
    (letrec ((supervise (lambda (active-workers)
                          (receive
                            ;; Handle node failures
                            (('NODE-DOWN node)
                             (display "Node failed: ")
                             (display node) (newline))
                            (let ((replacement
                                   (spawn-remote
                                    (backup-node)
                                    (worker-fn))))
                              (supervise (cons
                                           replacement
                                           (filter
                                            (lambda (w)

```

```

(not
  (on-node?
    w
    node)))
  active-workers))))

;; Distribute work across
cluster
(('work task client)
  (let ((worker (least-loaded
                  active-workers)))
    (send worker `('task
                  ,client)))
    (supervise
      active-workers))))))
(supervise workers))))

;; Fault-tolerant distributed computation
(define (distributed-map-reduce map-fn reduce-fn data)
  (let ((nodes (available-nodes)))
    ;; Distribute data across nodes
    (let ((chunks (distribute-data data nodes)))
      ;; Spawn mappers on each node
      (let ((map-futures
              (map (lambda (node-chunk)
                     (let ((node (car node-chunk))
                           (chunk (cdr node-chunk)))
                       (future-remote node (map map-fn
                                                  chunk))))
                   chunks)))
        ;; Collect and reduce results
        (let ((map-results (map await map-futures)))
          (fold-left reduce-fn (car map-results) (cdr
                                                    map-results))))))

;; Network partition handling
(define (partition-tolerant-operation data)
  (with-network-timeout 5000
    (lambda ()
      (distributed-computation data))
    (lambda ()
      ;; Fallback to local computation during partition
      (local-computation data))))

```

7.15. Concurrency Types

concurrency types actor type

The type system tracks concurrency-related properties:

Example 7.15.1 (Concurrency Types).

```

;; Actor types
(define counter-actor :: (Actor (U ('get ActorId)
                                     ('increment ActorId) ('set Natural ActorId))))

;; Future types
(define computation :: (Future Natural))
(define async-string :: (Future String))

;; Channel types
(define numbers :: (Channel Natural))
(define messages :: (Channel String))

;; STM types
(define account :: (TVar Natural))

```

```

(define shared-counter :: (TVar Natural))

;; Effect tracking for concurrency
(define concurrent-computation :: (-> Unit ~> {Async,
  STM} Natural))
(define (concurrent-computation)
  (let ((counter (make-tvar 0)))
    (let ((f1 (future (atomic (tvar-modify! counter
      (lambda (x) (+ x 1))))))
          (f2 (future (atomic (tvar-modify! counter
      (lambda (x) (+ x 1))))))
          (await f1)
          (await f2)
          (atomic (tvar-read counter))))))

;; Actor behavior types
(define (typed-server :: (-> (Channel (Request Natural
  String)) Unit ~> {Actor} Unit))
  (lambda (request-channel)
    (let loop ()
      (let ((request (channel-get request-channel)))
        (send (request-client request) (process-request
          request))
        (loop))))))

;; Deadlock detection in types
(define (potential-deadlock :: (-> (TVar Natural) (TVar
  Natural) ~> {STM, Deadlock} Unit))
  (lambda (tvar1 tvar2)
    ;; Type system can warn about potential circular
    ;; dependencies
    (atomic
      (let ((val1 (tvar-read tvar1)))
        (let ((val2 (tvar-read tvar2)))
          (tvar-write! tvar1 val2)
          (tvar-write! tvar2 val1))))))

```

7.16. Performance and Optimization

concurrency performance green threads

The concurrency implementation is optimized for performance:

Example 7.16.1 (Concurrency Performance).

```

;; Lightweight actors (green threads)
(define actors (map (lambda (i) (spawn (simple-actor i)))
  (range 0 10000)))
;; Can spawn thousands of actors with minimal overhead

;; Work-stealing scheduler
(define (cpu-intensive-work data)
  ;; Automatically distributed across available cores
  (parallel-map expensive-function data))

;; Lock-free data structures
(define lock-free-counter (make-atomic-counter 0))
(define (increment-counter!)
  (atomic-increment! lock-free-counter)) ; No locks, CAS
  operations

;; Actor batching for high throughput
(define (batch-actor batch-size)
  (let ((batch '()))
    (letrec ((loop (lambda ()

```

```

      (receive
        ((msg)
         (set! batch (cons msg batch))
         (if (>= (length batch) batch-size)
             (begin
              (process-batch batch)
              (set! batch '())
              (loop))
             (loop))))))

;; Memory-efficient message passing
(define (zero-copy-send pid large-data)
  ;; Large data shared between actors without copying
  (send-shared pid large-data))

;; NUMA-aware scheduling
(define (numa-spawn computation)
  ;; Spawn on local NUMA node for better memory locality
  (spawn-local computation))

```

7.17. Debugging and Monitoring

concurrency debugging deadlock detection

Tools for debugging concurrent programs:

Example 7.17.1 (Concurrency Debugging).

```

;; Actor monitoring
(define (monitored-actor computation)
  (spawn-monitored computation
    (lambda (pid state)
      (display "Actor ") (display pid)
      (display " state: ") (display state) (newline))))

;; Deadlock detection
(enable-deadlock-detection!)
(define (potential-deadlock)
  (atomic
    (let ((val1 (tvar-read tvar1)))
      ;; System detects potential circular wait
      (let ((val2 (tvar-read tvar2))) ; May warn or
        abort
        (do-something val1 val2))))))

;; Message tracing
(enable-message-tracing!)
(send traced-actor message) ; Logged for debugging

;; Performance profiling
(with-concurrency-profiler
  (lambda ()
    (distributed-computation large-dataset)))
;; Produces report on actor communication, load
distribution, etc.

;; Race condition detection
(define shared-data (make-atomic-box 0))
(spawn (lambda () (atomic-update! shared-data (lambda (x)
  (+ x 1)))))
(spawn (lambda () (atomic-update! shared-data (lambda (x)
  (* x 2)))))
;; Runtime can detect data races and report them

```

The concurrency system provides powerful abstractions for parallel and distributed computing while maintaining safety through the type system and runtime checks.

8. Foreign Function Interface

Lambdust provides a memory-safe, capability-based Foreign Function Interface (FFI) for interoperating with C libraries while maintaining type safety and security through static analysis and runtime checks.

8.1. FFI Overview

The FFI system provides:

- **Type Safety:** Foreign function signatures are statically checked
- **Memory Safety:** Automatic marshaling prevents buffer overflows and dangling pointers
- **Capability Security:** Access to foreign functions requires explicit capabilities
- **Error Handling:** Foreign function errors are converted to Scheme exceptions
- **Garbage Collection Integration:** Foreign pointers are tracked by the GC

8.2. Foreign Declarations

foreign declaration foreign-declare

Foreign functions must be declared before use:

foreign-declaration → (foreign-declare *library binding**)

binding → (*name* :: *foreign-type*)

foreign-type → (-> *c-type** *c-type*)

Example 8.2.1 (Foreign Declarations).

```
;; Math library functions
(foreign-declare "libm.so"
  (sin :: (-> Float64 Float64))
  (cos :: (-> Float64 Float64))
  (sqrt :: (-> Float64 Float64))
  (pow :: (-> Float64 Float64 Float64)))

;; String functions
(foreign-declare "libc.so"
  (strlen :: (-> CString Size))
  (strcmp :: (-> CString CString Int32))
  (strdup :: (-> CString CString))
  (free :: (-> Pointer Void)))
```

```
;; File I/O
(foreign-declare "libc.so"
  (fopen :: (-> CString CString FilePointer))
  (fclose :: (-> FilePointer Int32))
  (fread :: (-> Pointer Size Size FilePointer Size))
  (fwrite :: (-> Pointer Size Size FilePointer Size)))
```

```
;; System calls
(foreign-declare "libc.so"
  (getpid :: (-> Void Int32))
  (getenv :: (-> CString CString))
  (setenv :: (-> CString CString Int32 Int32)))
```

```
;; Custom library
(foreign-declare "./mylib.so"
  (my-function :: (-> Int32 Int32 Int32))
  (init-library :: (-> Void Int32))
  (cleanup-library :: (-> Void Void)))
```

8.3. C Type System

C types type marshaling

The FFI supports comprehensive C type mapping:

8.4. Primitive Types

Table 8.1: C Type Mapping

C Type	Lambdust Type	Description
void	Void	No value
char	Int8	8-bit signed integer
unsigned char	UInt8	8-bit unsigned integer
short	Int16	16-bit signed integer
unsigned short	UInt16	16-bit unsigned integer
int	Int32	32-bit signed integer
unsigned int	UInt32	32-bit unsigned integer
long long	Int64	64-bit signed integer
unsigned long long	UInt64	64-bit unsigned integer
float	Float32	32-bit floating-point
double	Float64	64-bit floating-point
size_t	Size	Architecture-dependent size
char*	CString	Null-terminated string
void*	Pointer	Generic pointer

8.5. Compound Types

Example 8.5.1 (Compound C Types).

```
;; Array types
(define-c-type (Array n T) (* T)) ; T[n] in C

;; Structure types
(define-c-struct Point
  (x :: Float64)
  (y :: Float64))

(define-c-struct Person
  (name :: CString)
  (age :: Int32)
  (height :: Float64))

;; Union types
(define-c-union Value
  (i :: Int32)
  (f :: Float64)
  (s :: CString))

;; Function pointer types
(define-c-type Callback (-> Int32 Int32))
(define-c-type CompareFunction (-> Pointer Pointer Int32))

;; Opaque pointer types (for library handles)
(define-c-opaque-type FileHandle)
(define-c-opaque-type DatabaseConnection)

;; Declarations using compound types
(foreign-declare "graphics.so"
  (draw-point :: (-> Point Void))
  (distance :: (-> Point Point Float64))
  (make-person :: (-> CString Int32 Float64 Person))
  (print-person :: (-> Person Void)))

(foreign-declare "libc.so"
  (qsort :: (-> Pointer Size Size CompareFunction Void)))
```

8.6. Function Calls

foreign-call with-capability

Foreign functions are called through the capability system:

$$\text{foreign-call} \rightarrow \begin{array}{l} \text{(foreign-call function-name argument*)} \\ \mid \\ \text{(with-capability capability-name body)} \end{array}$$

Example 8.6.1 (Foreign Function Calls).

```
;; Basic function calls with capabilities
(with-capability 'math-library
  (lambda ()
    (let ((x 2.0))
      (display "sin(2) = ")
      (display (foreign-call sin x))
      (newline))))

;; String operations
(with-capability 'libc
  (lambda ()
```

```
    (let ((str1 "Hello")
          (str2 "World"))
      (let ((len1 (foreign-call strlen str1))
            (cmp (foreign-call strcmp str1 str2)))
        (display "Length: ") (display len1) (newline)
        (display "Compare: ") (display cmp) (newline))))))

;; File operations with error handling
(define (read-file-contents filename)
  (with-capability 'file-io
    (lambda ()
      (let ((file (foreign-call fopen filename "r")))
        (if (null-pointer? file)
            (error "Cannot open file" filename)
            (begin
              ;; Read file contents
              (let ((buffer (allocate-buffer 1024)))
                (let ((bytes-read (foreign-call fread
                                   buffer 1 1024 file)))
                  (foreign-call fclose file)
                  (buffer->string buffer
                    bytes-read))))))))))

;; Callback functions
(define (my-compare-function :: CompareFunction)
  (lambda (a-ptr b-ptr)
    (let ((a (pointer->int32 a-ptr))
          (b (pointer->int32 b-ptr)))
      (cond ((< a b) -1)
            ((> a b) 1)
            (else 0)))))

;; Use callback with foreign function
(with-capability 'libc
  (lambda ()
    (let ((array (list->c-array '(3 1 4 1 5 9 2 6))))
      (foreign-call qsort array 8 4 my-compare-function)
      (c-array->list array 8))))
```

8.7. Memory Management

memory management garbage collection

The FFI provides safe memory management:

8.8. Automatic Memory Management

Example 8.8.1 (Automatic Memory Management).

```
;; Automatic string marshaling
(define (safe-string-length str)
  (with-capability 'libc
    (lambda ()
      ;; String automatically converted to C string
      ;; Memory automatically managed
      (foreign-call strlen str))))

;; Automatic cleanup with finalizers
(define (allocate-managed-buffer size)
  (let ((ptr (foreign-call malloc size)))
    (if (null-pointer? ptr)
        (error "Allocation failed")
        (begin
```

```

;; Register finalizer to free memory
(register-finalizer! ptr (lambda (p)
  (foreign-call free p)))
ptr))))

;; RAII-style resource management
(define-syntax with-file
  (syntax-rules ()
    ((with-file (var filename mode) body ...)
     (let ((var (foreign-call fopen filename mode)))
       (if (null-pointer? var)
           (error "Cannot open file" filename)
           (dynamic-wind
            (lambda () #f) ; Setup (already done)
            (lambda () body ...) ; Body
            (lambda () ; Cleanup
              (when (not (null-pointer? var))
                (foreign-call fclose var))))))))))

;; Usage
(with-file (file "data.txt" "r")
  (let ((buffer (allocate-buffer 1024)))
    (foreign-call fread buffer 1 1024 file)
    (buffer->string buffer)))

```

8.9. Manual Memory Management

Example 8.9.1 (Manual Memory Management).

```

;; Explicit allocation and deallocation
(define (manual-buffer-operations)
  (with-capability 'libc
    (lambda ()
      (let ((buffer (foreign-call malloc 1024)))
        (if (null-pointer? buffer)
            (error "Allocation failed")
            (begin
              ;; Use buffer
              (foreign-call memset buffer 0 1024)
              (let ((result (process-buffer buffer)))
                ;; Explicit cleanup
                (foreign-call free buffer)
                result)))))))

;; Reference counting for shared resources
(define-struct shared-resource (ptr refcount))

(define (acquire-shared-resource ptr)
  (let ((resource (make-shared-resource ptr 1)))
    (register-finalizer! resource release-shared-resource)
    resource))

(define (retain-shared-resource resource)
  (set-shared-resource-refcount! resource
    (+ (shared-resource-refcount resource) 1)))

(define (release-shared-resource resource)
  (let ((new-count (- (shared-resource-refcount resource)
    1)))
    (set-shared-resource-refcount! resource new-count)
    (when (= new-count 0)
      (foreign-call free (shared-resource-ptr
        resource))))))

```

```

;; Stack-allocated structures
(define-syntax with-stack-struct
  (syntax-rules ()
    ((with-stack-struct (var struct-type) body ...)
     (let ((var (allocate-stack struct-type)))
       body ...))) ; Automatically freed when scope exits

;; Usage
(with-stack-struct (point Point)
  (set-point-x! point 3.0)
  (set-point-y! point 4.0)
  (foreign-call draw-point point))

```

8.10. Capability System

capability security

The capability system provides fine-grained access control:

8.11. Capability Declaration

Example 8.11.1 (Capability Declaration).

```

;; Define capabilities with permissions
(define-capability math-library
  (libraries "libm.so")
  (functions sin cos tan sqrt pow log exp)
  (memory-limit 0) ; No heap allocation
  (file-access none) ; No file system access

(define-capability file-io
  (libraries "libc.so")
  (functions fopen fclose fread fwrite fseek ftell)
  (memory-limit 1048576) ; 1MB heap limit
  (file-access read-write) ; Full file access
  (paths "/tmp/*" "/var/data/*")) ; Restrict to specific paths

(define-capability network-client
  (libraries "libc.so" "libssl.so")
  (functions socket connect send recv ssl_connect)
  (memory-limit 10485760) ; 10MB heap limit
  (network-access client) ; Client connections only
  (domains "*.example.com" "api.service.org"))

(define-capability database
  (libraries "libpq.so")
  (functions PQconnectdb PQexec PQfinish PQgetvalue)
  (memory-limit 52428800) ; 50MB heap limit
  (network-access client)
  (domains "db.internal.com"))

;; Capability inheritance
(define-capability full-application
  (inherits file-io network-client database)
  (additional-functions my-custom-function))

```

8.12. Capability Usage

Example 8.12.1 (Capability Usage).

```
;; Function requiring specific capabilities
(define (process-data-file :: (-> String ~>
  {FFI[file-io]} String))
  (lambda (filename)
    (with-capability 'file-io
      (lambda ()
        (with-file (file filename "r")
          (let ((buffer (allocate-buffer 4096)))
            (let ((bytes-read (foreign-call fread buffer
              1 4096 file)))
              (let ((content (buffer->string buffer
                bytes-read)))
                (string-upcase content))))))))))

;; Capability-polymorphic function
(define (generic-computation :: (Pi (C : Capability) (->
  Unit ~> {FFI[C]} Natural)))
  (lambda (C)
    (lambda ()
      (with-capability C
        (lambda ()
          ;; Can only use functions allowed by capability
          C
          (computation-using-capability C))))))

;; Capability delegation
(define (delegate-capability child-capability
  parent-capability operation)
  (with-capability parent-capability
    (lambda ()
      (with-restricted-capability child-capability ;
        Subset of parent
        (lambda ()
          (operation))))))

;; Dynamic capability checking
(define (safe-operation capability-name)
  (if (capability-available? capability-name)
    (with-capability capability-name
      (lambda () (perform-operation)))
    (error "Required capability not available"
      capability-name)))

;; Temporary capability elevation
(define (elevated-operation)
  (with-temporary-capability 'admin-access 300 ; 5
    minute timeout
    (lambda ()
      (perform-admin-task))))
```

8.13. Error Handling

error handling exception mapping

The FFI converts C errors to Scheme exceptions:

Example 8.13.1 (Error Handling).

```
;; Automatic error checking
(define (safe-file-operation filename)
  (with-capability 'file-io
```

```
(lambda ()
  (with-ffi-error-handling
    (lambda ()
      (let ((file (foreign-call fopen filename "r")))
        ;; Null pointer automatically converted to
        exception
        (let ((size (foreign-call file-size file)))
          (foreign-call fclose file)
          size))))))

;; Custom error mapping
(define-ffi-error-map
  ((null-pointer? fopen) -> (file-not-found-error
    filename))
  ((< 0 malloc) -> (out-of-memory-error size))
  ((= -1 socket) -> (network-error "Cannot create
    socket"))))

;; Error recovery
(define (robust-network-call)
  (call/cc
    (lambda (escape)
      (with-exception-handler
        (lambda (e)
          (cond
            ((network-timeout-error? e)
              (escape 'timeout))
            ((network-connection-error? e)
              (escape 'connection-failed))
            (else (raise e))))
        (lambda ()
          (with-capability 'network-client
            (lambda ()
              (foreign-call network-operation))))))))))

;; Resource cleanup on error
(define (safe-resource-operation)
  (let ((resource #f))
    (dynamic-wind
      ;; Acquire resource
      (lambda ()
        (set! resource (foreign-call acquire-resource)))
      ;; Use resource
      (lambda ()
        (if (null-pointer? resource)
          (error "Resource acquisition failed")
          (foreign-call use-resource resource)))
      ;; Always cleanup
      (lambda ()
        (when (and resource (not (null-pointer?
          resource)))
          (foreign-call release-resource resource))))))

;; Errno handling
(define (system-call-with-errno operation)
  (with-capability 'system
    (lambda ()
      (foreign-call set-errno 0) ; Clear errno
      (let ((result (operation)))
        (let ((error-code (foreign-call get-errno)))
          (if (= error-code 0)
            result
            (error "System call failed"
              (foreign-call strerror
                error-code))))))))
```


8.14. Threading and Concurrency

FFI threading thread safety

The FFI handles threading considerations:

Example 8.14.1 (FFI Threading).

```
;; Thread-safe foreign function calls
(define (thread-safe-operation)
  (with-capability 'thread-safe-library
    (lambda ()
      ;; GIL automatically released for blocking calls
      (foreign-call-blocking long-running-c-function))))

;; Thread-local foreign state
(define (thread-local-context)
  (with-thread-local-capability 'database
    (lambda ()
      (let ((conn (foreign-call db-connect)))
        ;; Connection is thread-local
        (foreign-call db-query conn "SELECT * FROM
          table")))))

;; Async foreign function calls
(define (async-foreign-call function . args)
  (future-with-capability 'async-library
    (lambda ()
      (apply foreign-call function args))))

;; Usage
(let ((result-future (async-foreign-call
  expensive-computation 42)))
  ;; Do other work
  (let ((result (await result-future)))
    (display "Result: ") (display result) (newline)))

;; Callback thread safety
(define (thread-safe-callback :: CCallback)
  (lambda (data)
    (with-scheme-thread-context ; Ensure proper Scheme
      context
      (lambda ()
        ;; Safe to call Scheme functions from C callback
        (process-callback-data data)))))

;; Register thread-safe callback
(with-capability 'event-library
  (lambda ()
    (foreign-call register-callback
      thread-safe-callback)))

;; Foreign function call batching
(define (batch-foreign-calls operations)
  (with-capability 'batch-library
    (lambda ()
      ;; Minimize FFI overhead by batching
      (foreign-call-batch (list->c-array operations)))))
```

8.15. Platform Integration

platform integration native libraries

The FFI integrates with platform-specific libraries:

Example 8.15.1 (Platform Integration).

```
;; Platform-specific declarations
(cond-expand
  (linux
    (foreign-declare "libc.so.6"
      (epoll-create :: (-> Int32 Int32))
      (epoll-ctl :: (-> Int32 Int32 Int32 Pointer
        Int32))))
  (darwin
    (foreign-declare "libSystem.dylib"
      (kqueue :: (-> Void Int32))
      (kevent :: (-> Int32 Pointer Int32 Pointer Int32
        Pointer Int32))))
  (windows
    (foreign-declare "kernel32.dll"
      (CreateEvent :: (-> Pointer Int32 Int32 CString
        Pointer))
      (WaitForSingleObject :: (-> Pointer Int32 Int32))))

;; Cross-platform wrapper
(define (create-event-loop)
  (cond-expand
    (linux (with-capability 'system
      (lambda () (foreign-call epoll-create 10))))
    (darwin (with-capability 'system
      (lambda () (foreign-call kqueue))))
    (windows (with-capability 'system
      (lambda () (foreign-call CreateEvent
        null-pointer 0 0 null-pointer))))))

;; Dynamic library loading
(define (load-dynamic-library path)
  (with-capability 'dynamic-loading
    (lambda ()
      (let ((handle (foreign-call dlopen path RTLD-LAZY)))
        (if (null-pointer? handle)
          (error "Cannot load library" path
            (foreign-call dlerror))
          handle)))))

(define (get-dynamic-symbol handle symbol-name)
  (with-capability 'dynamic-loading
    (lambda ()
      (let ((symbol (foreign-call dlsym handle
        symbol-name)))
        (if (null-pointer? symbol)
          (error "Symbol not found" symbol-name
            symbol)))))

;; Plugin system
(define (load-plugin plugin-path)
  (let ((handle (load-dynamic-library plugin-path)))
    (let ((init-fn (get-dynamic-symbol handle
      "plugin_init"))
      (process-fn (get-dynamic-symbol handle
        "plugin_process"))
      (cleanup-fn (get-dynamic-symbol handle
        "plugin_cleanup")))
      (make-plugin handle init-fn process-fn
        cleanup-fn)))

;; System integration
(define (integrate-with-system-service service-name)
  (cond-expand
    (linux (systemd-integration service-name))
    (darwin (launchd-integration service-name))
    (windows (windows-service-integration service-name))))
```


8.16. Performance Considerations

FFI performance optimization

The FFI is optimized for performance:

Example 8.16.1 (FFI Performance).

```
;; Fast path for simple calls
(define (optimized-math-call x)
  ;; Minimal overhead for simple numeric functions
  (foreign-call-fast sin x)) ; Direct call, no marshaling

;; Bulk operations
(define (process-large-array data)
  (with-capability 'compute-library
    (lambda ()
      ;; Process entire array in C, avoiding per-element overhead
      (let ((c-array (scheme-array->c-array data)))
        (foreign-call process-array-bulk c-array
          (array-length data))
        (c-array->scheme-array c-array))))))

;; Memory mapping for large data
(define (process-large-file filename)
  (with-capability 'file-io
    (lambda ()
      (let ((mapping (foreign-call mmap-file filename)))
        (foreign-call process-mapped-data mapping)
        (foreign-call munmap mapping))))))

;; Zero-copy string operations
(define (zero-copy-string-operation str)
  (with-pinned-string str
    (lambda (c-str)
      ;; String memory pinned, passed directly to C
      (foreign-call string-process-in-place c-str))))

;; Inline assembly for critical operations
(define-inline-asm fast-checksum (data :: Pointer) (size
  :: Size) :: UInt32
  "
  mov %rdi, %rax    ; data pointer
  mov %rsi, %rcx    ; size
  xor %edx, %edx    ; checksum = 0
checksum_loop:
  add (%rax), %edx  ; Add word to checksum
  add $4, %rax      ; Next word
  sub $4, %rcx      ; Decrease remaining
  jnz checksum_loop ; Continue if not zero
  mov %edx, %eax    ; Return checksum
  ")

;; SIMD operations via FFI
(define (simd-vector-add v1 v2)
  (with-capability 'simd-library
    (lambda ()
      ;; Use vectorized instructions for array operations
      (foreign-call vector-add-simd v1 v2 (vector-length
        v1))))))
```

8.17. Security Considerations

FFI security sandboxing

The FFI includes security measures:

Example 8.17.1 (FFI Security).

```
;; Sandboxed foreign function execution
(define (sandboxed-computation data)
  (with-sandbox 'computation-sandbox
    (lambda ()
      ;; Limited capabilities, resource limits enforced
      (with-capability 'restricted-compute
        (lambda ()
          (foreign-call untrusted-computation data))))))

;; Input validation
(define (validated-foreign-call function input)
  (validate-ffi-input input function) ; Static + runtime
  validation
  (with-capability 'validated-access
    (lambda ()
      (foreign-call function input))))

;; Buffer overflow protection
(define (safe-buffer-operation buffer size)
  (with-bounds-checked-buffer buffer size
    (lambda (safe-buffer)
      ;; All buffer accesses are bounds-checked
      (foreign-call process-buffer safe-buffer size))))

;; Capability revocation
(define temp-capability
  (create-temporary-capability 'network-access 300)) ; 5
  minutes

;; Revoke capability early if needed
(when (security-breach-detected?)
  (revoke-capability! temp-capability))

;; Audit logging
(with-audit-logging 'foreign-function-calls
  (lambda ()
    (foreign-call sensitive-operation)))
;; All foreign calls logged for security analysis
```

The FFI provides powerful integration with native code while maintaining the safety and security properties that make Lambdust suitable for high-assurance applications.

9. Macro System Extensions

Lambdust extends the R7RS macro system with additional features for advanced metaprogramming while maintaining full backward compatibility with existing syntax-rules and syntax-case macros.

9.1. Macro System Overview

Lambdust's macro system provides:

- **R7RS Compatibility:** Full support for syntax-rules and syntax-case
- **Procedural Macros:** Macros as first-class procedures

- **Type-Aware Macros:** Integration with the gradual type system
- **Compile-Time Computation:** Evaluation during macro expansion
- **Macro Debugging:** Tools for debugging macro expansion
- **Hygienic Capture:** Safe variable capture mechanisms

9.2. R7RS Macro Support

syntax-rules syntax-case

All R7RS macro forms are fully supported:

9.2.1. Syntax Rules

Example 9.2.1 (Syntax Rules Macros).

```
;; Basic syntax-rules macro
(define-syntax when
  (syntax-rules ()
    ((when test body ...)
     (if test (begin body ...)))))

;; Pattern matching with literals
(define-syntax my-cond
  (syntax-rules (else)
    ((my-cond (else result))
     result)
    ((my-cond (test result))
     (if test result))
    ((my-cond (test result) clause ...)
     (if test result (my-cond clause ...)))))

;; Recursive pattern expansion
(define-syntax let*
  (syntax-rules ()
    ((let* () body ...)
     (begin body ...))
    ((let* ((var val) binding ...) body ...)
     (let ((var val))
       (let* (binding ...) body ...))))

;; Complex pattern matching
(define-syntax match-lambda
  (syntax-rules ()
    ((match-lambda (pattern body ...) ...)
     (lambda (expr)
       (match expr
        (pattern body ...) ...))))

;; Identifier macros
(define-syntax current-time
  (syntax-rules ()
    (current-time (get-current-time))))
```

9.2.2. Syntax Case

Example 9.2.2 (Syntax Case Macros).

```
;; Advanced syntax-case macro
(define-syntax define-class
  (lambda (stx)
    (syntax-case stx ()
      ((define-class name (field ...) method ...)
       (with-syntax ((constructor (format-id #'name
        "make-~a" #'name)))
         ((getter ...) (map (lambda (f)
                               (format-id f
            "~a-~a"
            #'name f))
            #'(field ...)))
          ((setter ...) (map (lambda (f)
                               (format-id f
            "set-~a-~a!"
            #'name f))
            #'(field ...))))))

      #'(begin
        (define-record-type name
          (constructor field ...)
          name?
          (field getter setter) ...)
          method ...))))))

;; Macro with compile-time computation
(define-syntax compile-time-factorial
  (lambda (stx)
    (syntax-case stx ()
      ((compile-time-factorial n)
       (number? (syntax->datum #'n))
       (with-syntax ((result (datum->syntax #'n
        (factorial (syntax->datum
         #'n)))))
        #'result))))))

;; Macro generating other macros
(define-syntax define-binary-op
  (lambda (stx)
    (syntax-case stx ()
      ((define-binary-op name op)
       #'(define-syntax name
          (syntax-rules ()
            ((name x y) (op x y))
            ((name x y z ...) (name (op x y) z
              ...)))))))

;; Usage
(define-binary-op add +)
(add 1 2 3 4 5) ; Expands to (+ (+ (+ (+ 1 2) 3) 4) 5)
```

9.3. Procedural Macros

procedural macro macro procedure

Lambdust allows macros to be defined as regular procedures:

Example 9.3.1 (Procedural Macros).

```
;; Macro as procedure
(define-macro (simple-when test . body)
```

```

` (if ,test (begin ,@body)))

;; Macro with parameter validation
(define-macro (validated-let bindings . body)
  (unless (and (list? bindings)
               (every (lambda (b) (and (list? b) (=
                                         (length b) 2)))
                     bindings))
    (error "Invalid let bindings" bindings))
  `(let ,bindings ,@body))

;; Macro generating code based on runtime information
(define-macro (optimized-loop var start end . body)
  (let ((step (if (> end start) 1 -1))
        (test (if (> end start) '< '>)))
    `(let loop ((,var ,start))
      (if (,test ,var ,end)
          (begin ,@body (loop (+ ,var ,step))))))

;; Higher-order macro
(define-macro (define-accessor-macro name field-extractor)
  `(define-macro (,name obj field)
    `((,field-extractor 'field) ,obj)))

;; Usage
(define-accessor-macro getter (lambda (f) (format-symbol
  "~a--~a" 'get f)))
(getter person name) ; Expands to (get-name person)

;; Macro with effects
(define-macro (logged-define name value)
  (display "Defining ") (display name) (newline)
  `(define ,name ,value))

```

9.4. Type-Aware Macros

type-aware macro compile-time type checking

Macros can interact with the type system:

Example 9.4.1 (Type-Aware Macros).

```

;; Macro that generates different code based on types
(define-syntax typed-accessor
  (lambda (stx)
    (syntax-case stx ()
      ((typed-accessor obj field-name)
       (let ((obj-type (infer-type #'obj)))
         (case obj-type
           ((Person) #'(person-field-name obj))
           ((Point) #'(point-field-name obj))
           (else (error "Unknown type for accessor"
                        obj-type)))))))

;; Macro with type constraints
(define-syntax safe-arithmetic
  (lambda (stx)
    (syntax-case stx ()
      ((safe-arithmetic op x y)
       (let ((x-type (infer-type #'x))
             (y-type (infer-type #'y)))
         (unless (and (numeric-type? x-type)
                      (numeric-type? y-type))
           (syntax-error "Non-numeric arguments to
                        arithmetic operation"
                          #'x #'y)))

```

```

    #'(op x y))))))

;; Type-directed code generation
(define-syntax generic-equal
  (lambda (stx)
    (syntax-case stx ()
      ((generic-equal x y)
       (let ((type (infer-type #'x)))
         (cond
          ((string-type? type) #'(string=? x y))
          ((number-type? type) #'(= x y))
          ((list-type? type) #'(equal? x y))
          (else #'(eqv? x y)))))))

;; Macro generating type annotations
(define-syntax define-typed
  (syntax-rules ()
    ((define-typed (name param ...) :: type body ...)
     (define name
      #:type (-> (typeof param) ... type)
      (lambda (param ...) body ...))))

;; Compile-time type computation
(define-syntax compute-result-type
  (lambda (stx)
    (syntax-case stx ()
      ((compute-result-type f x)
       (let* ((f-type (infer-type #'f))
              (x-type (infer-type #'x))
              (result-type (apply-function-type f-type
                                                  x-type)))
         (with-syntax ((result-type-expr (type->syntax
                                           result-type)))
           #'(the result-type-expr (f x)))))))

```

9.5. Compile-Time Computation

compile-time computation macro-time evaluation

Lambdust supports computation during macro expansion:

Example 9.5.1 (Compile-Time Computation).

```

;; Compile-time constants
(define-syntax pi-digits
  (syntax-rules ()
    (pi-digits
     ,(compute-pi-to-precision 50)))) ; Computed at
                                     compile time

;; Compile-time file reading
(define-syntax include-file-as-string
  (lambda (stx)
    (syntax-case stx ()
      ((include-file-as-string filename)
       (string? (syntax->datum #'filename))
       (let ((content (file->string (syntax->datum
                                     #'filename))))
         (datum->syntax #'filename content))))))

;; Compile-time code generation from data
(define-syntax generate-accessors
  (lambda (stx)
    (syntax-case stx ()
      ((generate-accessors struct-name field-file)

```

```

(let ((fields (read-fields-from-file
  (syntax->datum #'field-file))))
  (with-syntax (((accessor ...)
    (map (lambda (field)
      (format-id #'struct-name
        "~a-~a"
        #'struct-name
        field))
      fields)))
    #'(begin
      (define accessor ...) ...))))))

;; Compile-time optimization
(define-syntax optimized-power
  (lambda (stx)
    (syntax-case stx ()
      ((optimized-power base exp)
       (and (number? (syntax->datum #'exp))
            (exact-integer? (syntax->datum #'exp))
            (>= (syntax->datum #'exp) 0))
       (let ((n (syntax->datum #'exp)))
         (cond
          ((= n 0) #'1)
          ((= n 1) #'base)
          ((= n 2) #'(* base base))
          ((even? n)
           (let ((half (quotient n 2)))
             #'(let ((temp (optimized-power base ,half)))
                  (* temp temp))))
          (else
           #'(* base (optimized-power base ,(- n
            1))))))))))

;; Compile-time memoization
(define-syntax memoized-factorial
  (let ((memo-table (make-hash-table)))
    (lambda (stx)
      (syntax-case stx ()
        ((memoized-factorial n)
         (number? (syntax->datum #'n))
         (let ((value (syntax->datum #'n)))
           (unless (hash-table-exists? memo-table value)
             (hash-table-set! memo-table value (factorial
              value)))
           (datum->syntax #'n (hash-table-ref memo-table
            value)))))))))

```

9.6. Advanced Pattern Matching

pattern matching syntax patterns

Lambdust provides enhanced pattern matching in macros:

Example 9.6.1 (Advanced Pattern Matching).

```

;; Nested pattern matching
(define-syntax deep-match
  (syntax-rules ()
    ((deep-match ((a (b c) d) ...) body ...)
     (match-nested-structure a b c d body ...)))

;; Pattern with constraints
(define-syntax constrained-match
  (lambda (stx)
    (syntax-case stx ()

```

```

      ((constrained-match ((name type constraint) ...)
        body ...)
       (every identifier? #'(name ...))
       (validate-constraints #'((name type constraint)
        ...))
       #'(let ((name (cast-to-type name type)) ...)
         (when (and constraint ...)
           body ...))))))

;; Regex-like syntax patterns
(define-syntax define-pattern-macro
  (syntax-rules (...)
    ((define-pattern-macro name (pattern ...))
     (define-syntax name
       (syntax-rules ()
         ((name pattern ...) (matched-code pattern
          ...))))))

;; Usage
(define-pattern-macro binary-tree
  ((leaf value) → value)
  ((node left right) → (cons left right)))

;; Recursive pattern with accumulator
(define-syntax fold-syntax
  (syntax-rules ()
    ((fold-syntax op init (elem ...))
     (fold-syntax-helper op init elem ...))
    ((fold-syntax-helper op acc)
     acc)
    ((fold-syntax-helper op acc elem rest ...)
     (fold-syntax-helper op (op acc elem) rest ...)))

;; Pattern matching with guards
(define-syntax guarded-case
  (lambda (stx)
    (syntax-case stx (when)
      ((guarded-case expr
        ((pattern when guard) result) ...)
       #'(let ((temp expr))
         (cond
          ((and (pattern-match? temp 'pattern) guard)
           (with-pattern-bindings temp 'pattern
            result)) ...))))))

```

9.7. Macro Debugging

macro debugging macro expansion

Tools for debugging macro expansion:

Example 9.7.1 (Macro Debugging).

```

;; Trace macro expansion
(define-syntax traced-when
  (trace-macro
   (syntax-rules ()
     ((traced-when test body ...)
      (if test (begin body ...))))))

;; Usage shows expansion steps
(traced-when (> x 0) (display "positive"))
;; Trace: (traced-when (> x 0) (display "positive"))
;; -> (if (> x 0) (begin (display "positive")))

;; Step-by-step macro expansion

```

```

(expand-macro-steps '(let* ((x 1) (y (+ x 1))) (+ x y)))
;; Step 1: (let ((x 1)) (let* ((y (+ x 1))) (+ x y)))
;; Step 2: (let ((x 1)) (let ((y (+ x 1))) (+ x y)))
;; Final:  (let ((x 1)) (let ((y (+ x 1))) (+ x y)))

;; Macro expansion with environment info
(expand-with-environment '(my-macro x y)
  '((x . Number) (y . String)))

;; Debugging macro with assertions
(define-syntax debug-assert
  (lambda (stx)
    (syntax-case stx ()
      ((debug-assert expr message)
       (if (getenv "LAMBUST-DEBUG")
           #'(unless expr (error message))
           #'(begin))))))

;; Macro hygiene debugging
(define-syntax hygiene-test
  (lambda (stx)
    (syntax-case stx ()
      ((hygiene-test)
       (let ((temp-var (generate-temporary 'temp)))
         (with-syntax ((temp temp-var))
           #'(let ((temp 42))
              (display-hygiene-info 'temp))))))))

;; Interactive macro development
(define-interactive-macro test-macro
  ;; Provides REPL for testing macro during development
  (syntax-rules ()
    ((test-macro x) (processed x))))

;; Macro expansion warnings
(define-syntax deprecated-macro
  (lambda (stx)
    (syntax-warning "deprecated-macro is deprecated, use
      new-macro instead" stx)
    (syntax-case stx ()
      ((deprecated-macro args ...)
       #'(new-macro args ...))))

```

9.8. Hygienic Capture

hygienic capture intentional capture

Safe mechanisms for intentional variable capture:

Example 9.8.1 (Hygienic Capture).

```

;; Controlled capture with datum->syntax
(define-syntax with-result
  (lambda (stx)
    (syntax-case stx ()
      ((with-result body ...)
       (with-syntax ((result (datum->syntax stx 'result)))
         #'(let ((result 'uninitialized))
            body ...
            result))))))

;; Capture from calling context
(define-syntax capture-from-caller
  (lambda (stx)
    (syntax-case stx ()
      ((capture-from-caller var body ...)

```

```

(with-syntax ((captured-var (datum->syntax
  (syntax-context stx) (syntax->datum #'var))))
  #'(begin body ... captured-var))))))

;; Selective hygiene breaking
(define-syntax break-hygiene
  (syntax-rules ()
    ((break-hygiene (var ...) body ...)
     (let-syntax ((capture (syntax-rules ()
        ((capture v) (datum->syntax
          (quote body) 'v))))
       (let ((var (capture var)) ...)
         body ...))))))

;; Macro with controlled scope
(define-syntax with-fresh-variables
  (lambda (stx)
    (syntax-case stx ()
      ((with-fresh-variables (var ...) body ...)
       (with-syntax (((fresh-var ...) (map
        generate-temporary #'(var ...))))
         #'(let ((fresh-var 'fresh) ...)
            (let-syntax ((var (identifier-syntax
              fresh-var)) ...)
              body ...))))))

;; Context-sensitive capture
(define-syntax context-aware-macro
  (lambda (stx)
    (syntax-case stx ()
      ((context-aware-macro expr)
       (let ((context (syntax-context stx)))
         (if (inside-function-context? context)
             #'(function-specific-handling expr)
             #'(global-handling expr))))))

;; Macro with lexical scope analysis
(define-syntax scope-analyzing-macro
  (lambda (stx)
    (syntax-case stx ()
      ((scope-analyzing-macro var body ...)
       (let ((bindings (analyze-lexical-scope #'(body
        ...))))
         (if (member (syntax->datum #'var) bindings)
             #'(local-variable-version var body ...)
             #'(global-variable-version var body
              ...))))))

```

9.9. Macro Libraries

macro library syntax library

Libraries of useful macro utilities:

Example 9.9.1 (Macro Libraries).

```

;; String interpolation macro
(define-syntax string-interpolate
  (syntax-rules ()
    ((string-interpolate template (var value) ...)
     (string-template-expand template
      (list (cons 'var value) ...))))

;; Usage
(let ((name "Alice") (age 30))

```

```

(string-interpolate "Hello, {name}! You are {age} years
  old."
  (name name) (age age)))

;; Async/await syntax
(define-syntax async-block
  (syntax-rules (await)
    ((async-block body ...)
     (make-async-computation (lambda () body ...)))))

(define-syntax await
  (syntax-rules ()
    ((await future-expr)
     (future-wait future-expr))))

;; Pattern matching library
(define-syntax match
  (syntax-rules (else)
    ((match expr (pattern body ...) ... (else
      default-body ...))
     (pattern-match expr
      (list (cons 'pattern (lambda (bindings) body ...))
        ...)
      (lambda () default-body ...)))
    ((match expr (pattern body ...) ...)
     (match expr (pattern body ...) ... (else (error "No
      match"))))))

;; Loop macros
(define-syntax for
  (syntax-rules (in from to by)
    ((for var in lst body ...)
     (for-each (lambda (var) body ...) lst))
    ((for var from start to end body ...)
     (do ((var start (+ var 1)))
       (> var end)
       body ...))
    ((for var from start to end by step body ...)
     (do ((var start (+ var step)))
       (> var end)
       body ...))))

;; Comprehension macros
(define-syntax list-comprehension
  (syntax-rules (for in if)
    ((list-comprehension expr for var in lst)
     (map (lambda (var) expr) lst))
    ((list-comprehension expr for var in lst if pred)
     (filter-map (lambda (var) (and pred expr)) lst))))

;; Usage
(list-comprehension (* x x) for x in '(1 2 3 4 5))
;; → (1 4 9 16 25)

;; Anaphoric macros
(define-syntax aif
  (syntax-rules ()
    ((aif test then else)
     (let ((it test))
       (if it then else))))

(define-syntax awhen
  (syntax-rules ()
    ((awhen test body ...)
     (let ((it test))
       (when it body ...))))

;; Class definition macro
(define-syntax define-class

```

```

(syntax-rules (extends implements method field)
  ((define-class name (extends super) (field name type
    init) ... (method signature body ...) ...)
   (begin
    (define-record-type name
      (make-name field-name ...)
      name?
      (field-name field-accessor field-setter!) ...)
    (define-method (signature) body ...) ...))))


```

9.10. Macro Performance

macro performance compile-time optimization

Optimizing macro expansion performance:

Example 9.10.1 (Macro Performance).

```

;; Memoized macro expansion
(define-syntax fast-recursive-macro
  (let ((memo-table (make-hash-table equal?)))
    (lambda (stx)
      (let ((key (syntax->datum stx)))
        (hash-table-ref memo-table key
          (lambda ()
            (let ((result (expensive-macro-computation
              stx)))
              (hash-table-set! memo-table key result)
              result)))))))

;; Compile-time constant folding
(define-syntax optimizing-arithmetic
  (lambda (stx)
    (syntax-case stx ()
      ((optimizing-arithmetic op x y)
       (and (number? (syntax->datum #'x))
        (number? (syntax->datum #'y)))
       ;; Compute at compile time
       (datum->syntax stx
        ((eval (syntax->datum #'op))
         (syntax->datum #'x)
         (syntax->datum #'y))))
      ((optimizing-arithmetic op x y)
       ;; Runtime computation
       #'(op x y))))

;; Batch code generation
(define-syntax generate-many
  (lambda (stx)
    (syntax-case stx ()
      ((generate-many template (data ...))
       (with-syntax (((generated ...)
         (map (lambda (d)
              (instantiate-template
                #'template d))
              #'(data ...))))
        #'(begin generated ...))))))

;; Lazy macro expansion
(define-syntax lazy-macro
  (lambda (stx)
    (syntax-case stx ()
      ((lazy-macro complex-computation)
       #'(delay (force-macro-expansion
        complex-computation)))))

```



```
;; Parallel macro processing (when safe)
(define-syntax parallel-map-macro
  (lambda (stx)
    (syntax-case stx ()
      ((parallel-map-macro template (item ...))
       (let ((results (parallel-map
                        (lambda (i) (instantiate-template
                                    #'template i))
                        #'(item ...))))
         (with-syntax (((result ...) results))
           #'(begin result ...)))))))
```

The macro system provides powerful metaprogramming capabilities while maintaining the hygiene and safety properties that make Scheme macros reliable and composable.

10. Standard Library

The Lambdust standard library extends R7RS-small with additional modules for advanced programming while maintaining complete backward compatibility. All R7RS-small procedures are available unchanged.

10.1. Library Organization

The standard library is organized into modules:

- **R7RS Core:** Complete R7RS-small implementation
- **Extended Base:** Enhanced versions of core libraries
- **Gradual Types:** Type system integration
- **Effects:** Algebraic effects and handlers
- **Concurrency:** Actor model and parallelism
- **Collections:** Advanced data structures
- **Text:** Unicode and pattern matching
- **I/O:** Enhanced input/output operations
- **FFI:** Foreign function interface
- **Meta:** Reflection and metaprogramming

10.2. R7RS-small Compatibility

R7RS compatibility base library

All R7RS-small libraries are fully supported:

Example 10.2.1 (R7RS Libraries).

```
;; Import R7RS libraries
(import (scheme base)
        (scheme case-lambda)
        (scheme char)
        (scheme complex)
```

```
(scheme cxx)
(scheme eval)
(scheme file)
(scheme inexact)
(scheme lazy)
(scheme load)
(scheme process-context)
(scheme read)
(scheme repl)
(scheme time)
(scheme write))
```

```
;; All R7RS procedures work unchanged
(define (r7rs-example)
  (let* ((numbers '(3 1 4 1 5 9 2 6))
         (doubled (map (lambda (x) (* x 2)) numbers))
         (filtered (filter (lambda (x) (> x 5)) doubled)))
    (fold-left + 0 filtered)))
```

```
;; R7RS file operations
(call-with-output-file "output.txt"
  (lambda (port)
    (write '(hello world) port)
    (newline port)))
```

```
;; R7RS parameter objects
(parameterize ((current-output-port (open-output-string)))
  (display "Hello")
  (get-output-string (current-output-port)))
```

10.3. Extended Base Library

extended base

The (lambdust base) library extends R7RS base:

10.3.1. Enhanced List Operations

Procedure: (take lst k)

Returns the first k elements of lst .

Procedure: (drop lst k)

Returns lst with the first k elements removed.

Procedure: (split-at lst k)

Returns two values: (take lst k) and (drop lst k).

Example 10.3.1 (Enhanced List Operations).

```
(import (lambdust base))

;; List utilities
(take '(1 2 3 4 5) 3)      ; → (1 2 3)
(drop '(1 2 3 4 5) 2)     ; → (3 4 5)
(split-at '(1 2 3 4 5) 2)  ; → (1 2) (3 4 5)

;; List comprehensions
(list-comprehension (x * x) (x in '(1 2 3 4 5)) (x > 2))
; → (9 16 25)

;; Lazy lists
(define fibs
  (lazy-list 0 1 (lazy-map + fibs (lazy-tail fibs))))

(take (lazy-list->list fibs) 10)
; → (0 1 1 2 3 5 8 13 21 34)

;; List zipping
(zip '(1 2 3) '(a b c) '(x y z)) ; → ((1 a x) (2 b y)
    (3 c z))
(zip-with + '(1 2 3) '(4 5 6))   ; → (5 7 9)

;; Grouping and partitioning
(group-by even? '(1 2 3 4 5 6)) ; → ((#f 1 3 5) (#t 2
    4 6))
(partition even? '(1 2 3 4 5 6)) ; → (2 4 6) (1 3 5)
```

10.3.2. Enhanced String Operations**Procedure:** (string-split str delimiter)

Splits *str* on *delimiter* returning a list of sub-strings.

Procedure: (string-join strings separator)

Joins *strings* with *separator*.

Example 10.3.2 (Enhanced String Operations).

```
;; String splitting and joining
(string-split "hello,world,scheme" ",") ; → ("hello"
    "world" "scheme")
(string-join '("hello" "world") " ")    ; → "hello world"

;; String formatting
(format "Hello, ~a! You have ~d messages." "Alice" 5)
; → "Hello, Alice! You have 5 messages."

;; String templates
(string-template "Hello, {name}!" '((name . "Bob")))
; → "Hello, Bob!"

;; Case conversion
(string-titlecase "hello world") ; → "Hello World"
(string-capitalize "hello")      ; → "Hello"
```

```
;; String predicates
(string-prefix? "hello" "hello world") ; → #t
(string-suffix? "world" "hello world") ; → #t
(string-contains? "ell" "hello")        ; → 1

;; Unicode operations
(string-normalize "café" 'nfc)           ; NFC
normalization
(string-grapheme-count "👨👩👧👦") ; → 1 (family
emoji)
```

10.4. Type System Integration

type library

The (lambdust types) library provides type system integration:

Example 10.4.1 (Type System Library).

```
(import (lambdust types))

;; Type predicates
(define-predicate natural? (lambda (x) (and (integer? x)
    (>= x 0))))
(define-predicate positive? (lambda (x) (and (real? x) (>
    x 0))))

;; Type assertions
(assert-type 42 Natural) ; Succeeds
(assert-type -5 Natural) ; Error

;; Type conversion
(cast-type "123" String Number) ; → 123
(safe-cast "123" Number)         ; → (Just 123)
(safe-cast "abc" Number)         ; → Nothing

;; Contract checking
(define/contract factorial
  (-> Natural Natural)
  (lambda (n)
    (if (<= n 1) 1 (* n (factorial (- n 1))))))

;; Higher-order contracts
(define/contract map-numbers
  (-> (-> Number Number) (List Number) (List Number))
  (lambda (f lst) (map f lst)))

;; Refinement types
(define NonEmptyList (Refinement (List Any) (lambda (lst)
    (not (null? lst)))))
(define/contract safe-car
  (-> NonEmptyList Any)
  (lambda (lst) (car lst)))

;; Gradual typing utilities
(dynamic->typed 42) ; Dynamic →
typed value
(typed->dynamic (the Natural 42)) ; Typed →
dynamic value
```


10.5. Effects Library

effects library

The `(lambdust effects)` library provides effect operations:

Example 10.5.1 (Effects Library).
`(import (lambdust effects))`

```
;; Built-in effects
(define (stateful-computation)
  (perform State set 0)
  (perform State modify +1)
  (perform State modify +1)
  (perform State get))

;; Exception effects
(define (safe-division x y)
  (if (= y 0)
      (perform Exception raise "Division by zero")
      (/ x y)))

;; I/O effects
(define (interactive-program)
  (perform IO write-line "Enter your name:")
  (let ((name (perform IO read-line)))
    (perform IO write-line (string-append "Hello, " name
                                           "!"))))

;; Custom effects
(define-effect Logger
  (log :: (-> String Unit))
  (set-level :: (-> Symbol Unit)))

(define (logged-computation)
  (perform Logger set-level 'debug)
  (perform Logger log "Starting computation")
  (let ((result (complex-calculation)))
    (perform Logger log "Computation complete")
    result))

;; Effect composition
(define (multi-effect-computation)
  (perform Logger log "Starting")
  (perform State set 0)
  (let ((input (perform IO read-line)))
    (let ((value (string->number input)))
      (if value
          (begin
             (perform State set value)
             (perform Logger log "Value set")
             (perform State get))
          (perform Exception raise "Invalid input")))))

;; Effect handlers
(define (run-with-state-and-logging initial computation)
  (with-state-handler initial
    (with-logging-handler 'console
      (with-exception-handler
        (lambda (e) (display "Error: ") (display e)
                      (newline))
        computation))))
```

10.6. Concurrency Library

concurrency library

The `(lambdust concurrent)` library provides concurrency primitives:

Example 10.6.1 (Concurrency Library).
`(import (lambdust concurrent))`

```
;; Actor operations
(define (worker-actor id)
  (let loop ()
    (receive
      (('work data reply-to)
       (let ((result (process-data data)))
         (send reply-to result)))
      (loop))
    (('shutdown) 'done)))

(define worker (spawn worker-actor 1))
(send worker `(work ,some-data ,self))

;; Future operations
(define future-1 (future (expensive-computation-1)))
(define future-2 (future (expensive-computation-2)))

;; Wait for both
(let ((result-1 (await future-1))
      (result-2 (await future-2)))
  (+ result-1 result-2))

;; Parallel operations
(define results (parallel-map expensive-function
                              data-list))

;; STM operations
(define account1 (make-tvar 1000))
(define account2 (make-tvar 500))

(define (transfer amount from to)
  (atomic
    (let ((from-bal (tvar-read from))
          (to-bal (tvar-read to)))
      (when (>= from-bal amount)
        (tvar-write! from (- from-bal amount))
        (tvar-write! to (+ to-bal amount))))))

;; Channel operations
(define numbers-channel (make-channel))
(spawn (lambda ()
         (do ((i 0 (+ i 1))) ((= i 10))
           (channel-put! numbers-channel i))))

(let loop ((sum 0) (count 0))
  (if (= count 10)
      sum
      (loop (+ sum (channel-get numbers-channel)) (+ count 1)))))
```

10.7. Collections Library

collections data structures

The (`lambdust collections`) library provides advanced data structures:

Example 10.7.1 (Collections Library).

```
(import (lambdust collections))

;; Sets
(define s1 (set 1 2 3 4))
(define s2 (set 3 4 5 6))
(set-union s1 s2)           ; → {1 2 3 4 5 6}
(set-intersection s1 s2)    ; → {3 4}
(set-difference s1 s2)      ; → {1 2}

;; Maps/Dictionaries
(define ages (map-from-alist '(("Alice" . 25) ("Bob" . 30))))
(map-ref ages "Alice")      ; → 25
(map-set ages "Charlie" 35) ; → new map with
                             Charlie
(map-keys ages)             ; → ("Alice"
                             "Bob")

;; Bags/Multisets
(define bag1 (bag 'a 'b 'b 'c))
(bag-count bag1 'b)         ; → 2
(bag-add bag1 'b)           ; → bag with 3
                             b's

;; Sequences
(define seq (sequence 1 2 3 4 5))
(sequence-ref seq 2)        ; → 3
(sequence-append seq (sequence 6 7)) ; → (1 2 3 4 5 6
7)

;; Persistent vectors
(define v1 (vector-from-list '(1 2 3 4 5)))
(define v2 (vector-set v1 2 'new)) ; v1 unchanged
(vector->list v2)                ; → (1 2 new 4 5)

;; Finger trees
(define ft (finger-tree 1 2 3 4 5))
(finger-tree-split-at ft 2)      ; → (1 2) (3 4 5)

;; Priority queues
(define pq (priority-queue < 5 3 8 1 9))
(priority-queue-min pq)         ; → 1
(priority-queue-remove-min pq) ; → queue
without 1

;; Tries
(define trie (trie-from-alist '(("cat" . 1) ("car" . 2)
("card" . 3))))
(trie-keys-with-prefix trie "ca") ; → ("cat" "car"
"card")
```

10.8. Text Processing

text processing regular expressions

The (`lambdust text`) library provides text processing:

Example 10.8.1 (Text Processing).

```
(import (lambdust text))
```

```
;; Regular expressions
(define email-regex (regex
"([\\w.-]+)@([\\w.-]+)\\.([\\w+])"))
(regex-match email-regex "alice@example.com")
; → #<match "alice@example.com" ("alice" "example" "com")>

(regex-replace email-regex "Contact: alice@example.com"
"Email: $1 at $2 dot $3")
; → "Contact: Email: alice at example dot com"

(regex-find-all (regex "\\b\\w+\\b") "Hello world from
Lambdust")
; → ("Hello" "world" "from" "Lambdust")

;; String patterns
(define phone-pattern (pattern "{3d}-{3d}-{4d}"))
(pattern-match phone-pattern "(555)-123-4567")
; → ((area-code . "555") (exchange . "123") (number .
"4567"))

;; Text templates
(define template (text-template "Dear {{name}},\n\nYour
balance is ${balance}."))
(template-render template
'((name . "Alice") (balance . "1,234.56"))))

;; Parsing combinators
(define number-parser
(sequence (optional (char #\-))
(one-or-more (char-range #\0 #\9))
(optional (sequence (char #\.)
(one-or-more (char-range
#\0 #\9))))))

(parse number-parser "-123.45") ; → -123.45

;; CSV parsing
(csv-read-string
"name,age,city\nAlice,25,Boston\nBob,30,Seattle")
; → (("name" "age" "city") ("Alice" "25" "Boston") ("Bob"
"30" "Seattle"))

;; Markdown parsing
(markdown->html "# Hello\n\nThis is bold text.")
; → "<h1>Hello</h1>\n<p>This is <strong>bold</strong>
text.</p>"
```

10.9. I/O Library

I/O library input output

The (`lambdust io`) library provides enhanced I/O:

Example 10.9.1 (I/O Library).

```
(import (lambdust io))
```

```
;; File operations
(with-input-from-file "data.txt"
(lambda ()
(let ((lines (read-lines)))
(filter (lambda (line) (string-contains? line
"important")) lines))))

;; Binary I/O
(with-binary-input-from-file "image.png"
```

```

(lambda ()
  (let ((header (read-bytes 8)))
    (if (png-header? header)
        (read-png-data)
        (error "Not a PNG file")))))

;; Streaming I/O
(define (process-large-file filename processor)
  (with-input-stream (file-stream filename)
    (stream-fold processor (stream-initial-value)
                  (stream-lines (current-input-stream)))))

;; Network I/O
(with-tcp-connection "api.example.com" 80
  (lambda (socket)
    (write-line "GET /data HTTP/1.0" socket)
    (write-line "" socket)
    (read-http-response socket)))

;; Asynchronous I/O
(define (async-file-read filename)
  (async-with-input-from-file filename
    (lambda ()
      (read-string (file-size filename)))))

;; Directory operations
(directory-list "/usr/local/bin") ; → list of
  files
(directory-walk "/home/user"
  (lambda (path type)
    (when (eq? type 'file)
      (display path) (newline)))))

;; Path manipulation
(path-join "/home" "user" "documents" "file.txt")
; → "/home/user/documents/file.txt"
(path-extension "document.pdf") ; → "pdf"
(path-basename "/path/to/file.txt") ; → "file.txt"

;; Temporary files
(with-temporary-file "temp-data-"
  (lambda (temp-path)
    (with-output-to-file temp-path
      (lambda () (write large-data))))
  (process-file temp-path)))

```

10.10. FFI Library

FFI library

The `(lambdust ffi)` library provides foreign function interface:

Example 10.10.1 (FFI Library).
`(import (lambdust ffi))`

```

;; Basic foreign functions
(foreign-declare "libm.so"
  (sin :: (-> Float64 Float64))
  (cos :: (-> Float64 Float64)))

;; Structure definitions
(define-c-struct Point
  (x :: Float64)
  (y :: Float64))

```

```

;; Using foreign functions
(with-capability 'math-library
  (lambda ()
    (let ((angle 1.57)) ; π/2
      (display "sin(π/2) = ")
      (display (foreign-call sin angle))
      (newline))))

;; Memory management
(define (process-with-buffer size)
  (let ((buffer (allocate-buffer size)))
    (dynamic-wind
      (lambda () #f)
      (lambda () (process-buffer buffer))
      (lambda () (free-buffer buffer)))))

;; Callback functions
(define compare-function
  (make-c-callback (-> Pointer Pointer Int32)
    (lambda (a b)
      (let ((val-a (pointer->integer a))
            (val-b (pointer->integer b)))
        (cond ((< val-a val-b) -1)
              ((> val-a val-b) 1)
              (else 0))))))

;; Array marshaling
(define numbers '(3 1 4 1 5 9))
(let ((c-array (list->c-array numbers Int32)))
  (with-capability 'libc
    (lambda ()
      (foreign-call qsort c-array 6 4 compare-function))
    (c-array->list c-array)))

;; Library loading
(define graphics-lib (load-dynamic-library
  "libgraphics.so"))
(define draw-line (get-foreign-procedure graphics-lib
  "draw_line"
  (-> Float64 Float64 Float64 Float64
  Void)))

```

10.11. Metaprogramming Library

metaprogramming reflection

The `(lambdust meta)` library provides reflection and metaprogramming:

Example 10.11.1 (Metaprogramming Library).
`(import (lambdust meta))`

```

;; Reflection
(procedure-name +) ; → "+"
(procedure-arity +) ; → (0 . #f) ; 0
  or more args
(procedure-source factorial) ; → source code
  if available

;; Environment inspection
(current-environment-variables) ; → list of
  bound variables
(variable-type 'x (current-environment)) ; → type of
  variable x

```

```
(variable-defined? 'undefined-var)      ; → #f

;; Code generation
(define-code-generator simple-loop
  (lambda (var start end body)
    `(let loop ((,var ,start))
      (if (>= ,var ,end)
          'done
          (begin ,body (loop (+ ,var 1)))))))

(simple-loop i 0 10 (display i))

;; Macro introspection
(macro-transformer? when)                ; → #t
(expand-macro-once '(when (> x 0) (display "positive")))
; → (if (> x 0) (begin (display "positive")))

;; Runtime compilation
(define dynamic-procedure
  (compile-lambda '(lambda (x y) (+ (* x x) (* y y)))))

(dynamic-procedure 3 4)                   ; → 25

;; Module introspection
(module-exports '(scheme base))           ; → list of
  exported names
(module-imports '(myapp main))            ; → list of
  imported modules

;; Performance profiling
(with-profiler
  (lambda ()
    (expensive-computation))
  (lambda (profile)
    (display-profile-report profile)))

;; Memory introspection
(gc-stats)                               ; → GC statistics
(object-size my-large-structure)          ; → approximate
  size in bytes
(heap-usage)                             ; → current heap
  usage
```

10.12. SRFI Support

SRFI Scheme Requests for Implementation

Lambdust provides extensive SRFI support:

Example 10.12.1 (SRFI Libraries).

```
;; SRFI-1: List library
(import (srfi 1))
(take '(1 2 3 4 5) 3)                    ; → (1 2 3)
(unfold negative? (lambda (x) x) (lambda (x) (- x 1)) 5)
; → (5 4 3 2 1 0)

;; SRFI-13: String library
(import (srfi 13))
(string-tokenize "Hello, world!")        ; → ("Hello"
  "world")
(string-pad-right "hello" 10)            ; → "hello  "

;; SRFI-41: Streams
(import (srfi 41))
(define nats (stream-iterate (lambda (x) (+ x 1)) 0))
```

```
(stream->list (stream-take nats 10))      ; → (0 1 2 3 4 5
  6 7 8 9)

;; SRFI-113: Sets and bags
(import (srfi 113))
(define s (set number-comparator 1 2 3))
(set-contains? s 2)                      ; → #t

;; SRFI-121: Generators
(import (srfi 121))
(define g (list->generator '(1 2 3 4 5)))
(generator->list (gtake g 3))             ; → (1 2 3)

;; SRFI-128: Comparators
(import (srfi 128))
(define string-ci-comparator
  (make-comparator string? string-ci=? string-ci<?
    string-ci-hash))

;; SRFI-151: Bitwise operations
(import (srfi 151))
(bitwise-and 6 10)                      ; → 2
(arithmetic-shift 1 10)                  ; → 1024

;; SRFI-158: Generators and accumulators
(import (srfi 158))
(define acc (list-accumulator))
(acc 1) (acc 2) (acc 3)                  ; → (1 2 3)
(acc)
```

10.13. Testing Library

testing unit tests

The (lambdust test) library provides testing facilities:

Example 10.13.1 (Testing Library).

```
(import (lambdust test))

;; Basic assertions
(test-assert (> 5 3))
(test-equal 4 (+ 2 2))
(test-approximate 3.14159 22/7 0.01)

;; Test groups
(test-group "arithmetic tests"
  (test-equal "addition" 7 (+ 3 4))
  (test-equal "multiplication" 12 (* 3 4))
  (test-equal "division" 2 (/ 8 4)))

;; Property-based testing
(test-property "addition is commutative"
  (lambda (x y) (= (+ x y) (+ y x)))
  (generate-integers -100 100))

;; Test fixtures
(define-test-fixture database-tests
  (setup (lambda () (init-test-database)))
  (teardown (lambda () (cleanup-test-database))))

(test-case "user creation"
  (let ((user (create-user "testuser"))))
    (test-assert (user? user))
    (test-equal "testuser" (user-name user))))
```

```

(test-case "user deletion"
  (create-user "tempuser")
  (delete-user "tempuser")
  (test-equal #f (find-user "tempuser")))

;; Mock objects
(define-mock database-connection
  (query (lambda (sql) '((id 1) (name "Alice"))))
  (execute (lambda (sql) 'success)))

(with-mock database-connection
  (test-equal '((id 1) (name "Alice"))
    (database-query "SELECT * FROM users")))

;; Benchmark tests
(test-benchmark "list operations"
  (benchmark "append" (lambda () (append (range 0 1000)
    (range 1000 2000))))
  (benchmark "map" (lambda () (map (lambda (x) (* x 2))
    (range 0 1000)))))

;; Coverage analysis
(with-coverage-analysis
  (run-all-tests)
  (generate-coverage-report))

```

10.14. Debugging Library

debugging tracing

The `(lambdust debug)` library provides debugging tools:

Example 10.14.1 (Debugging Library).
`(import (lambdust debug))`

```

;; Tracing
(trace factorial)
(factorial 5)
;; Trace output:
;; | (factorial 5)
;; | (factorial 4)
;; | | (factorial 3)
;; | | | (factorial 2)
;; | | | | (factorial 1)
;; | | | | 1
;; | | | 1
;; | | 2
;; | 6
;; 120

;; Breakpoints
(define (buggy-function x)
  (let ((intermediate (complex-calculation x)))
    (breakpoint "checking intermediate value"
      intermediate)
    (final-calculation intermediate)))

;; Step debugging
(step-debug
  (lambda ()
    (let ((x 10))
      (let ((y (* x 2)))
        (+ x y))))))

;; Profiling

```

```

(profile fibonacci 30)
;; Shows time spent in each function call

;; Memory debugging
(with-memory-debugging
  (lambda ()
    (let ((large-list (make-list 100000 'item)))
      (process-list large-list))))
;; Reports memory allocations and potential leaks

;; Call stack inspection
(define (deep-function)
  (display-call-stack)
  (display-environment-variables))

;; Assertion debugging
(assert (> x 0) "x must be positive" x)
(debug-assert (sorted? lst) "list must be sorted" lst)

;; Interactive debugging
(debugger-break-on-error #t)
;; Drops into REPL when error occurs

;; Performance monitoring
(with-performance-monitor
  (lambda ()
    (large-computation))
  (lambda (stats)
    (display "Execution time: ") (display (stats 'time))
    (newline)
    (display "Memory used: ") (display (stats 'memory))
    (newline)))

```

The standard library provides comprehensive functionality while maintaining R7RS compatibility and enabling advanced programming techniques through gradual adoption of Lambdust's extended features.

11. Formal Denotational Semantics

This section provides a formal denotational semantics for Lambdust, extending R7RS-small with advanced language features including gradual dependent typing, effect systems, concurrency, and FFI. The semantics is designed to support mechanical verification with Isabelle/HOL and automated translation from Event-B specifications.

The notation extends the R7RS conventions [2] with Lambdust-specific domains and operations. The core R7RS semantics is preserved as a conservative extension.

$\langle \dots \rangle$	sequence formation
$s \downarrow k$	k th member of the sequence s (1-based)
$\#s$	length of sequence s
$s \S t$	concatenation of sequences s and t
$s \upharpoonright k$	drop the first k members of sequence s
$t \rightarrow a, b$	McCarthy conditional “if t then a else b ”
$\rho[x/i]$	substitution “ ρ with x for i ”
$x \text{ in } \mathbf{D}$	injection of x into domain $\mathbf{D}$
$x \mid \mathbf{D}$	projection of x to domain $\mathbf{D}$
$\llbracket P \rrbracket$	denotational meaning of program P
$\models$	semantic entailment

The Lambdust semantics maintains R7RS compatibility while providing rigorous foundations for:

- Gradual dependent typing with type-level computation
- Effect systems with algebraic effects and handlers
- Actor-based concurrency with fault tolerance
- Memory-safe FFI with capability-based security
- Proof-carrying code via Curry-Howard correspondence

11.0.1. Extended Abstract Syntax

$K \in \text{Con}$	constants, including quotations
$I \in \text{Ide}$	identifiers (variables)
$T \in \text{Typ}$	type expressions
$F \in \text{Eff}$	effect specifications
$E \in \text{Exp}$	expressions
$\Gamma \in \text{Com} = \text{Exp}$	commands
$P \in \text{Prf}$	proof terms

R7RS Core (unchanged):

$$\begin{aligned} \text{Exp} \longrightarrow & K \mid I \mid (E_0 \ E^*) \\ & \mid (\text{lambda } (I^*) \ \Gamma^* \ E_0) \\ & \mid (\text{lambda } (I^* \ . \ I) \ \Gamma^* \ E_0) \\ & \mid (\text{lambda } I \ \Gamma^* \ E_0) \\ & \mid (\text{if } E_0 \ E_1 \ E_2) \mid (\text{if } E_0 \ E_1) \\ & \mid (\text{set! } I \ E) \end{aligned}$$
Lambdust Type Extensions:

$\mid (: \ I \ T)$	type annotation
$\mid (\text{Pi } (I \ T_1) \ T_2)$	dependent function type
$\mid (\text{Sigma } (I \ T_1) \ T_2)$	dependent product type
$\mid (\text{Refinement } (I \ T) \ E)$	refinement type
$\mid (\text{the } T \ E)$	explicit type ascription

Effect System:

$\mid (\text{perform } I \ E)$	effect invocation
$\mid (\text{handle } E \ (I \ (I \ I^* \ E)^*))$	effect handler
$\mid (\text{lift } E)$	effect lifting

Concurrency:

$\mid (\text{spawn } E)$	actor spawning
$\mid (\text{send } E \ E)$	message sending
$\mid (\text{receive } ((E \ E)^*))$	pattern-based message reception
$\mid (\text{future } E)$	asynchronous computation
$\mid (\text{await } E)$	synchronization

FFI:

$\mid (\text{foreign-call } I \ T \ E^*)$	foreign function call
$\mid (\text{foreign-data } I \ T)$	foreign data access
$\mid (\text{capability } I \ E)$	capability-based access

Proof System:

$\mid (\text{prove } T \ P)$	proof term construction
$\mid (\text{extract } P)$	program extraction from proof
$\mid (\text{qed } P)$	proof completion

11.0.2. Extended Domain Equations

R7RS Domains (preserved):

$\alpha \in \mathbf{L}$		locations
$\nu \in \mathbf{N}$		natural numbers
$\mathbf{T}$	$= \{\text{false}, \text{true}\}$	booleans
$\mathbf{Q}$		symbols
$\mathbf{H}$		characters
$\mathbf{R}$		numbers
$\mathbf{E}_p$	$= \mathbf{L} \times \mathbf{L} \times \mathbf{T}$	pairs
$\mathbf{E}_v$	$= \mathbf{L}^* \times \mathbf{T}$	vectors
$\mathbf{E}_s$	$= \mathbf{L}^* \times \mathbf{T}$	strings
$\mathbf{M}$	$= \{\text{false}, \text{true}, \text{null}, \text{undefined}, \text{unspecified}\}$	miscellaneous
$\phi \in \mathbf{F}$	$= \mathbf{L} \times (\mathbf{E}^* \rightarrow \mathbf{P} \rightarrow \mathbf{K} \rightarrow \mathbf{C})$	procedure values
$\epsilon \in \mathbf{E}$	$= \mathbf{Q} + \mathbf{H} + \mathbf{R} + \mathbf{E}_p + \mathbf{E}_v + \mathbf{E}_s + \mathbf{M} + \mathbf{F} + \dots$	expressed values
$\sigma \in \mathbf{S}$	$= \mathbf{L} \rightarrow (\mathbf{E} \times \mathbf{T})$	stores
$\rho \in \mathbf{U}$	$= \mathbf{Ide} \rightarrow \mathbf{L}$	environments
$\theta \in \mathbf{C}$	$= \mathbf{S} \rightarrow \mathbf{A}$	command continuations
$\kappa \in \mathbf{K}$	$= \mathbf{E}^* \rightarrow \mathbf{C}$	expression continuations
$\mathbf{A}$		answers
$\mathbf{X}$		errors
$\omega \in \mathbf{P}$	$= (\mathbf{F} \times \mathbf{F} \times \mathbf{P}) + \{\text{root}\}$	dynamic points

Lambdust Type System:

$\tau \in \mathbf{T}$	$= \mathbf{T}_{\text{base}} + \mathbf{T}_{\text{dep}} + \mathbf{T}_{\text{eff}}$	types
$\mathbf{T}_{\text{base}}$	$= \{\text{Dynamic, Number, String, Boolean, Symbol}\}$	base types
$\mathbf{T}_{\text{dep}}$	$= \Pi\text{-types} + \Sigma\text{-types} + \text{Refinements}$	dependent types
$\mathbf{T}_{\text{eff}}$	$= \mathbf{T} \rightarrow \mathbf{F} \rightarrow \mathbf{T}$	effectful types
$\delta \in \mathbf{T}_{\text{ctx}}$	$= \mathbf{Ide} \rightarrow \mathbf{T}$	type environments
$\xi \in \mathbf{T}_{\text{sub}}$	$= \mathbf{T} \times \mathbf{T}$	subtyping relation

Effect System:

$\psi \in \mathbf{F}$	$= \mathcal{P}(\text{EffectLabel})$	effect sets
EffectLabel	$= \mathbf{Q} \times \mathbf{T} \times \mathbf{T}$	labeled effects
$\eta \in \text{Handler}$	$= \text{EffectLabel} \rightarrow \mathbf{F}$	effect handlers
$\mu \in \text{Monad}$	$= \text{Type} \rightarrow \text{Type}$	monadic types

Concurrency System:

$\alpha \in \text{ActorId}$	$= \mathbf{N}$	actor identifiers
$\lambda \in \text{Message}$	$= \mathbf{E} \times \text{ActorId}$	messages
$\chi \in \text{Mailbox}$	$= \text{Message}^*$	actor mailboxes
$\zeta \in \text{ActorSystem}$	$= \text{ActorId} \rightarrow \text{Actor}$	actor system state
Actor	$= \text{Behavior} \times \chi \times \text{State}$	actor structure
$\phi_f \in \text{Future}$	$= \mathbf{E} + \{\text{pending}\}$	future values

FFI System:

$\gamma \in \text{CType}$	$= \{\text{int32, int64, float64, ptr, ...}\}$	C data types
$\iota \in \text{FFIBinding}$	$= \mathbf{Q} \times \text{CType} \times \mathbf{L}$	foreign bindings
$\kappa_c \in \text{Capability}$	$= \text{Resource} \times \text{Permission}$	access capabilities
Resource	$= \text{LibraryId} \times \text{FunctionId}$	foreign resources
Permission	$= \{\text{read, write, exec}\}$	access permissions

Proof System:

$\pi \in \mathbf{P}$	$= \text{ProofTerm}$	proof terms
ProofTerm	$= \lambda\text{-terms} + \text{Induction} + \text{Tactics}$	proof structures
$\varphi \in \text{Proposition}$	$= \mathbf{T}$	propositions as types
$\Gamma \in \text{Context}$	$= \mathbf{Ide} \rightarrow \mathbf{T}$	proof contexts

11.0.3. Extended Semantic Functions

R7RS Core (preserved): $\mathcal{K} : \text{Con} \rightarrow \mathbf{E}$ $\mathcal{E} : \text{Exp} \rightarrow \mathbf{U} \rightarrow \mathbf{P} \rightarrow \mathbf{K} \rightarrow \mathbf{C}$ $\mathcal{E}^* : \text{Exp}^* \rightarrow \mathbf{U} \rightarrow \mathbf{P} \rightarrow \mathbf{K} \rightarrow \mathbf{C}$ $\mathcal{C} : \text{Com}^* \rightarrow \mathbf{U} \rightarrow \mathbf{P} \rightarrow \mathbf{C} \rightarrow \mathbf{C}$ **Lambdust Extensions:** $\mathcal{T} : \text{Typ} \rightarrow \text{TypeDenotation}$ $\mathcal{F} : \text{Eff} \rightarrow \text{EffectDenotation}$ $\mathcal{A} : \text{ActorExp} \rightarrow \text{ActorSystem} \rightarrow \text{ActorSystem}$ $\mathcal{I} : \text{FFIExp} \rightarrow \text{Capability} \rightarrow \mathbf{E}$ $\mathcal{P} : \text{Prf} \rightarrow \text{ProofDenotation}$

11.0.4. Type System Semantics

The type system provides gradual dependent typing with four levels:

1. **Dynamic:** No static checking, full R7RS compatibility
2. **Contracts:** Runtime type checking with gradual migration
3. **Static:** Hindley-Milner style inference with extensions
4. **Dependent:** Full dependent types with proof obligations

$$\mathcal{T}[\text{Dynamic}] = \lambda\rho \epsilon . \epsilon$$

$$\begin{aligned} \mathcal{T}[(\text{Pi}(I T_1) T_2)] = \\ \lambda\rho . \{ \epsilon \mid \epsilon \in F \wedge \\ \forall \epsilon' \in \mathcal{T}[T_1] \rho . \\ \text{aplicate } \epsilon(\epsilon') \in \mathcal{T}[T_2](\rho[\epsilon'/I]) \} \end{aligned}$$

$$\begin{aligned} \mathcal{T}[(\text{Refinement}(I T) E)] = \\ \lambda\rho . \{ \epsilon \mid \epsilon \in \mathcal{T}[T] \rho \wedge \\ \text{truish}(\mathcal{E}[E](\rho[\epsilon/I])) \} \end{aligned}$$

11.0.5. Effect System Semantics

Effects are modeled as algebraic effects with handlers following the approach of [1]. The effect system ensures that side effects are properly tracked and contained.

$$\begin{aligned} \mathcal{E}[(\text{perform} I E)] = \\ \lambda\rho\omega\kappa . \mathcal{E}[E]\rho\omega \\ (\text{single}(\lambda\epsilon . \text{invoke-effect } I \epsilon \omega \kappa)) \end{aligned}$$

$$\begin{aligned} \mathcal{E}[(\text{handle} E (I (I_i I_i^* E_i)^*))] = \\ \lambda\rho\omega\kappa . \mathcal{E}[E]\rho \\ (\text{extend-handler } \omega (\text{make-handler } (I_i I_i^* E_i)^* \rho)) \\ \kappa \end{aligned}$$

$$\begin{aligned} \text{invoke-effect} : Q \rightarrow E \rightarrow P \rightarrow K \rightarrow C \\ \text{invoke-effect} = \\ \lambda\text{label} \epsilon \omega \kappa . \text{find-handler label } \omega (\\ \lambda\text{handler} . \text{handler } \epsilon \kappa, \\ \text{wrong "unhandled effect: label"}) \end{aligned}$$

11.0.6. Actor System Semantics

The actor system provides fault-tolerant concurrency with supervision trees and location transparency.

$$\begin{aligned} \mathcal{E}[(\text{spawn} E)] = \\ \lambda\rho\omega\kappa . \lambda\zeta . \text{fresh-actor-id } \zeta (\\ \lambda\alpha . \text{send } \alpha \kappa \\ (\text{spawn-actor } \alpha (\mathcal{E}[E]\rho\omega) \zeta)) \end{aligned}$$

$$\begin{aligned} \mathcal{E}[(\text{send} E_1 E_2)] = \\ \lambda\rho\omega\kappa . \mathcal{E}[E_1]\rho\omega (\\ \text{single}(\lambda\alpha . \mathcal{E}[E_2]\rho\omega (\\ \text{single}(\lambda\epsilon . \text{enqueue-message } \alpha \epsilon \kappa)))) \end{aligned}$$

$$\begin{aligned} \text{spawn-actor} : \text{ActorId} \rightarrow E \rightarrow \text{ActorSystem} \rightarrow \text{ActorSystem} \\ \text{spawn-actor} = \\ \lambda\alpha \text{behavior } \zeta . \zeta[\alpha \mapsto (\text{behavior}, \langle \rangle, \text{fresh-state})] \end{aligned}$$

11.0.7. FFI System Semantics

The FFI provides capability-based access to foreign functions with automatic memory management and type safety.

$$\begin{aligned} \mathcal{E}[(\text{foreign-call} I T E^*)] = \\ \lambda\rho\omega\kappa . \mathcal{E}^*[E^*]\rho\omega (\\ \lambda\epsilon^* . \text{check-capability } I (\\ \lambda\text{cap} . \text{marshal-call } I T \epsilon^* \text{cap} \kappa)) \end{aligned}$$

$$\begin{aligned} \text{marshal-call} : Q \rightarrow T \rightarrow E^* \rightarrow \text{Capability} \rightarrow K \rightarrow C \\ \text{marshal-call} = \\ \lambda\text{fname ret-type } \epsilon^* \text{cap} \kappa . \text{convert-args } \epsilon^* (\\ \lambda\text{c-args} . \text{invoke-foreign } \text{fname c-args } (\\ \lambda\text{c-result} . \text{convert-result ret-type c-result } \kappa)) \end{aligned}$$

11.0.8. Proof System Semantics

The proof system implements Curry-Howard correspondence, enabling program extraction from constructive proofs.

$$\begin{aligned} \mathcal{P}[(\text{prove} T P)] = \\ \lambda\Gamma . \text{check-proof } P T \Gamma (\\ \lambda\text{verified} . \text{verified} \rightarrow \mathcal{P}[P]\Gamma, \\ \text{wrong "proof verification failed"}) \end{aligned}$$

$$\begin{aligned} \mathcal{P}[(\text{extract} P)] = \\ \lambda\Gamma . \text{extract-program } (\mathcal{P}[P]\Gamma) \end{aligned}$$

$$\begin{aligned} \text{check-proof} : P \rightarrow T \rightarrow \text{Context} \rightarrow (T \rightarrow C) \rightarrow C \\ \text{check-proof} = \\ \lambda\text{proof prop } \Gamma \kappa . \text{type-of-proof } \text{proof } \Gamma (\\ \lambda\text{inferred} . \text{types-equivalent inferred prop} \rightarrow \\ \kappa \text{ true}, \\ \kappa \text{ false}) \end{aligned}$$

11.0.9. Conservative Extension Properties

The Lambdust semantics satisfies the following key properties:

R7RS Compatibility: For any R7RS-compliant program P , $\mathcal{E}_{\text{Lambdust}}[P] = \mathcal{E}_{\text{R7RS}}[P]$ when run in Dynamic mode.

Type Safety: Well-typed programs cannot go wrong:

$$\vdash P : \tau \implies \forall \sigma . \mathcal{E}[P]\emptyset\text{root}(\lambda\epsilon . \epsilon) \neq \text{wrong}$$

Effect Safety: Effects are properly contained by handlers:

$$\vdash P : \tau \text{ in } \Psi \implies \text{effects}(\mathcal{E}[P]) \subseteq \Psi$$

Memory Safety: FFI operations cannot access unauthorized memory:

$$\text{safe}(\text{capability}) \implies \text{safe}(\mathcal{I}[\llbracket \text{ffi-call} \rrbracket] \text{capability})$$

Proof Soundness: Extracted programs satisfy their specifications:

$$\vdash P : \text{proof-of}(\phi) \implies \mathcal{E}[\llbracket \text{extract}(P) \rrbracket] \models \phi$$

11.0.10. Relationship to Event-B and Isabelle/HOL

The semantics is designed to facilitate:

- **Event-B Translation:** Each Lambdust construct maps to Event-B machines with precisely defined refinement relationships.
- **Isabelle/HOL Verification:** Domain equations and semantic functions are directly expressible in Isabelle/HOL for mechanical verification.
- **Proof Carrying Code:** The proof system enables automatic generation of machine-checkable correctness certificates.

The complete formal development provides a mathematically rigorous foundation for high-assurance software development using functional programming with dependent types.

12. Examples

This section provides comprehensive examples demonstrating Lambdust's features in realistic programming scenarios, showing how the various language components work together.

12.1. Web Server with Effects

A complete web server implementation using effects for I/O, state management, and error handling:

Example 12.1.1 (Web Server Implementation).

```
(import (lambdust base)
  (lambdust effects)
  (lambdust concurrent)
  (lambdust collections)
  (lambdust text))

;; Define HTTP-related effects
(define-effect HTTP
  (read-request :: (-> Unit (Map String String)))
  (write-response :: (-> Natural String String Unit))
  (log-access :: (-> String Unit)))

;; Define application state effect
(define-effect AppState
  (get-counter :: (-> Unit Natural))
  (increment-counter :: (-> Unit Unit))
  (get-users :: (-> Unit (List String)))
  (add-user :: (-> String Unit)))

;; Route handler type
(define RouteHandler :: (-> (Map String String) ~> {HTTP, AppState} Unit))

;; Individual route handlers
(define (handle-home :: RouteHandler)
  (lambda (request)
    (let ((count (perform AppState get-counter)))
      (perform HTTP write-response 200 "text/html"
        (format "<h1>Welcome!</h1><p>Visits: ~d</p>" count))))))

(define (handle-api-counter :: RouteHandler)
  (lambda (request)
    (perform AppState increment-counter)
    (let ((count (perform AppState get-counter)))
      (perform HTTP write-response 200 "application/json"
        (format "{\"count\": ~d}" count)))))

(define (handle-api-users :: RouteHandler)
  (lambda (request)
    (let ((method (map-ref request "method")))
      (cond
        ((string=? method "GET")
         (let ((users (perform AppState get-users)))
           (perform HTTP write-response 200 "application/json"
             (format "{\"users\": [~a]}"
              (string-join (map (lambda (u) (format "~a" u)) users) ","))))))
        ((string=? method "POST")
         (let ((body (map-ref request "body")))
           (let ((user-data (json-parse body)))
             (let ((name (map-ref user-data "name")))
               (if name
                 (begin
                  (perform AppState add-user name)
                  (perform HTTP write-response 201 "application/json"
                    (format "{\"message\": \"User ~a created\"" name)))
                  (perform HTTP write-response 400 "application/json"
                    "{\"error\": \"Name required\"}")))))
             (format "{\"error\": \"Name required\"}")))))))))))
```

```

;; Routing system
(define (route-request :: (-> (Map String String) ~> {HTTP, AppState} Unit))
  (lambda (request)
    (let ((path (map-ref request "path"))
          (method (map-ref request "method")))
      (perform HTTP log-access (format "~a ~a" method path))
      (cond
        ((string=? path "/") (handle-home request))
        ((string=? path "/api/counter") (handle-api-counter request))
        ((string=? path "/api/users") (handle-api-users request))
        (else
         (perform HTTP write-response 404 "text/plain" "Not Found"))))))

;; Server state
(define-struct server-state
  (counter :: Natural)
  (users :: (List String)))

;; Effect handlers
(define (http-handler :: (-> (Channel HTTPRequest)
                             (-> Unit ~> {HTTP} A)
                             (-> Unit ~> {} A)))
  (lambda (request-channel computation)
    (lambda ()
      (handle (computation)
              (HTTP
               ((read-request k)
                (let ((req (channel-get request-channel)))
                  (k (request->map req))))
               ((write-response status content-type body k)
                (let ((response (make-http-response status content-type body)))
                  (send-response response)
                  (k 'unit)))
               ((log-access message k)
                (begin
                 (display "[") (display (current-timestamp)) (display "]" " ")
                 (display message) (newline)
                 (k 'unit))))))))))

(define (app-state-handler :: (-> (TVar server-state)
                                  (-> Unit ~> {AppState} A)
                                  (-> Unit ~> {STM} A)))
  (lambda (state-tvar computation)
    (lambda ()
      (handle (computation)
              (AppState
               ((get-counter k)
                (let ((state (atomic (tvar-read state-tvar))))
                  (k (server-state-counter state))))
               ((increment-counter k)
                (atomic
                 (let ((state (tvar-read state-tvar)))
                   (tvar-write! state-tvar
                                (make-server-state (+ (server-state-counter state) 1)
                                                    (server-state-users state))))
                 (k 'unit)))
               ((get-users k)
                (let ((state (atomic (tvar-read state-tvar))))
                  (k (server-state-users state))))
               ((add-user name k)
                (atomic
                 (let ((state (tvar-read state-tvar)))
                   (tvar-write! state-tvar
                                (make-server-state (server-state-counter state)
                                                    (cons name (server-state-users state))))
                 (k 'unit))))))))))

;; Main server

```


Database operations with dependent types ensuring query safety:

[illegible]

```

                                ~> {Database}
                                (List (Record schema fields))))))
(lambda (schema fields query)
  (with-capability 'database-access
    (lambda ()
      (let ((sql (compile-query query))
            (connection (perform Database get-connection)))
        (with-query-result (foreign-call PQexec connection sql)
          (lambda (result)
            (validate-result-schema result schema fields)
            (parse-result result schema fields))))))))))

;; Compile-time SQL validation
(define-syntax sql-query
  (lambda (stx)
    (syntax-case stx ()
      ((sql-query schema query-string)
       (validate-sql-syntax (syntax->datum #'query-string)
                             (syntax->datum #'schema))
       #'(compile-validated-query schema query-string))))))

;; Safe query building
(define (find-user-by-email :: (-> (Email String) ~> {Database} (Maybe (Record UserSchema))))
  (lambda (email)
    (let ((query (select UserSchema '(id name email age created_at)
                        (where (= email (param email))))))
      (let ((results (execute-query UserSchema '(id name email age created_at) query)))
        (if (null? results)
            'Nothing
            (list 'Just (car results)))))))

(define (get-user-posts :: (-> Natural ~> {Database} (List (Record PostSchema))))
  (lambda (user-id)
    (let ((query (select PostSchema '(id title content published)
                        (where (= user_id (param user-id))))))
      (execute-query PostSchema '(id title content published) query)))

;; Transaction safety
(define (create-user-with-post :: (-> String (Email String) Natural String String
                                   ~> {Database} (Either String Natural)))
  (lambda (name email age post-title post-content)
    (atomic-transaction
      (lambda ()
        ;; Validate inputs at type level
        (assert-type name (NonEmpty String))
        (assert-type email (Email String))
        (assert-type age (Range 0 120))
        (assert-type post-title (NonEmpty String))

        ;; Insert user
        (let ((user-id (perform Database insert UserSchema
                                   `((name . ,name)
                                      (email . ,email)
                                      (age . ,age)
                                      (created_at . ,(current-timestamp))))))

          ;; Insert post
          (let ((post-id (perform Database insert PostSchema
                                   `((user_id . ,user-id)
                                      (title . ,post-title)
                                      (content . ,post-content)
                                      (published . ,#f)))))

            (list 'Right user-id)))))))

;; Migration with dependent types
(define (migration-001-add-email-index :: (-> Unit ~> {Database} Unit))
  (lambda ()
    (let ((index-exists? (perform Database index-exists? 'users 'email)))

```

```

(unless index-exists?
  (perform Database create-index 'users 'email 'unique))))))

;; Database effect handler with connection pooling
(define (database-handler :: (-> ConnectionPool
                             (-> Unit ~> {Database} A)
                             (-> Unit ~> {Actor} A)))
  (lambda (pool computation)
    (lambda ()
      (handle (computation)
        (Database
          ((get-connection k)
            (let ((conn (pool-acquire pool)))
              (k conn)))
          ((execute-sql sql k)
            (with-connection-from-pool pool
              (lambda (conn)
                (let ((result (foreign-call PQexec conn sql)))
                  (k result))))))
          ((insert schema values k)
            (let ((sql (generate-insert-sql schema values)))
              (with-connection-from-pool pool
                (lambda (conn)
                  (let ((result (foreign-call PQexec conn sql)))
                    (k (extract-inserted-id result)))))))
          ((begin-transaction k)
            (perform Database execute-sql "BEGIN")
            (k 'unit))
          ((commit-transaction k)
            (perform Database execute-sql "COMMIT")
            (k 'unit))
          ((rollback-transaction k)
            (perform Database execute-sql "ROLLBACK")
            (k 'unit))))))

;; Usage example
(define (user-management-example)
  (let ((pool (create-connection-pool "postgresql://localhost/mydb" 10)))
    ((database-handler pool
      (lambda ()
        ;; Create user
        (let ((result (create-user-with-post "Alice Smith"
                                              "alice@example.com"
                                              28
                                              "My First Post"
                                              "Hello, world!")))

          (match result
            (('Right user-id)
              (display "Created user with ID: ") (display user-id) (newline)
              ;; Find the user
              (let ((user (find-user-by-email "alice@example.com")))
                (match user
                  (('Just user-record)
                    (display "Found user: ") (display (record-ref user-record 'name)) (newline)
                    ;; Get their posts
                    (let ((posts (get-user-posts user-id)))
                      (display "Posts: ") (display (length posts)) (newline)))
                  ('Nothing
                    (display "User not found") (newline))))))
            (('Left error)
              (display "Error creating user: ") (display error) (newline)))))))

```

12.3. Distributed Actor System

A fault-tolerant distributed computation system:

Example 12.3.1 (Distributed Actor System).

```

(import (lambdust base)
  (lambdust concurrent)
  (lambdust effects)
  (lambdust collections)
  (lambdust text))

;; Message types for distributed system
(define-union-type WorkerMessage
  (ComputeTask Natural (Channel Any))
  (HealthCheck (Channel Symbol))
  (Shutdown))

(define-union-type SupervisorMessage
  (RegisterWorker ActorId String)
  (WorkerFailed ActorId String)
  (DistributeWork Any (Channel Any))
  (GetStats (Channel (Map String Natural))))

;; Worker actor with failure simulation
(define (worker-actor :: (-> String Natural (-> Unit ~> {Actor} Unit)))
  (lambda (node-name failure-rate)
    (lambda ()
      (let ((processed-count 0))
        (let loop ()
          (receive
            (('ComputeTask task reply-channel)
              ;; Simulate potential failure
              (if (< (random 100) failure-rate)
                (error "Worker failure during task processing")
                (begin
                  (let ((result (expensive-computation task)))
                    (send reply-channel result)
                    (set! processed-count (+ processed-count 1))
                    (loop))))))
            (('HealthCheck reply-channel)
              (send reply-channel 'healthy)
              (loop)))
            (('Shutdown)
              (display "Worker on ") (display node-name) (display " shutting down" (newline)))))))

;; Supervisor actor with fault tolerance
(define (supervisor-actor :: (-> (List String) (-> Unit ~> {Actor} Unit)))
  (lambda (node-names)
    (lambda ()
      (let ((workers (map-from-alist '()))
            (pending-work (make-queue))
            (failed-workers (make-set)))

        ;; Spawn workers on different nodes
        (for-each
          (lambda (node)
            (let ((worker-pid (spawn-remote node (worker-actor node 5)))) ; 5% failure rate
              (set! workers (map-set workers worker-pid node))))
          node-names)

        (let loop ()
          (receive
            (('RegisterWorker pid node-name)
              (set! workers (map-set workers pid node-name))
              (display "Registered worker ") (display pid)
              (display " on node ") (display node-name) (newline)
              (loop)))
            (('WorkerFailed pid reason)
              (display "Worker ") (display pid) (display " failed: ") (display reason) (newline)
              (set! failed-workers (set-add failed-workers pid))))))


```

```

    (set! workers (map-remove workers pid))

    ;; Respawn worker on same node
    (let ((node-name (map-ref workers pid)))
      (when node-name
        (let ((new-worker (spawn-remote node-name (worker-actor node-name 5))))
          (set! workers (map-set workers new-worker node-name))
          (display "Respawned worker ") (display new-worker)
          (display " on node ") (display node-name) (newline))))
      (loop))

    (('DistributeWork work reply-channel)
      (if (map-empty? workers)
          (send reply-channel 'no-workers-available)
          (let ((worker-pid (least-loaded-worker workers)))
            (send worker-pid (list 'ComputeTask work reply-channel))))
      (loop))

    (('GetStats reply-channel)
      (let ((stats (map-from-alist
                    `((active-workers . ,(map-size workers))
                      (failed-workers . ,(set-size failed-workers))
                      (pending-work . ,(queue-length pending-work))))))
        (send reply-channel stats))
      (loop))))))

;; Health monitoring actor
(define (health-monitor :: (-> ActorId (-> Unit ~> {Actor} Unit)))
  (lambda (supervisor-pid)
    (lambda ()
      (let loop ()
        (sleep 5000) ; Check every 5 seconds

        ;; Get supervisor stats
        (send supervisor-pid (list 'GetStats self))
        (receive
          ((stats)
            (display "System health - Active workers: ")
            (display (map-ref stats 'active-workers))
            (display ", Failed workers: ")
            (display (map-ref stats 'failed-workers)) (newline))

          ;; Alert if too many failures
          (when (> (map-ref stats 'failed-workers) 3)
            (display "WARNING: High failure rate detected!") (newline))))
        (loop))))))

;; Load balancer for distributing work
(define (load-balancer :: (-> ActorId (-> Unit ~> {Actor} Unit)))
  (lambda (supervisor-pid)
    (lambda ()
      (let ((work-queue (make-channel))
            (active-tasks (map-from-alist '())))

        ;; Spawn work generator
        (spawn
          (lambda ()
            (do ((i 0 (+ i 1)))
                ((>= i 100))
              (channel-put! work-queue i)
              (sleep 100)))) ; Generate work every 100ms

        (let loop ()
          (select
            ;; New work available
            ((work-queue work)
              (let ((result-channel (make-channel)))
                (send supervisor-pid (list 'DistributeWork work result-channel))

```

```

    (set! active-tasks (map-set active-tasks work result-channel))
    (display "Distributed work: ") (display work) (newline)))

;; Check for completed work
(timeout 1000
 (for-each
  (lambda (work-result-pair)
    (let ((work (car work-result-pair))
          (result-channel (cdr work-result-pair)))
      (select
       ((result-channel result)
        (display "Work ") (display work)
        (display " completed with result: ") (display result) (newline)
        (set! active-tasks (map-remove active-tasks work)))
       (timeout 0 'no-result))))
    (map->alist active-tasks))))

(loop))))))

;; Network partition handler
(define (partition-handler :: (-> (List ActorId) (-> Unit ~> {Actor} Unit)))
  (lambda (critical-actors)
    (lambda ()
      (let loop ()
        (sleep 10000) ; Check every 10 seconds

        (for-each
         (lambda (actor-pid)
           (with-timeout 2000 ; 2 second timeout
            (lambda ()
              (send actor-pid (list 'HealthCheck self))
              (receive
               (('healthy) 'ok)))
            (lambda () ; Timeout handler
              (display "Actor ") (display actor-pid)
              (display " appears unreachable - possible network partition") (newline)
              ;; Could implement partition tolerance logic here
              )))
          critical-actors)

        (loop))))))

;; Main distributed system
(define (run-distributed-system :: (-> (List String) Unit ~> {Actor} Unit))
  (lambda (node-names)
    (display "Starting distributed computation system..." (newline)

    ;; Start supervisor
    (let ((supervisor (spawn (supervisor-actor node-names))))
      (display "Supervisor started: ") (display supervisor) (newline)

      ;; Start health monitor
      (let ((monitor (spawn (health-monitor supervisor))))
        (display "Health monitor started: ") (display monitor) (newline))

      ;; Start load balancer
      (let ((balancer (spawn (load-balancer supervisor))))
        (display "Load balancer started: ") (display balancer) (newline))

      ;; Start partition handler
      (let ((partition-handler (spawn (partition-handler (list supervisor)))))
        (display "Partition handler started: ") (display partition-handler) (newline))

      ;; Keep main thread alive
      (let loop ()
        (sleep 30000) ; Status every 30 seconds
        (display "System running..." (newline)
        (loop))))))

```

```
;; Helper functions
(define (expensive-computation :: (-> Natural Natural))
  (lambda (n)
    ;; Simulate expensive computation
    (let ((result 0))
      (do ((i 0 (+ i 1)))
          ((>= i (* n 1000)) result)
        (set! result (+ result (modulo i 100)))))))

(define (least-loaded-worker :: (-> (Map ActorId String) ActorId))
  (lambda (workers)
    ;; Simple round-robin for this example
    (car (map-keys workers))))

;; Usage
(run-distributed-system '("node1.cluster.local"
                          "node2.cluster.local"
                          "node3.cluster.local"))
```

12.4. Compiler with Dependent Types

A mini-compiler demonstrating dependent types for ensuring type safety:

Example 12.4.1 (Type-Safe Compiler).

```
(import (lambdust base)
        (lambdust dependent-types)
        (lambdust effects)
        (lambdust collections))

;; AST types with dependent typing
(define-inductive-type (Expr ctx)
  (Var :: (Pi (name : Symbol) (-> (name \in ctx) (Expr ctx))))
  (Lit :: (-> Natural (Expr ctx)))
  (Add :: (-> (Expr ctx) (Expr ctx) (Expr ctx)))
  (Let :: (Pi (name : Symbol) (-> (Expr ctx)
                                   (Expr (extend-context ctx name Natural))
                                   (Expr ctx))))
  (App :: (Pi (A B : Type) (-> (Expr ctx) ; Function
                                (Expr ctx) ; Argument
                                (constraint :: (typeof-expr func ctx) = (-> A B))
                                (constraint :: (typeof-expr arg ctx) = A)
                                (Expr ctx)))))

;; Type context operations
(define-type Context (List (Pair Symbol Type)))

(define (empty-context :: Context) '())

(define (extend-context :: (-> Context Symbol Type Context))
  (lambda (ctx name type)
    (cons (cons name type) ctx)))

(define (lookup-context :: (-> Context Symbol (Maybe Type)))
  (lambda (ctx name)
    (let ((entry (assq name ctx)))
      (if entry (list 'Just (cdr entry)) 'Nothing))))

;; Dependent type checking
(define (typeof-expr :: (Pi (ctx : Context)
                             (-> (Expr ctx) Type)))
  (lambda (ctx expr)
    (match expr
      (('Var name proof-in-context)
       (match (lookup-context ctx name)
         (('Just type) type)
```

```

    ('Nothing (error "Unbound variable" name))))

(('Lit n)
 'Natural)

(('Add e1 e2)
 (let ((t1 (typeof-expr ctx e1))
       (t2 (typeof-expr ctx e2)))
   (if (and (eq? t1 'Natural) (eq? t2 'Natural))
       'Natural
       (error "Type error in addition" t1 t2))))

(('Let name binding body)
 (let ((binding-type (typeof-expr ctx binding))
       (extended-ctx (extend-context ctx name binding-type)))
   (typeof-expr extended-ctx body)))

(('App func arg)
 (let ((func-type (typeof-expr ctx func))
       (arg-type (typeof-expr ctx arg)))
   (match func-type
     (('-> from to)
      (if (type-equal? arg-type from)
          to
          (error "Argument type mismatch" arg-type from)))
     (other
      (error "Not a function type" other))))))

;; Well-typed expression construction
(define (make-var :: (Pi (ctx : Context) (name : Symbol)
  (-> (proof :: (name \in ctx))
      (Expr ctx))))
  (lambda (ctx name proof)
    (list 'Var name proof)))

(define (make-lit :: (Pi (ctx : Context) (-> Natural (Expr ctx))))
  (lambda (ctx n)
    (list 'Lit n)))

(define (make-add :: (Pi (ctx : Context)
  (-> (e1 : (Expr ctx))
      (e2 : (Expr ctx))
      (constraint :: (typeof-expr ctx e1) = Natural)
      (constraint :: (typeof-expr ctx e2) = Natural)
      (Expr ctx))))
  (lambda (ctx e1 e2 proof1 proof2)
    (list 'Add e1 e2)))

;; Compiler phases with proofs
(define-effect Compiler
  (fresh-var :: (-> Unit Symbol))
  (emit-code :: (-> String Unit))
  (get-env :: (-> Unit (Map Symbol Natural)))
  (set-env :: (-> Symbol Natural Unit)))

(define (compile-expr :: (Pi (ctx : Context)
  (-> (expr : (Expr ctx))
      (proof :: (well-typed? expr ctx))
      ~> {Compiler} Unit))))
  (lambda (ctx expr proof)
    (match expr
      (('Var name _)
       (perform Compiler emit-code (format "LOAD ~a" name)))

      (('Lit n)
       (perform Compiler emit-code (format "CONST ~d" n)))

      (('Add e1 e2)
       (compile-expr ctx e1 (extract-proof-1 proof))

```



```

    (compile-expr ctx e2 (extract-proof-2 proof))
    (perform Compiler emit-code "ADD"))

(('Let name binding body)
 (compile-expr ctx binding (extract-binding-proof proof))
 (let ((var-slot (perform Compiler fresh-var)))
   (perform Compiler emit-code (format "STORE ~a" var-slot))
   (let ((extended-ctx (extend-context ctx name 'Natural)))
     (compile-expr extended-ctx body (extract-body-proof proof)))))

(('App func arg)
 (compile-expr ctx arg (extract-arg-proof proof))
 (compile-expr ctx func (extract-func-proof proof))
 (perform Compiler emit-code "CALL"))))

;; Code generation with optimization
(define (optimize-code :: (-> (List String) ~> {Compiler} (List String)))
  (lambda (instructions)
    (let ((optimized '()))
      (let loop ((instrs instructions))
        (match instrs
          (()
            (reverse optimized))

          ;; Constant folding
          (((CONST n1) (CONST n2) ADD . rest)
            (loop (cons (format "CONST ~d" (+ n1 n2)) rest)))

          ;; Dead code elimination
          ((STORE var LOAD var . rest)
            (loop rest)) ; Store followed by load of same var

          ((instr . rest)
            (set! optimized (cons instr optimized))
            (loop rest))))))

;; Compiler with error handling
(define (compile-program :: (-> (Expr empty-context)
                                (Either String (List String))))
  (lambda (program)
    (with-exception-handler
      (lambda (e)
        (list 'Left (format "Compilation error: ~a" e)))
      (lambda ()
        (let ((instructions '())
              (var-counter 0)
              (env (map-from-alist '())))
          ((handle
            (lambda ()
              (compile-expr empty-context program
                (derive-well-typed-proof program empty-context))
              (list 'Right (reverse instructions)))

            (Compiler
              ((fresh-var k)
                (let ((var (format "tmp~d" var-counter)))
                  (set! var-counter (+ var-counter 1))
                  (k var)))

              ((emit-code code k)
                (set! instructions (cons code instructions))
                (k 'unit))

              ((get-env k)
                (k env))

              ((set-env var val k)
                (set! env (map-set env var val))
```

```

(k 'unit))))))))))

;; Virtual machine with type safety
(define (run-vm :: (-> (List String) (Map Symbol Natural) Natural))
  (lambda (instructions env)
    (let ((stack '())
          (pc 0))
      (let loop ()
        (if (>= pc (length instructions))
            (if (null? stack)
                0
                (car stack))
            (let ((instr (list-ref instructions pc)))
              (match (string-split instr " ")
                (("CONST" n-str)
                 (set! stack (cons (string->number n-str) stack))
                 (set! pc (+ pc 1)))

                ("LOAD" var)
                (let ((val (map-ref env var)))
                  (if val
                      (begin
                        (set! stack (cons val stack))
                        (set! pc (+ pc 1))
                        (error "Undefined variable" var))))

                ("STORE" var)
                (if (null? stack)
                    (error "Stack underflow")
                    (begin
                     (set! env (map-set env var (car stack)))
                     (set! stack (cdr stack))
                     (set! pc (+ pc 1))))

                ("ADD")
                (if (< (length stack) 2)
                    (error "Stack underflow")
                    (let ((b (car stack))
                          (a (cadr stack)))
                      (set! stack (cons (+ a b) (cddr stack)))
                      (set! pc (+ pc 1))))

                (other
                 (error "Unknown instruction" other))))

        (loop))))))

;; Example usage with proofs
(define example-program
  ;; let x = 5 in let y = 10 in x + y
  (make-let empty-context 'x
    (make-lit empty-context 5)
    (make-let (extend-context empty-context 'x 'Natural) 'y
      (make-lit (extend-context empty-context 'x 'Natural) 10)
      (make-add (extend-context
        (extend-context empty-context 'x 'Natural) 'y 'Natural)
        (make-var (extend-context
          (extend-context empty-context 'x 'Natural) 'y 'Natural)
            'x (proof-x-in-extended-context))
        (make-var (extend-context
          (extend-context empty-context 'x 'Natural) 'y 'Natural)
            'y (proof-y-in-extended-context))
        (proof-x-is-natural) (proof-y-is-natural))))))

;; Compile and run
(define (demo-compiler)
  (match (compile-program example-program)
    (('Right instructions)
     (display "Compiled instructions:") (newline)))

```

```

(for-each (lambda (instr) (display " ") (display instr) (newline)) instructions)
(let ((result (run-vm instructions (map-from-alist '()))))
  (display "Result: ") (display result) (newline)))
(('Left error)
 (display "Compilation failed: ") (display error) (newline)))

```

12.5. Proof-Carrying Code

An example demonstrating program extraction from proofs:

Example 12.5.1 (Proof-Carrying Code).

```

(import (lambdust base)
        (lambdust dependent-types)
        (lambdust effects))

;; Mathematical propositions as types
(define-type (Even n) (Exists (k : Natural) (= n (* 2 k))))
(define-type (Odd n) (Exists (k : Natural) (= n (+ (* 2 k) 1))))
(define-type (Prime n) (And (> n 1) (Forall (k : Natural)
                                             (-> (Divides k n)
                                                  (Or (= k 1) (= k n))))))

;; Verified sorting algorithm
(theorem insertion-sort-correct :: (Pi (A : Type) (ord : (TotalOrder A))
                                     (xs : (List A))
                                     (Exists (ys : (List A))
                                              (And (Sorted ord ys)
                                                    (Permutation xs ys)))))

(proof-by-induction xs
 (base-case
  ;; Empty list is sorted and permutation of itself
  (witness '() (pair (sorted-nil ord) (permutation-nil))))

 (inductive-step x xs ih
  ;; ih : Exists (ys : List A) (And (Sorted ord ys) (Permutation xs ys))
  (match ih
   ((witness sorted-xs (pair sorted-proof perm-proof))
    ;; Insert x into sorted list
    (let ((inserted-list (insert ord x sorted-xs)))
     (witness inserted-list
      (pair (insert-preserves-sorted ord x sorted-xs sorted-proof)
            (insert-preserves-permutation ord x xs sorted-xs perm-proof)))))))

;; Extract the sorting algorithm
(define insertion-sort (extract insertion-sort-correct))
;; insertion-sort :: (Pi (A : Type) (-> (TotalOrder A) (List A) (List A)))

;; Binary search with proof of correctness
(theorem binary-search-correct :: (Pi (A : Type) (ord : (TotalOrder A))
                                     (xs : (SortedList ord A)) (x : A)
                                     (Exists (result : (Maybe Natural))
                                              (Match result
                                               ('Nothing (Not (Member x xs)))
                                               ('Just index) (And (< index (length xs))
                                                                (= (list-ref xs index) x))))))

(proof
 (let ((search-result (binary-search-impl ord xs x 0 (length xs))))
  (witness search-result (binary-search-correctness-proof ord xs x 0 (length xs)))))

;; The implementation with proof
(define (binary-search-impl :: (Pi (A : Type) (ord : (TotalOrder A))
                                   (-> (xs : (SortedList ord A)) (x : A)
                                       (low high : Natural)
                                       (constraint :: (<= low high))
                                       (constraint :: (<= high (length xs)))))

```

```

                                (Maybe Natural))))
(lambda (A ord xs x low high low-le-high high-le-length)
  (if (= low high)
      'Nothing
      (let ((mid (quotient (+ low high) 2)))
        (let ((mid-val (list-ref xs mid)))
          (match (compare ord x mid-val)
            ('equal (list 'Just mid))
            ('less (binary-search-impl A ord xs x low mid
                                     (quotient-preserves-order low high)
                                     (mid-le-high mid high)))
            ('greater (binary-search-impl A ord xs x (+ mid 1) high
                                     (mid-plus-1-ge-low mid low)
                                     high-le-length)))))))

;; Extract verified binary search
(define binary-search (extract binary-search-correct))

;; Verified quicksort with complexity proof
(theorem quicksort-correct-and-efficient ::
  (Pi (A : Type) (ord : (TotalOrder A)) (xs : (List A))
    (Exists (ys : (List A))
      (And (Sorted ord ys)
            (Permutation xs ys)
            (Complexity (quicksort-time-complexity (length xs)))))))
(proof
  ;; Proof by strong induction on list length
  (strong-induction (length xs)
    (base-case-empty
      (witness '() (triple (sorted-nil ord) (permutation-nil) (0 1))))

    (base-case-singleton x
      (witness (list x) (triple (sorted-singleton ord x)
                                (permutation-singleton x)
                                (0 1))))

    (inductive-step xs ih
      (let ((pivot (choose-pivot xs))
            (partitioned (partition ord pivot xs)))
        (match partitioned
          ((partition-result smaller equal larger smaller-proof larger-proof))
          ;; Apply induction hypothesis to smaller sublists
          (let ((sorted-smaller (ih smaller (smaller-length-proof smaller xs)))
                (sorted-larger (ih larger (larger-length-proof larger xs))))
            (let ((result (append sorted-smaller (cons pivot sorted-larger))))
              (witness result
                (triple (concatenation-sorted ord sorted-smaller pivot sorted-larger)
                      (partition-permutation-proof smaller equal larger xs pivot)
                      (partition-complexity-proof xs smaller larger))))))))))

;; Extract quicksort with guaranteed complexity
(define quicksort (extract quicksort-correct-and-efficient))

;; Verified matrix multiplication
(theorem matrix-multiply-correct ::
  (Pi (m n p : Natural) (A : Type) (ring : (Ring A))
    (-> (M1 : (Matrix m n A)) (M2 : (Matrix n p A))
      (result : (Matrix m p A))
      (= result (matrix-product ring M1 M2))))
(proof-by-triple-induction m n p
  (lambda (i j k)
    ;; result[i,j] = sum(M1[i,k] * M2[k,j] for k in 0..n-1)
    (sum-correctness-proof ring M1 M2 i j)))

;; Extract verified matrix multiplication
(define matrix-multiply (extract matrix-multiply-correct))

;; Crypto algorithm with security proof
(theorem rsa-correctness ::

```

```

(Pi (p q : Prime) (constraint :: (not (= p q)))
  (Let ((n (* p q))
        (phi (* (- p 1) (- q 1)))
        (e (choose-public-exponent phi))
        (d (modular-inverse e phi)))
    (Forall (message : (Range 0 n))
      (= message (mod (expt (mod (expt message e) n) d) n)))))
(proof
  ;; Proof using Euler's theorem and properties of modular arithmetic
  (euler-theorem-application p q phi e d)))

;; Extract RSA implementation with correctness guarantee
(define rsa-encrypt-decrypt (extract rsa-correctness))

;; Concurrent algorithm with safety proof
(theorem concurrent-counter-safe ::
  (Pi (initial : Natural) (threads : Natural)
    (Exists (final : Natural)
      (And (>= final initial)
        (Forall (execution : (ConcurrentExecution threads))
          (Data-Race-Free execution))))))
(proof
  ;; Proof using STM properties and atomicity guarantees
  (stm-atomicity-proof initial threads))

;; Extract safe concurrent counter
(define concurrent-counter (extract concurrent-counter-safe))

;; Usage example with proof checking
(define (demo-proof-carrying-code)
  ;; The extracted functions come with runtime proof checking
  (let ((unsorted-list '(3 1 4 1 5 9 2 6)))

    ;; Sorting with proof
    (let ((sorted-result (insertion-sort natural-order unsorted-list)))
      (display "Sorted: ") (display sorted-result) (newline)

      ;; The runtime system can verify the postcondition
      (assert-postcondition sorted-result
        (and (sorted? natural-order sorted-result)
          (permutation? unsorted-list sorted-result))))

    ;; Binary search with proof
    (let ((search-result (binary-search natural-order sorted-result 4)))
      (display "Search result: ") (display search-result) (newline)

      ;; Verify search postcondition
      (match search-result
        (('Just index)
          (assert-postcondition index
            (and (< index (length sorted-result))
              (= (list-ref sorted-result index) 4))))
        ('Nothing
          (assert-postcondition 'nothing
            (not (member 4 sorted-result)))))))

;; Proof certificate generation
(define (generate-proof-certificate program proof)
  (let ((certificate
    (make-proof-certificate
      (program-hash program)
      (proof-hash proof)
      (verification-timestamp)
      (proof-checker-version))))
    (sign-certificate certificate (get-private-key))
    certificate))

;; Proof certificate verification
(define (verify-proof-certificate program certificate)

```

```

(and (verify-certificate-signature certificate (get-public-key))
     (= (program-hash program) (certificate-program-hash certificate))
     (valid-proof? (certificate-proof-hash certificate))
     (trusted-checker? (certificate-checker-version certificate))))

;; Runtime proof checking
(define-syntax with-proof-checking
  (syntax-rules ()
    ((with-proof-checking computation)
     (with-exception-handler
      (lambda (e)
        (if (proof-violation? e)
            (begin
              (display "PROOF VIOLATION: ") (display e) (newline)
              (abort-execution))
            (raise e)))
      (lambda () computation)))))

;; Example of proof-carrying code deployment
(define (deploy-verified-program program proof certificate)
  (when (verify-proof-certificate program certificate)
    (display "Certificate verified - deploying program") (newline)
    (with-proof-checking
     (install-program program proof))))

```

These comprehensive examples demonstrate how Lambdust's advanced features work together in realistic programming scenarios, showing the practical benefits of gradual dependent typing, effect systems, concurrency, and proof-carrying code.

Appendix A. R7RS Compatibility Reference

This appendix provides a detailed mapping between R7RS-small features and their Lambdust implementations, ensuring complete backward compatibility.

A.0.1. Syntactic Compatibility

All R7RS-small syntactic forms are supported without modification:

- All special forms: `lambda`, `if`, `set!`, `quote`, etc.
- All derived expressions: `cond`, `case`, `and`, `or`, etc.
- All literal syntaxes: numbers, strings, characters, booleans, symbols
- All reader macros: `quote`, `quasiquote`, `unquote`, `unquote-splicing`

A.0.2. Semantic Compatibility

The denotational semantics preserves R7RS behavior:

- Evaluation order and proper tail recursion
- Lexical scoping and closure semantics
- Continuation semantics via `call/cc`
- Numeric tower and exactness

Appendix B. SRFI Support Matrix

This appendix lists SRFI (Scheme Request for Implementation) support in Lambdust.

SRFI	Status	Description
SRFI-1	Complete	List Library
SRFI-6	Complete	Basic String Ports
SRFI-8	Complete	receive: Binding to multiple values
SRFI-9	Complete	Defining Record Types
SRFI-11	Complete	Syntax for receiving multiple values
SRFI-13	Complete	String Libraries
SRFI-14	Complete	Character-set Library
SRFI-16	Complete	Syntax for procedures of variable arity
SRFI-17	Complete	Generalized set!
SRFI-23	Complete	Error reporting mechanism
SRFI-30	Complete	Nested Multi-line Comments
SRFI-39	Complete	Parameter objects
SRFI-41	Complete	Streams
SRFI-46	Complete	Basic syntax-rules Extensions
SRFI-62	Complete	S-expression comments
SRFI-87	Complete	=> in case clauses
SRFI-111	Complete	Boxes
SRFI-112	Complete	Environment Inquiry
SRFI-113	Complete	Sets and bags
SRFI-121	Complete	Generators
SRFI-128	Complete	Comparators (reduced)
SRFI-132	Complete	Sort Libraries
SRFI-151	Complete	Bitwise Operations
SRFI-158	Complete	Generators and Accumulators

Appendix C. Migration Guide from R7RS

This appendix provides guidance for migrating existing R7RS Scheme code to Lambdust.

C.0.1. Zero-Change Migration

Most R7RS programs run unchanged in Lambdust:

```
;; This R7RS program works unchanged in Lambdust
(define (factorial n)
  (if (<= n 1)
      1
      (* n (factorial (- n 1)))))
```

C.0.2. Gradual Enhancement

Add type annotations incrementally:

```
;; Add type annotation for better error checking
(define (factorial n)
  #:type (-> Natural Natural)
  (if (<= n 1)
      1
      (* n (factorial (- n 1)))))
```

Appendix D. Implementation Notes

This appendix provides guidance for implementers of the Lambdust language.

D.0.1. Parser Implementation

The parser should support:

- All R7RS lexical elements
- Extended token types: `::`, `→`, `⇒`, `|`, `~>`
- Keyword syntax: `#:identifier`
- Type annotation syntax

D.0.2. Type Checker Integration

The type checker operates in four modes:

1. **Dynamic Mode:** No type checking (R7RS compatible)
2. **Gradual Mode:** Optional type checking with contracts
3. **Static Mode:** Full Hindley-Milner inference
4. **Dependent Mode:** Dependent type checking with proof obligations

REFERENCES

- [1] Gordon D. Plotkin and John Power. Algebraic operations and generic effects. *Applied Categorical Structures*, 11(1):69–94, 2003.
- [2] Alex Shinn, John Cowan, and Arthur A. Gleckler. Revised⁷ report on the algorithmic language scheme. Technical report, Scheme Working Group, July 2013. Small language specification.

Index

The principal entry for each term, procedure, or keyword is listed first, separated from the other entries by a semicolon.